

Design & Analysis of Algorithms (10-12M)

Text-books:

- Introduction to Algorithms
 - CLR (Cormen)
- Fundamentals of Algorithms
 - Sahani

Syllabus:

1. Analysis of Algorithms

- Algorithm concept & lifecycle
- Need for analysis
- Methodology of analysis
- Types of analysis
- ** → Asymptotic Notations (AsN)
 - Definitions & properties
 - Problem solving
- * → Framework for analysing recursive & non-recursive algorithms.
- * → Analysis of Program segments with loops.
 - Space complexities

2. Design Strategies:

i) Divide & Conquer

- Control Abstraction
- Max Min
- Merge Sort
- Quick sort
- Binary Search
- Matrix Multiplication

→ long integer multiplication

→ Master's theorem

(ii) Greedy Method

*

→ Control Abstraction

→ Job seq. with deadlines

→ knapsack problem

→ Merge patterns

→ Huffman coding

→ Spanning trees

→ Shortest paths

(iii) Dynamic Programming

**

→ General method

→ DP vs GM v D&C

→ Multistage Graphs

→ 0/1 knapsack

→ All pairs shortest path

→ Travelling salesperson

→ Matrix chain Product

→ Longest common subsequence

→ Bellman Ford algorithm

→ OBST

→ Reliable System Design

→ kadane Algorithm

(iv) Graph Techniques

→ Traversal

→ Parenthesization theorem

→ Components; Articulation points.

(v) Heap algorithms

(vi) Sorting techniques

→ Classification

→ Case studies

Algorithm: Consists of finite set of steps/stmts to solve a given problem.

→ Each step/stmt may involve one/more fundamental operation.

→ Every operation must be

i) Definite (i.e., clear) (unambiguous)

ii) Effective (i.e., finite time)

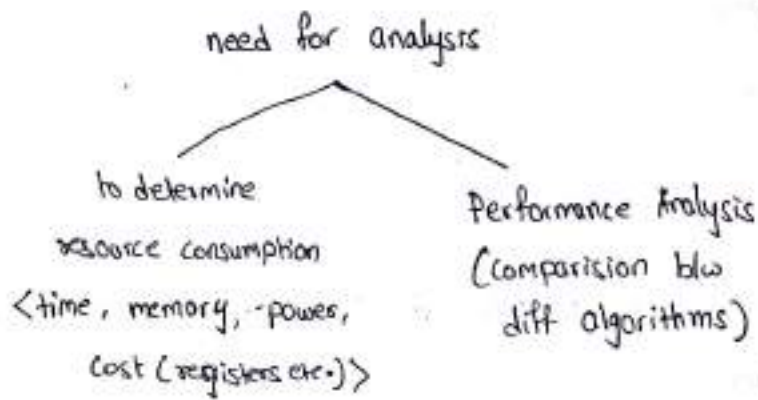
→ Every algorithm must halt in finite time.

→ Every algorithm may accept one or more inputs and must produce atleast one output.

Lifecycle steps:

1. Problem definition
2. Requirements.
3. Design [logic]
4. Develop the algorithm (Pseudocode)
5. Validation (Proving its correctness)
6. Analysis
7. Implementation
8. Testing & Debugging

Analysis



Methodology of analysis:

Aposteriori Analysis (Experimentation analysis)

→ This analysis done by converting algorithm into a program and we analyse by running it on a platform.

This analysis depends on the processors, memory speed, OS, programming language (compiler) i.e., platform dependent.

Advantages:

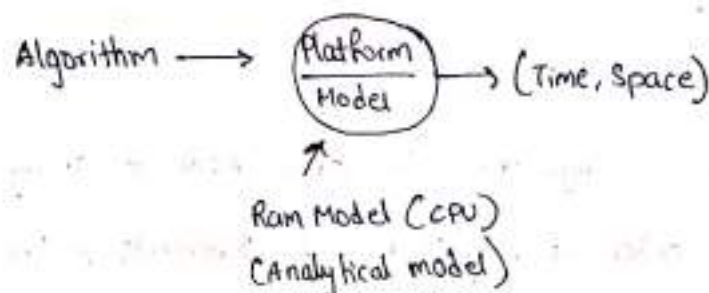
- * Gives exact values in units of time/space.

Drawback:

- * reports different values from platform to platform
- * difficult to make performance comparision.
- * Difficult to determine the exact times ($\because$ OS and other programs could run in parallel).

Here ~~are~~

Apriori Analysis: (Platform Independent Analysis)



This is a hypothetical model with infinite memory and we assume that it takes one unit of time for every fundamental operation.

→ This gives estimated time.

Adv:

→ This analysis do help in performance comparison.

→ Easy to carry out.

Components of analytic framework used in apriori analysis

→ A language for describing algorithm (Pseudocode)

→ A computational model (ram model) that algorithms execute within it.

→ A metric for measuring running time of algorithm.
i.e., basic / fundamental operation

→ An approach for characterizing running times of algorithms
(i.e., Asymptotic notation)

Algorithm Test

time-apriori mode (Step count method)

{	1. $x \leftarrow y + z$	2. $1 + (n+1) + n$
	2. for $i \leftarrow 1$ to n	$\begin{matrix} \nearrow \text{assignment} & \nearrow \text{comparisons} \\ 1 + (n+1) + n & \nearrow \text{increments} \end{matrix}$
	$x \leftarrow y + z$	$n + n$
	3. for $i \leftarrow 1$ to n	$1 + (n+1) + n$
	for $j \leftarrow 1$ to n	$n + n(n+1) + n \cdot n$
	$a \leftarrow b + c$;	$\frac{n \cdot n + n \cdot n}{2}$
}		2n $5 + 8n + 4n^2$

∴ time required is

$$T(n) = 4n^2 + 8n + 5$$

↓
input size

The running time of algorithm in apriori mode of analysis is determined by order of magnitude of statements of the algorithm.

Here, order of magnitude refers to the frequency of the fundamental operation in the statement.

Algorithm Test

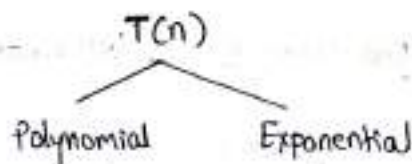
order of magnitude

```
{  
  1.  $x \leftarrow y + z$  ----- 1  
  2. for  $i \leftarrow 1$  to  $n$  } -----  $n$   
     $z \leftarrow y + z$   
  3. for  $i \leftarrow 1$  to  $n$  } -----  $n^2$   
    for  $j \leftarrow 1$  to  $n$   
       $a \leftarrow b + c$   
}
```

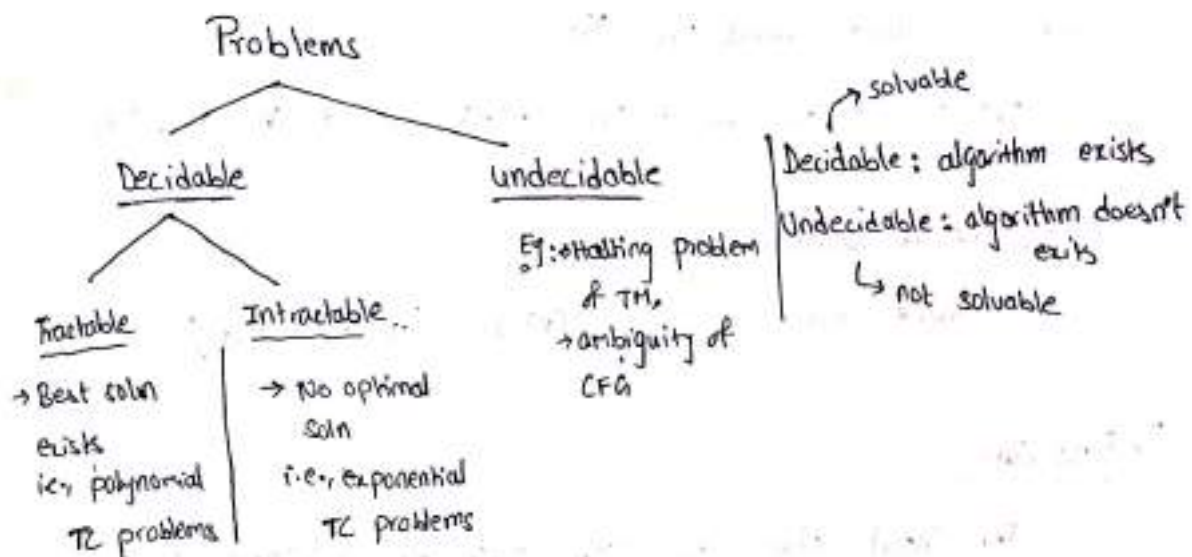
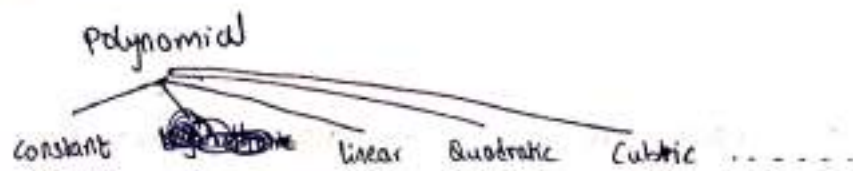
$T(n) = n^2 + n + 1$

The objective of Apriori analysis representing running time of algorithm is to represent the time of an algorithm as a mathematical function of input size.

↳ derived or expressed using step count (or) order of magnitude



→ Polynomial func have lesser rate of growth and exponential functions have faster rate of growth.



→ Tractable problems fall under the class of P (Polynomial)

→ NP Problems (Non-Deterministic Polynomial)

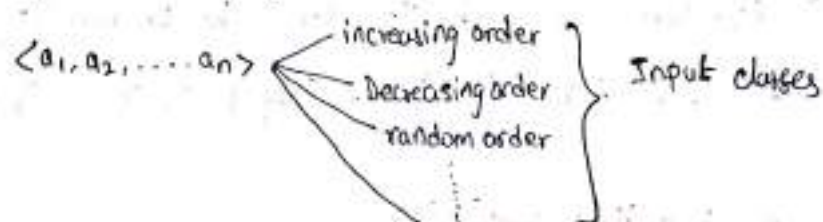
Those are problems that are solvable in polynomial time on non-deterministic TM.

However they take exponential time on deterministic TM.

→ Most of the intractable problems are NP.

Types of Analysis / Behaviour of algorithm:

→ Input of fixed size (say 'n')



Let no of input classes be k.

Based on this we broadly classify the input classes into 3 categories

(i) Worst-Case:

The input class for which algorithm does maximum work is called worst-case input. The corresponding time taken is called worst case time.

This is called, worst case behaviour of an algorithm.

Eg: linear search $O(n)$

Quick sort $O(n^2)$ (sorted input)

(ii) Best Case:

The input class for which algo does minimum work is best case & corresponding time is best case time.

Eg: Linear search $O(1)$

Quick Sort $O(n \log n)$

(iii) Average Case Analysis: (Not needed for gate)

This is carried out in 3 steps:

(i) Determine all the input classes.

$\langle I_1, I_2, I_3, \dots, I_k \rangle$ (say size 'n')

(ii) Determine the time taken by the algorithm for each input class. (say $t_1, t_2, \dots, t_k$)

(iii) Associate with each input class, the probability with which algo may take it from. (say $P_1, P_2, P_3, \dots, P_k$)

Avg case time complexity

$$A(n) = \sum_{i=1}^k t_i * P_i$$

Let $w(n)$, $B(n)$, $A(n)$ be worst case, best case, avg case times.

$$\Rightarrow B(n) \leq A(n) \leq w(n)$$

Note:

$\rightarrow B(n) = A(n) = w(n) \Rightarrow$ Uniform Behaviour

Eg: Merge Sort $\dots O(n \log n)$.

$\rightarrow [B(n) = A(n)] < w(n)$

Eg: Quick Sort $\dots B(n) = A(n) = O(n \log n)$
 $w(n) = O(n^2)$

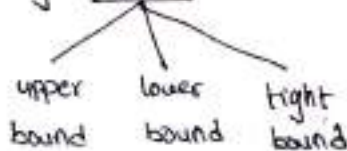
$\rightarrow B(n) < [A(n) = w(n)]$

Eg: linear search $\dots B(n) = O(1)$
 $A(n) = w(n) = O(n)$

$\rightarrow B(n) < A(n) < w(n)$

Asymptotic Notations (ASN):

Asymptotic notation is a mathematical tool which is used for representing bounds of a function.



ASN

Big Notations

$\rightarrow \text{Big-oh}(O)$

$\hookrightarrow$ upper bound

$\rightarrow \text{Big-omega}(\Omega)$

$\hookrightarrow$ lower bound

$\rightarrow \text{Theta}$ $\dots$ tight bound

Small/Little notations

$\rightarrow \text{Small-oh}(o) \dots$ Proper upper bound

$\rightarrow \text{Small-omega}(\omega) \dots$ Proper lower bound

Let 'f' & 'g' be functions from set of real numbers to real numbers.

(i) Big-Oh (O) : (upper bound)

$f(x)$ is $O(g(x))$

iff $f(x) \leq C \cdot g(x)$ for some $C > 0$ whenever $x > k$
(k is a constant)

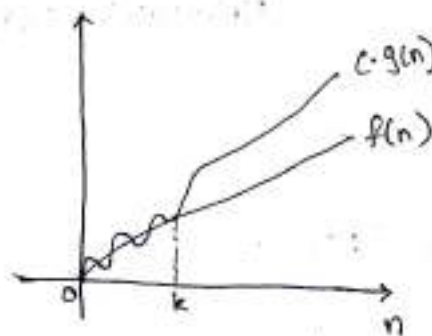
Ex: $f(n) = 1 + n + n^2 = O(n^2)$

$$1 + n + n^2 < n^2 + n^2 + n^2$$

$$f(n) < 3 \cdot n^2, \quad n > 1$$

$\downarrow \quad \downarrow$
 $C \quad g(n)$

$$\therefore f(n) = O(n^2)$$



$$\text{also } f(n) = 1 + n + n^2 < 3n^3$$

$$\Rightarrow f(n) = O(n^3)$$

$$\therefore f(n) \text{ is } O(n^2), O(n^3), O(n^4) \dots$$

$\therefore$ A function can have infinite number of upper bounds

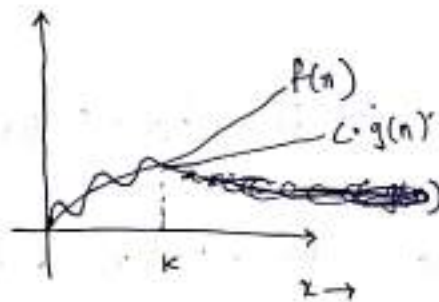
By default we consider the upper bound that is ~~an~~ asymptotically closest to the given function.

$\therefore f(n) = O(n^2)$ is more appropriate.

(ii) Big-omega (Ω): (Lower bound of a function)

$f(x)$ is called $\Omega(g(x))$

iff $f(x) \geq c \cdot g(x)$ for some $c > 0$ &
whenever $x > k$



$$f(n) = 1 + n + n^2 > n^2$$

$$\Rightarrow f(n) = \Omega(n^2)$$

$$\therefore f(n) = 1 + n + n^2 > 1 \Rightarrow f(n) = \Omega(1)$$

$$f(n) = 1 + n + n^2 > n \Rightarrow f(n) = \Omega(n)$$

$\therefore$ A function can have several lower bounds

$\therefore \Omega(n^2)$ is asymptotically closer to $f(n)$

$\therefore f(n) = \Omega(n^2)$ is more appropriate.

$$\text{eg: } f(n) = n \begin{cases} O(n) \\ \Omega(n) \end{cases}$$

$$f(n) = 2^{100} \begin{cases} O(1) \\ \Omega(1) \end{cases}$$

$$f(n) = n + \log n \begin{cases} O(n) \\ \Omega(n) \end{cases} \dots \because n + \log n > n$$

(iii) Theta(θ) : (Tight bound)

$f(x)$ is ~~not~~ equal to $\theta(g(x))$

$$\text{iff } c_1 \cdot g(x) \leq f(x) \leq c_2 \cdot g(x) \quad c_1 > 0 \wedge c_2 > 0 \\ \text{whenever } x > k$$

In other words

$$f(x) = \theta(g(x)) \text{ iff } f(x) = O(g(x)) \wedge f(x) = \Omega(g(x))$$

$\therefore$ In the cases where $f(x) = O(g(x))$ & $f(x) = \Omega(g(x))$ it is more appropriate representation would be

$$f(x) = \theta(g(x))$$

Eg:

$$f(n) = 10^{10} \begin{cases} O(1) \\ \Omega(1) \end{cases} \therefore \theta(1)$$

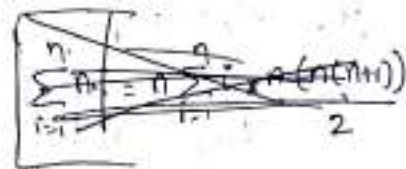
$$f(n) = n^2 + n \log n + 2^{100} \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases} \therefore \theta(n^2)$$

$$f(n) = \sum_{i=1}^n 1 \begin{cases} O(n) \\ \Omega(n) \end{cases} \therefore \theta(n)$$

$$\therefore \sum_{i=1}^n 1 = n$$

$$f(n) = \sum_{i=1}^{100} 1 \begin{cases} O(1) \\ \Omega(1) \end{cases} \therefore \theta(1)$$

$$f(n) = \sum_{i=1}^n n \cdot 1 \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases}$$



$$f(n) = \sum_{i=1}^n i \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases}$$

$$f(n) = \sum_{i=1}^n O(n) \begin{cases} O(n^2) \\ \Omega(n^2) \end{cases}$$

$$\therefore \sum_{i=1}^n O(n) = O(n) \sum_{i=1}^n 1 = O(n^2)$$

$$f(n) = \sum_{i=1}^{100} O(n) \begin{cases} O(n) \\ \Omega(n) \end{cases}$$

$$\rightarrow f(n) = \sum_{i=1}^n i^2 < O(n^3) \\ \text{and } \Omega(n^3)$$

$$\rightarrow f(n) = \sum_{i=1}^n i^3 < O(n^4) \\ \text{and } \Omega(n^4)$$

$$\rightarrow f(n) = \sum_{i=1}^n \frac{1}{i} < O(\log n) \\ \text{and } \Omega(\log n)$$

$$\therefore \sum_{i=1}^n \frac{1}{i} = \int_1^n \frac{1}{x} dx$$

$$= (\log x)_1^n = \log n$$

$$\rightarrow f(n) = \sum_{i=1}^n 2^i$$

$$= 2^1 + 2^2 + \dots + 2^n$$

$$= 1 + 2 + 2^2 + \dots + 2^n - 1$$

$$= \frac{2^{n+1} - 1}{2 - 1} - 1$$

$$= 2^{n+1} - 1$$

$$\leq 2^2 \cdot 2^n$$

$$\leq 2^2 \cdot 2^n$$

$$= O(2^n)$$

$$\therefore f(n) \text{ is } O(2^n)$$

$$f(n) \text{ is } \Omega(2^n)$$

$$\rightarrow f(n) = \sum_{i=1}^n i \cdot 2^i$$

$$f(n) = 1 \cdot 2 + 2 \cdot 2^2 + 3 \cdot 2^3 + \dots + n \cdot 2^n$$

$$2 \cdot f(n) = 1 \cdot 2^2 + 2 \cdot 2^3 + 3 \cdot 2^4 + \dots + n \cdot 2^{n+1}$$

$$2f(n) - f(n) = -2 - 2^2 - 2^3 - \dots - 2^n + n \cdot 2^{n+1}$$

Note:

$$\sum_{i=1}^n O(1) \approx \text{for } i=1 \text{ to } n \\ a = a + b$$

$$\sum_{i=1}^n O(n) \approx \sum_{i=1}^n \sum_{j=1}^n O(1)$$

$$\approx \text{for } i=1 \text{ to } n \\ \text{for } j=1 \text{ to } n \\ a = a + b$$

$$\Rightarrow f(n) = 1 - (1 + 2 + 2^2 + \dots + 2^n) + n \cdot 2^{n+1}$$

$$f(n) = 1 + n \cdot 2^{n+1} - \frac{2^{n+1} - 1}{2 - 1}$$

$$f(n) = 2 + n \cdot 2^{n+1} - 2^{n+1} = \underline{(n-1)2^{n+1} + 2}$$

$$\Rightarrow f(n) = O(n \cdot 2^n)$$

$$f(n) = \Omega(n \cdot 2^n)$$

$$\sum_{i=1}^n i \cdot 2^i = (n-1)2^{n+1} + 2$$

$$* \rightarrow f(n) = \sum_{i=1}^n \frac{1}{2^i}$$

$$= \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \dots + \frac{1}{2^n}$$

$$= \frac{2^{n-1} + 2^{n-2} + \dots + 2^2 + 2^1 + 1}{2^n}$$

$$= \frac{2^n - 1}{2^n} = 1 - \frac{1}{2^n}$$

$$\therefore f(n) = O(1)$$

$$\therefore 1 - \frac{1}{2^n} \leq 1$$

$$\rightarrow f(x) = \sum_{i=0}^{\infty} x^i \text{ where } x < 1$$

$$\Rightarrow f(x) = \frac{1}{1-x}$$

$$\therefore f(x) = O(1)$$

$$(\because x < 1 \Rightarrow 1-x > 0 \Rightarrow \frac{1}{1-x} < 1)$$

$$(\because x \text{ is a constant})$$

$$\rightarrow f(n) = \sum_{i=j}^n a^i = \frac{a^{n+1} - a^j}{a-1}$$

$$\Rightarrow f(n) = O(a^n)$$

$$\rightarrow f(n) = \prod_{i=1}^n 1 = O(1)$$

$$\rightarrow f(n) = \prod_{i=1}^n i$$

$$\Rightarrow f(n) = n!$$

$$= 1 \cdot 2 \cdot 3 \cdot \dots \cdot n < n \cdot n \cdot n \cdot \dots \cdot n \quad (n \text{ times})$$

$$\Rightarrow f(n) < n^n$$

$$\therefore f(n) = n! = O(n^n)$$

However

$$f(n) \neq \Omega(n^n)$$

$$\rightarrow f(n) = \log n!$$

$$\log n! < \log n^n$$

$$\Rightarrow \log n! < n \log n$$

$$\Rightarrow \log n! = O(n \log n)$$

$$\rightarrow f(n) = \sum_{i=1}^n \log i$$

$$= \log 1 + \log 2 + \dots + \log n$$

$$= \log n!$$

$$= O(n \log n)$$

$$\text{However } f(n) \neq \Omega(n \log n)$$

$$\rightarrow f(n) = (\log n)!$$

$$< (\log n)^{(\log n)}$$

$$\Rightarrow f(n) = O((\log n)^{\log n})$$

$$\text{also } (\log_e n)^{\log_e n} = n^{\log_e (\log_e n)}$$

$$\Rightarrow f(n) = O(n^{\log(\log n)})$$

$$(\because a^{\log_c b} = b^{\log_c a})$$

$$\begin{aligned} n! &= \Omega(n) \\ &= \Omega(n^2) \\ &= \Omega(2^n) \\ &= \Omega(n^n) \end{aligned}$$

$$\begin{aligned} f(n) &= n! = O(n^n) \\ n! &\neq \Omega(n^n) \\ n! &= \Omega(n) \\ &= \Omega(n^2) \\ &= \Omega(2^n) = \Omega(3^n) \\ &= \Omega(n^n) \end{aligned}$$

$$a^{\log_c b} = b^{\log_c a}$$

$$\begin{aligned} \sum_{i=1}^n \sqrt{i} &= \sqrt{1} + \sqrt{2} + \dots + \sqrt{n} \\ &\approx \int_1^n \sqrt{x} \, dx \\ &= \frac{2}{3} x^{3/2} \Big|_1^n \\ &= \frac{2}{3} n^{3/2} \\ \therefore O(n^{3/2}) \end{aligned}$$

→ Smaller functions are in the order of bigger functions

$$\text{i.e., } f(x) = O(g(x))$$

smaller function bigger function

→ Bigger functions are in the order of smaller functions.

$$f(x) = \Omega(g(x))$$

bigger function smaller function

Ex: if $f(x) = O(g(x))$
then $g(x) = \Omega(f(x))$

Ex: Consider $f(n) = n^2$ & $g(n) = n \log n$

$$\Rightarrow f(n) = \Omega(g(n))$$

&

$$g(n) = O(f(n))$$

Small / Little Notations:

The bounds provided by big~~o~~ notations (O, Ω) may or may not be asymptotically tight.

Ex: $f(n) = n = O(n)$ is a tight upper bound

whereas $O(n^2)$ is a loose upper bound

slly $f(n) = n = \Omega(n)$ is a tight lower bound

$f(n) = n = \Omega(\log n)$ is a loose lower bound.

However the bound provided by small notations are always loose.

(iv) Small-oh (o) : (Proper upper bound)

$f(x)$ is $o(g(x))$

iff $f(x) < c \cdot g(x)$, for all $c > 0$ & $x > k$



$$f: f(n) = n^2 = O(n^2)$$

$$\neq o(n^2)$$

$$f(n) = n^2 = o(n^3) \\ = o(n^4)$$

$$\because n^2 < c \cdot n^3, \forall c > 0, n > 1$$

(v) Small-omega (ω) : (Proper lower bound)

$f(x)$ is $\omega(g(x))$

iff $f(x) > c \cdot g(x)$, $\forall c > 0$ & $x > k$

$$f(n) = n^2$$

$$\Rightarrow f(n) = \Omega(n^2), \Omega(n \log n), \Omega(n), \dots$$

$$f(n) = \omega(n \log n), \omega(n), \dots$$

Analogy b/w ASNs & real numbers:

$\rightarrow$ Let f & g be functions ; a & b be real numbers.

$$f(n) \text{ is } O(g(n)) \iff a \leq b$$

$$f(n) \text{ is } \Omega(g(n)) \iff a \geq b$$

$$f(n) \text{ is } \Theta(g(n)) \iff a = b$$

$$f(n) \text{ is } o(g(n)) \iff a < b$$

$$f(n) \text{ is } \omega(g(n)) \iff a > b$$

Note:

$$\rightarrow f(n) = o(g(n)) \Rightarrow f(n) = O(g(n))$$

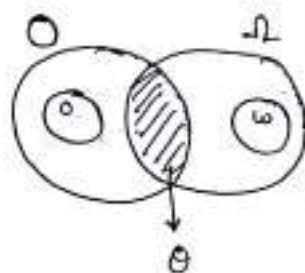
reverse need not to be true.

$$\Rightarrow o \subseteq O$$

$$\rightarrow f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$$

reverse need not to be true

$$\Rightarrow \omega \subseteq \Omega$$



Properties of Asymptotic Notations:

i) Discrete Properties:

	Big-Oh	Big-Omega	Theta	Small-oh	Small-Omega
Reflexive	✓	✓	✓	X	X
Symmetric	X	X	✓	X	X
Transitive	✓	✓	✓	✓	✓
Transpose Symmetry	$f(n)$ is $O(g(n))$ iff $g(n)$ is $\Omega(f(n))$		—	$f(n)$ is $o(g(n))$ if $g(n)$ is $\omega(f(n))$	

Reflexive:

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

$$f(n) = \Theta(f(n))$$

$$f(n) \neq o(f(n))$$

$$f(n) \neq \omega(f(n))$$

Symmetric:

If $f(n) = O(g(n))$ then it not necessary that $g(n) = O(f(n))$

$\therefore$ Big-oh is not symmetric

Silly Ω is not symmetric

Transitivity:

$$f(n) = O(g(n)) \text{ \& } g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$$

$\therefore$ ~~transitive~~ transitive
silly for Ω & Θ theorems, o , ω

ii) General Properties:

$\rightarrow$ if $f(n) = O(g(n))$ then

$$a \cdot f(n) = O(g(n))$$

where a is a constant.

$\rightarrow$ If $f(n) = O(g(n))$ and $d(n) = O(e(n))$ then

$$i) f(n) + d(n) = O(\max(g(n), e(n)))$$

$$ii) f(n) \cdot d(n) = O(g(n) \cdot e(n))$$

$\rightarrow \Rightarrow \log n^c = O(\log n)$ where c is constant

$$* \rightarrow (\log n)^x = O(n^y) \quad (x > 0, y > 0)$$

Proof:

$$(\log n)^x \text{ vs } n^y$$

log on both side

$$\log(\log n)^x \quad \log n^y$$

$$x \log(\log n) \quad y \log n$$

$$* x \log(\log n) < y \log n \quad (\because x, y \text{ are constants})$$

$$\therefore \text{of } (\log n)^x$$

$$(\log n)^x = O(n^y)$$

$\rightarrow n^2$ is $O(a^n)$, $a > 1$

(x & a are constants)

Dominance Relation:

Constants $\prec$ logarithmic $\prec$ polynomial $\prec$ Exponential
 $\prec \dots$ Asymptotically lesser

Eg: Which of the below is asymptotically larger.

$$f = \sqrt{n} \quad g = \log n$$

Sol:

applying log

$$\log(f) = \log \sqrt{n} = \frac{1}{2} \log n$$

$$\log(g) = \log(\log n)$$

$$\Rightarrow \log(f) > \log(g)$$

$\Rightarrow f$ is asymptotically larger than g

$$\Rightarrow g = O(f)$$

Tricotomy Property:

tricotomy property of real numbers:

For any two real numbers a & b one of the below 3 conditions must hold.

(i) $a > b$

(ii) $a < b$

(iii) $a = b$

However, for any two functions it is not needed that they satisfy tricotomy property

Eg: $f(n) = n$; $g(n) = n^{1+\sin n}$

f & g here doesn't fall under any case

$$(f \not\prec g, f \not\succ g, f \not\sim g)$$

Q1) Consider the equality $\sum_{i=1}^n i^3 = x$ & the following choices for x

- a) $\theta(n^4)$ b) $\theta(n^5)$ c) $o(n^5)$ d) $\Omega(n^3)$

Sol:

$$\sum_{i=1}^n i^3 = x = \frac{n^2(n+1)^2}{4} = \frac{n^4 + 2n^3 + n^2}{4}$$

$$= O(n^4)$$

(or)

$$\Omega(n^4)$$

(or)

$$\theta(n^4)$$

$$\therefore \text{opt (a) + (b) + (d)}$$

(c)

Q2) Let $w(n)$ & $A(n)$ represent worst case and average case time of an algorithm of input size 'n'. which is always true?

- a) $A(n) = O(w(n))$ b) $A(n) = \Omega(w(n))$
c) $A(n) = \theta(w(n))$ d) $A(n) = o(w(n))$

Sol:

$w(n)$ could be sometime equal to $A(n)$

$\therefore$ opt (b) is not possible.

$\therefore$ opt (a)

Ex: Check the correctness of below relations

i) $w(n \log n) = O(n \log n)$ --- True

ii) $2^{n+1} = O(2^n)$ --- True.

iii) $2^{2n} = O(2^n)$ --- False

$\therefore 2^{2n} = (2^2)^n = 4^n > 2^n \therefore \text{False}$

Note:

$$\rightarrow \log_b a = \frac{1}{\log_a b}$$

$$\rightarrow \log_b a = \frac{\log_x a}{\log_x b}$$

(iv) $(n+k)^m \neq O(n^m)$ False.
 $(k, m) > 0$

(v) $\sqrt{\log n} = O(\log \log n)$ ~~True~~ False

$$\sqrt{\log n} \quad \log \log n$$

$$\frac{1}{2} (\log \log n) \quad \log(\log \log n)$$

$$\Rightarrow \sqrt{\log n} \gg \log(\log n)$$

$$\therefore \sqrt{\log n} \neq O(\log \log n)$$

(vi) $\log n = \Omega(1/n)$ True

^{**}_{*} (vii) $2^{n^2} = O(n!)$ False

if $2^{n^2} = O(n!)$ then

$2^{n^2} = O(n^n)$ must be true too

$$\Rightarrow 2^{n^2} < n^n$$

$$\log 2^{n^2} < \log n^n$$

$$n^2 < n \log n$$

which is false

(viii) $n^2 = O(2^{2 \log n})$ True

n^2	$2^{2 \log n}$
$\log n^2$	$\log 2^{2 \log n}$
$2 \log n$	$2 \log n (\log 2)$

(or)

$$2^{2 \log n} = 2^{\log_2 n^2} = n^2 (\log_2 2) = n^2$$

$\therefore$ true

(ix) $a^n \neq O(n^x)$ $a > 1, x > 0$ True

79

29/09/20

Asymptotic Comparisons

(ii)

$$f(n) = n^2 \log n, \quad g(n) = n \log^{10} n$$

$$\log^2 n = \log(\log n)$$

$$\log^2 n = (\log n)^2$$

$$\log(f(n)) = 2 \log n + \log(\log n)$$

$$\log(g(n)) = \log n + \log^{10} n$$

$$\log(f(n)) > \log(g(n))$$

$$\Rightarrow f(n) = \Omega(g(n))$$

$$g(n) = O(f(n))$$

take common factors aside

$$f(n) = (n \log n) (n), \quad g(n) = (n \log n) (\log n)^9$$

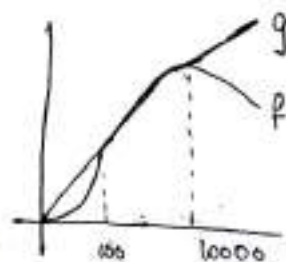
So we compare

$$f(n) = n, \quad g(n) = (\log n)^9$$

$$\Rightarrow g(n) = O(f(n))$$

$$(iii) \quad f(n) = \begin{cases} n^3, & 0 < n \leq 10000 \\ n, & n > 10000 \end{cases}$$

$$g(n) = \begin{cases} n, & 0 \leq n \leq 100 \\ n^3, & n > 100 \end{cases}$$



$$\therefore f(n) = O(g(n)) \quad \forall n \geq 100$$

$$\text{also } f(n) = o(g(n)) \quad \forall n \geq 10000$$

(H1/03) $10/\log n < 0.0001 n^2$

$$10 \log n < 10^{-4} n$$

$$\log n < 10^{-5} n$$

let $n = 10^x$

$$\log_{10} 10^x < 10^{-5} 10^x$$

$$x < 10^{x-5}$$

put $x=1 \Rightarrow 1 < 10^{-4} \times$

$x=2 \Rightarrow 2 < 10^{-3} \times$

$\Rightarrow x=5 \Rightarrow 5 < 10^0 \times$

$\Rightarrow x=6 \Rightarrow 6 < 10^1 \checkmark$

$\therefore$ Min value of $x=6$

(H4/04) $1, n^2; n \log n; n\sqrt{n}; e^n; n; 2^n; 1/n$

sol:

compare $n\sqrt{n}$ & $n \log n$

$$n\sqrt{n} \geq n \log n$$

$$\Rightarrow n\sqrt{n} \geq n \log n$$

~~$\left(\frac{1}{n} = O(1)\right)$~~

$$\therefore \frac{1}{n} < n < n \log n < n\sqrt{n} < n^2 < 2^n < e^n$$

(ii) $2^n; n^{3/2}; n \log n; n^{\log n}$

sol:

2^n is exponential

~~$n^{3/2}, n \log n$~~ $n^{3/2}, n \log n, n^{\log n}$ are polynomial

$$n^{3/2} > n \log n$$

$$n \log n \text{ \& } n^{\log n}$$

log on both sides

$\log(n \log n)$	$\log(n^{\log n})$	$\log(n^{3/2})$
$\log n + \log(\log n)$	$(\log n)(\log n)$	$\frac{3}{2} \log n$
$O(\log n)$	$O((\log n)^2)$	

$$\therefore n^{\log n} > n \log n \text{ \& } n^{\log n} > n^{3/2}$$

$$\therefore n \log n < n^{\log n}$$

$$n \log n < n^{3/2} < n^{\log n} < 2^n$$

(iii) $n^{1/3}; e^n; n^{3/4}; n \log^9 n; 1.001^n$

exponential

$n^{3/4}$	$n \log^9 n$
$\log(n^{3/4})$	$\log(n \log^9 n)$
$\frac{3}{4} \log n$	$\log n + 9 \log(\log n)$
$O(\log n)$	$O(\log n)$

Now we cancel out common terms & compare

$n^{3/4}$	$n \log^9 n$	
$\log(n^{3/4})$	$\log(\log^9 n)$	} $\Rightarrow n^{3/4} > n \log^9 n$
$\frac{3}{4} \log n$	$9 \log(\log n)$	

$$\Rightarrow n^{1/3} < n \log n < n^{3/4} < 1.001^n < e^n$$

H1/06

$$\begin{array}{l} \text{---} g(n) \\ \text{---} f(n) \end{array} \quad \left. \begin{array}{l} f(n) = O(g(n)) \\ g(n) \neq O(f(n)) \end{array} \right\} \Rightarrow g(n) \text{ is loose upper bound of } f(n)$$

$$\begin{array}{l} g(n) = O(h(n)) \\ h(n) = O(g(n)) \end{array} \left. \right\} \Rightarrow g(n) \text{ is tight upper bound of } h(n) \text{ and vice versa i.e., } g = \Theta(h)$$

$$\therefore f < (g=h)$$

$$\therefore f(n) = O(h(n)) \quad \checkmark \quad (\text{transitive property})$$

$$f(n) + h(n) = O(g + h) \quad \checkmark$$

$$h(n) \neq O(f(n)) \quad \checkmark$$

$$f(n) \cdot g(n) \neq O(g(n) \cdot h(n)) \quad \times$$

Framework for analysis of non-recursive & recursive algorithm

↓
general problems without design strategies.

no. of comparisons

H1/09

Naive/Brute Force

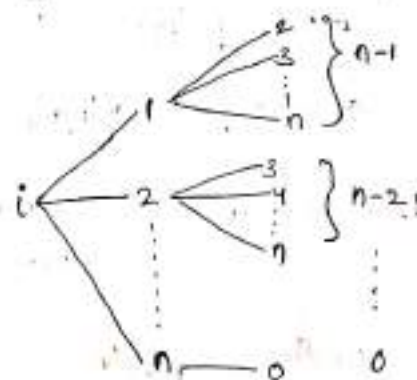
Algo Leaders (A.N)

```

{
  for i=1 to n
  {
    for j=i+1 to n
    {
      if (A[i] < A[j])
        break;
    }
    if (j=n+1)
      print(A[i]);
  }
}

```

$$TC: O(n^2)$$



$$\begin{aligned} T(n) &= 0 + 1 + 2 + \dots + n-1 \\ &= \frac{n(n-1)}{2} \end{aligned}$$

$$\Rightarrow T(n) = O(n^2)$$

Optimized algorithm:

```

Algo LEADER (A, n)
{
    L ← A[n];
    for i ← n-1 down to 1
    {
        if (A[i] > L)
        {
            print(A[i]);
            L ← A[i];
        }
    }
}
TC: O(n)

```

∴ Time complexity of efficient algorithm = $O(n)$

(H1/08) Let no of levels = k

∴ height = k

$$1+2+3+\dots+k = \text{no of nodes} = n$$

$$\frac{k(k+1)}{2} = n$$

$$k^2 + k - 2n = 0$$

$$k = \frac{-1 \pm \sqrt{1+8n}}{2}$$

$$\Rightarrow k = O(\sqrt{n})$$

(H1/09)

n MultiQueue operations

$$\Rightarrow O(1) + O(1) + \dots + O(1) \quad (n \text{ times})$$

$$\therefore O(n)$$

Assuming Queue is not initially empty

(H/10)

Out of linear & circular queue circular is more efficient.

24

and in circular insertion & deletion can be done in $O(1)$ time

$\therefore$ opt@

Calculate TC for below algorithm

(H/10)

Algorithm what(n)

```
{
  if(n==1) return;
  else
  {
    what(n-1);
    B(n)
  }
}
```

a) Assume TC of $B(n) = O(1)$:

TC of recursive algorithm is computed by deriving a recurrence relation.

Let $T(n)$ represent TC of what(n)

$$T(n) = \begin{cases} c, & n=1 \\ a + T(n-1) + b, & n>1 \end{cases}$$

$\downarrow$ time for comparison $\downarrow$ $B(n)$

$$T(n) = \begin{cases} c, & n=1 \\ d + T(n-1), & n>1 \end{cases}$$

$$T(n) = T(n-1) + d \quad \text{--- (1)}$$

$$T(n-1) = T(n-2) + d \quad \text{--- (2)}$$

$$\textcircled{2} \text{ in } \textcircled{1} \Rightarrow T(n) = T(n-2) + 2d$$

$$= T(n-3) + 3d$$

$$T(n) = T(n-k) + kd$$

To terminate the condition

$$n-k=1$$

$$\Rightarrow k=n-1$$

$$T(n) = T(1) + (n-1)d$$

$$T(n) = c + dn - d$$

$$\left. \begin{array}{l} T(n) = O(n) \\ T(n) = \Omega(n) \end{array} \right\} \Theta(n)$$

b) Assume TC of $B(n) = O(n)$

$$T(n) = \begin{cases} c, & n=1 \\ a + T(n-1) + n, & n>1 \end{cases}$$

$$T(n) = a + T(n-1) + n$$

$$= a + [a + T(n-2) + n] + n$$

$$= 2a + T(n-2) + 2n$$

$\vdots$

$$= ka + T(n-k) + k(n + (n-1) + (n-2) + \dots + (n-(k-1)))$$

$$n-k=1 \Rightarrow k=n-1$$

$$T(n) = (n-1)a + T(1) + (n-1)n$$

$$= (n-1)a + c + \frac{n(n-1)}{2}$$

$$= \frac{n^2}{2} - \frac{n}{2} + a + c$$

$$T(n) = \frac{n^2}{2} - \frac{n}{2} + a + c$$

$$\therefore T(n) = O(n^2)$$

c) Assume TC of $B(n) = O(1/n)$

$$T(n) = \begin{cases} c, & n=1 \\ a + T(n-1) + \frac{1}{n}, & n>1 \end{cases}$$

$$T(n) = a + T(n-1) + \frac{1}{n}$$

$$= a + \left(a + T(n-2) + \frac{1}{n-1} \right) + \frac{1}{n}$$

$$= 2a + T(n-2) + \frac{1}{n} + \frac{1}{n-1}$$

$$= 2a + \left(T(n-3) + a + \frac{1}{n-2} \right) + \frac{1}{n} + \frac{1}{n-1}$$

$$= 3a + T(n-3) + \left(\frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} \right)$$

⋮

$$= ka + T(n-k) + \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \dots + \frac{1}{n-(k-1)}$$

$$n-k=1 \Rightarrow k=n-1$$

$$= (n-1)a + T(1) + \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \dots + \frac{1}{2}$$

$$= (n-1)a + c + \underbrace{1 + \frac{1}{2} + \dots + \frac{1}{n}}_{\log n - 1}$$

$$T(n) = (n-1)a + c + \log n - 1$$

$$\Rightarrow T(n) = O(n)$$

Note:

→ If $T(n) = a + T(n-1) + \frac{1}{n}$, then $T(n) = O(n)$ } So constant 'a' also matters.

→ If $T(n) = a + T(n-1) + \frac{1}{n}$, then $T(n) = O(\log n)$

(H/n)

$$T(n) = \begin{cases} c, & n \leq 2 \\ a + T(\sqrt{n}), & n > 2 \end{cases}$$

$$T(n) = a + T(n^{1/2})$$

$$= a + \left(a + T(n^{1/4}) \right)$$

$$= 2a + T(n^{1/4})$$

$$= 2a + T(n^{1/2^{(2)}})$$

$$= T(n^{1/2^k}) + ka$$

$$= T(2) + \left(\frac{1}{2} \log n \right) a$$

$$\Rightarrow \therefore T(n) = O(\log n)$$

$$n^{1/2^k} = 2$$

$$\frac{1}{2^k} = \log_2 n$$

$$2^k = \frac{1}{\log_2 n} \Rightarrow 2^k = \log_2 n$$

$$k = \frac{\log n}{2}$$

(H/12)

$$T(n) = \begin{cases} c, & n=2 \\ a + T(\sqrt{n}), & n>2 \end{cases}$$

$$T(n) = T(n^{1/2}) + a$$

$$T(n) = T(n^{1/4}) + 2a$$

$$T(n) = T((n^{1/4})^{1/2}) + 3a$$

$$T(n) = T(n^{1/8}) + 3a$$

$$T(n) = T(n^{1/2^k}) + ka$$

$$T(n) = T(2) + a \cdot \log_2 \log_2 n$$

$$T(n) = c + a \cdot \log_2 \log_2 n$$

$$\Rightarrow T(n) = O(\log_2 \log_2 n)$$

$$n^{1/2^k} = 2$$

$$\frac{1}{2^k} = \log_n 2$$

$$2^k = \log_2 n$$

$$\Rightarrow k = \log_2 (\log_2 n)$$

Ex: Find TC for below algorithm.

Algo A(n)

{ if (n==2) return

else

return A(√n) + n;

}

$$T(n) = \begin{cases} c, & n=2 \\ a + T(\sqrt{n}), & n>2 \end{cases}$$

$$\Rightarrow T(n) = O(\log_2 \log_2 n)$$

(4/14)

$$T(n) = \begin{cases} c, & n=2 \\ a + 2T(\sqrt{n}), & n>2 \end{cases}$$

$$T(n) = 2T(n^{1/2}) + a$$

$$= 2[2T(n^{1/2^2}) + a] + a$$

$$= 2^2 T(n^{1/4}) + 2a + a$$

$$= 2^3 T(n^{1/8}) + 3a + 2a + a \quad n^{1/2^k} = 2$$

$$= 2^k T(n^{1/2^k}) + k a$$

$$\log n^{1/2^k} = 2 \log_2 2$$

$$\frac{1}{2^k} \log_2 n = 1$$

$$2^k = \log_2 n$$

$$k = \log \log n$$

$$= \frac{\log(\log n)}{2} T(2) + a \left(\frac{\log(\log n)}{2} \right)$$

$$T(n) = \frac{\log n}{2} + a \left(\frac{\log \log n}{2} \right)$$

$$\Rightarrow T(n) = O(\log n)$$

$$= 2^k T(n^{1/2^k}) + a(2^k - 1)$$

$$= 2^{\log \log n} T(2) + a(2^{\log \log n} - 1)$$

$$= c \cdot \log n + a \cdot \log n - a$$

$$\therefore T(n) = O(\log n)$$

(4/13)

$$T(n) = \begin{cases} c, & n=1 \\ 2T(n-1) + a, & n>1 \end{cases}$$

$$T(n) = 2T(n-1) + a$$

$$= 2[2T(n-2) + a] + a$$

$$= 2^2 T(n-2) + 2a + a$$

$$= 2^3 [2T(n-3) + a] + 2a + a$$

$$= 2^3 \cdot T(n-3) + 2^2 a + 2a + a$$

$$= 2^k T(n-k) + (2^{k-1} + 2^{k-2} + \dots + 2 + 1) a$$

$$= 2^k T(n-k) + (2^k - 1) a$$

$$n-k=1$$

$$\Rightarrow k=n-1$$

$$= 2^{n-1} T(1) + (2^{n-1} - 1) a$$

$$= c \cdot 2^{n-1} + a \cdot 2^{n-1} - a$$

$$= (a+c) 2^{n-1} - a$$

$$\therefore T(n) = O(2^n)$$

H/15

i) Assume $B(n)$ is ~~$O(1)$~~

$$T(n) = \begin{cases} c, & n=1 \\ 2T\left(\frac{n}{2}\right) + a + 1, & n>1 \end{cases}$$

comparision $B(n)$

$$T(n) = 2T\left(\frac{n}{2}\right) + a \quad (\because a+1 \text{ is const})$$

$$= 2 \left[2T\left(\frac{n}{4}\right) + a \right] + a$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2a + a$$

$$= 2^2 \left[2T\left(\frac{n}{2^3}\right) + a \right] + 2a + a$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 2^2 a + 2a + a$$

$$= 2^k T\left(\frac{n}{2^k}\right) + a(2^{k-1} + 2^{k-2} + \dots + 2 + 1)$$

$$= 2^k T\left(\frac{n}{2^k}\right) + (2^k - 1) a$$

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$\Rightarrow k = \log n$$

$$= 2^{\log n} T(1) + (2^{\log n} - 1) a$$

$$= nc + an - a$$

$$\therefore T(n) = O(n)$$

Assume $B(n)$ is $O(n)$

$$(ii) \quad T(n) = \begin{cases} c, & n=1 \\ 2T(n/2) + a + n, & n>1 \end{cases}$$

$$\Rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n + a$$

$$= 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2} + a\right] + n + a$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + n + 2a + n + a$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2n + 2a + a$$

$$= 2^2 \left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} + a\right] + 2n + 2a + a$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + n + 2^2 a + 2n + 2a + a$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3n + (2^2 + 2 + 2^0)a$$

$\vdots$

$$= 2^k T\left(\frac{n}{2^k}\right) + kn + (2^{k-1} + 2^{k-2} + \dots + 2 + 1)a$$

$$= 2^k T\left(\frac{n}{2^k}\right) + kn + (2^k - 1)a$$

$$= 2^{\log_2 n} T(1) + (\log_2 n)a + (2^{\log_2 n} - 1)a \quad \left| \begin{array}{l} \frac{n}{2^k} = 1 \\ \Rightarrow 2^k = n \\ k = \log_2 n \end{array} \right.$$

$$= \log n(c) + a \log n$$

$$= c \log n + n \log n + an - a$$

$$\Rightarrow T(n) = O(n \log n)$$

4/16

$$T(n) = \begin{cases} 2 & , n=2 \\ \sqrt{n} T(\sqrt{n}) + n & , n>2 \end{cases}$$

$$T(n) = n^{1/2} T(n^{1/2}) + n$$

$$= n^{1/2} [n^{1/4} T(n^{1/4}) + n^{1/2}] + n$$

$$= n^{\frac{1}{2} + \frac{1}{4}} T(n^{1/4}) + n + n$$

$$= n^{\frac{1}{2} + \frac{1}{2^2}} T(n^{1/4}) + 2n$$

$$= n^{\frac{1}{2} + \frac{1}{2^2}} [n^{\frac{1}{8}} T(n^{1/8}) + n^{1/4}] + 2n$$

$$= n^{\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3}} T(n^{1/8}) + n^{\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^2}} + 2n$$

$$= n^{\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3}} T(n^{1/8}) + 3n$$

$$\frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^k} = \frac{1/2 (1 - \frac{1}{2^k})}{1/2}$$

$$= n^{\frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^k}} T(n^{1/2^k}) + kn$$

$$n^{1/2^k} = 2$$

$$\frac{1}{2^k} = \log_2 2$$

$$2^k = \log_2 n$$

$$k = \log \log n$$

$$= n \cdot n^{-1/2^k} T(n^{1/2^k}) + kn$$

$$= n \cdot \frac{1}{n^{1/2^k}} T(n^{1/2^k}) + kn$$

$$= n \cdot \frac{1}{2} T(2) + n \cdot \log \log n$$

$$= n + n \log \log n$$

$$T(n) = O(n \log \log n)$$

(*)/17

Best case:

Best case is when less no of steps are executed

$\therefore$ 2 recursive calls per one call will be called

$$\Rightarrow T(n) = 2T(n/2) + a \quad \therefore n/2^k = 1$$

$$\Rightarrow 2^k T(n/2^k) + (2^k - 1)a \quad \begin{matrix} 2^k = n \\ k = \log n \end{matrix}$$

$$= \log n T(1) + (n-1)a$$

$$\Rightarrow T(n) = O(n)$$

Worst case:

worst case is when ~~less~~ more no of recursive calls are executed per each call

$$T(n) = 8T(n/2) + a$$

$$= 8^2 T(n/2^2) + 8a + a$$

$$= 8^3 T(n/2^3) + 8^2 a + 8a + a$$

$$\Rightarrow 8^k T(n/2^k) + a(8^{k-1} + 8^{k-2} + \dots + 8a + a)$$

$$= 8^{\log_2 n} T(1) + a \left(\frac{8^k - 1}{8 - 1} \right)$$

$$\frac{n}{2^k} = 1$$

$$k = \log n$$

$$= 8^{\log_2 n} T(1) + \frac{a}{7} (8^{\log_2 n} - 1)$$

$$8^{\log_2 n} = 2^{3 \log_2 n}$$

$$= n^3$$

$$= n^3(1) + \frac{a}{7} (n^3 - 1)$$

$$\therefore T(n) = O(n^3)$$

Note:

From previous problem, we can say

if $T(n) = aT(n/2) + c$ then

$$T(n) = O(n^{\log_2 a}) \quad \left. \begin{array}{l} \text{if } a \neq 1 \end{array} \right\}$$

slly if $T(n) = aT(n/b) + c$ then

$$T(n) = O(n^{\log_b a})$$

if $a=1$, then $T(n) = O(\log_b n)$
(in both cases)

(H/B) 's' is a string of length n

$$T(n) = \begin{cases} c, & n=0 \\ 2T(n-1) + a, & n>1 \end{cases}$$

$$T(n) = 2T(n-1) + a$$

$$= 2[2T(n-2) + a] + a$$

$$= 2^2 T(n-2) + (2^2 - 1)a$$

$$= 2^k T(n-k) + (2^k - 1)a$$

$$\begin{aligned} n-k &= 0 \\ \Rightarrow k &= n \end{aligned}$$

$$= 2^{n-1} T(1) + (2^{n-1} - 1)a$$

$$= 2^{n-1} \cdot c + 2^{n-1} \cdot a - a$$

$$\Rightarrow T(n) = O(n)$$

Note:

$$1) T(n) = T(n-1) + c \quad \text{---} \quad O(n)$$

$$2) T(n) = T(n-1) + n \quad \text{---} \quad O(n^2)$$

$$3) T(n) = T(n-1) + \frac{1}{n} + a \quad \text{---} \quad O(n)$$

$$4) T(n) = T(n-1) + \frac{1}{n} \quad \text{---} \quad O(\log n)$$

$$5) T(n) = 2T(n-1) + C \dots\dots\dots O(2^n)$$

$$6) T(n) = 2T(n-1) + n \dots\dots\dots O(2^n) \dots\dots \text{solve it}$$

$$7) T(n) = T(\sqrt{n}) + C \dots\dots\dots O(\log \log n)$$

$$8) T(n) = 2T(\sqrt{n}) + C \dots\dots\dots O(\log n)$$

$$9) T(n) = 2T(n/2) + C \dots\dots\dots O(n)$$

$$10) T(n) = 2T(n/2) + n \dots\dots\dots O(n \log n)$$

$$11) T(n) = T(n/2) + C \dots\dots\dots O(\log n)$$

Analysing running time of program segments with loops:

→ for $i \leftarrow 1$ to n $\dots\dots\dots n$ times

$C = C + 1;$ $\dots\dots\dots$ repeats n times

$\therefore T.C : O(n)$

→ for $i \leftarrow 1$ to n $\dots\dots\dots n$ times

for $j \leftarrow 1$ to $n/2$ $\dots\dots\dots n/2$ times

$C = C + 1;$ $\dots\dots\dots n \cdot \frac{n}{2}$ times

$$\therefore T.C = (n) \left(\frac{n}{2} \right) = \frac{n^2}{2} = O(n^2)$$

If initial value of C is 0, then

value of C in the end will be $\frac{n^2}{2}$

→ for $i \leftarrow 1$ to n $\dots\dots\dots n$ times

for $j \leftarrow 1$ to $n/4$ $\dots\dots\dots n/4$ times

for $k \leftarrow 1$ to n $\dots\dots\dots 1$ time (i.e., $n \cdot \frac{n}{4}$ times)

break; $\dots\dots\dots n \cdot \frac{n}{4}$ times

$$\therefore T.C : n \cdot \frac{n}{4} \Rightarrow T.C : O(n^2)$$

→ For ($i=1; i \leq n; i++$) ----- n

For ($j=1; j \leq n; j++$) ----- n

For ($k=n/2; k \leq n; k+=n/2$) ----- 2

$C = C+1;$

TC: $n * n + 2 = O(n^2)$

value of C in the end $= 2n^2$

→ $i=1;$

while ($i \leq n$)

{

$i = i * 2;$

}

Assume that above loop is repeated k times

$$\Rightarrow 2^{k+1} > n$$

$$k+1 > \log n$$

$$k > \log n - 1$$

$$\therefore TC: O(\log n)$$

We repeat the loop until below condition is violated

$$2^k \leq n$$

$$k \leq \log_2 n$$

$$\Rightarrow \text{Time} \dots O(k) = O(\log n)$$

→ $i=n;$

while ($i > 0$)

{

$i = i/2;$

}

$$TC: O(\log n)$$

→ For ($i=1; i \leq n; i++$) ----- n

For ($j=1; j \leq n; j=2*j$) ----- $\log n$

$C = C+1;$

$$TC: O(n \log n)$$

→ $m = 2^n$

for ($i=1; i \leq n; i++$) ----- n

for ($j=1; j \leq m; j=2*j$) ----- $\log_2 m = n$

$c = c+1;$

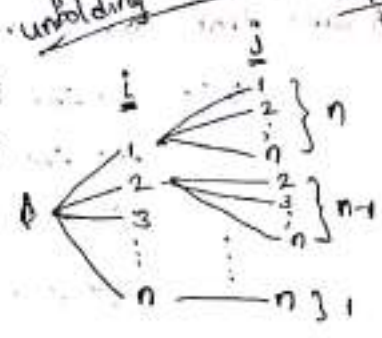
TC: $O(n^2)$

→ for $i \leftarrow 1$ to n ----- n

for $j \leftarrow i$ to n ----- $n-i+1$

$c = c+1;$

unfolding has to done if loop variables depend on each other



$$TC = 1 + 2 + 3 + \dots + n$$

$$= \frac{n(n+1)}{2} = O(n^2)$$

$$\text{value of } c = \frac{n(n+1)}{2}$$

→ $i = n;$

while ($i > 0$) ----- $\log_2 n$

{

$j = 1;$

while ($j \leq n$) ----- $\log_2 n$

{

$j = 2*j;$

$$\therefore (\log_2 n)(\log_2 n)$$

}

$i = i/2;$

}

$j = 2*j$ has max frequency

i.e., it is executed $(\log_2 n)^2$ times

$$\therefore TC: O(\log^2 n)$$

02/10/20

```
→ k=1; i=1;
while (k<=n)
{
    i++;
    k=k+i;
}
```

Assume that loop is repeated x times

ie, one time for $k=1$

2nd time for $k=3$

3rd time for $k=6$

x^{th} time for $k=1+2+\dots+x$

$$\Rightarrow 1+2+3+\dots+x \leq n$$

$$\frac{x^2+x}{2} \leq n$$

$$x^2+x-2n \leq 0$$

$$\cancel{x^2+x} \leq 2n \quad x^2+x \leq 2n$$

$\therefore x$ is $O(\sqrt{n})$

prob

```
→ int fun(int n)
{
    int i, j, p, q = 0;
    for (i=1; i<=n; ++i)
    {
        p=0;
        for (j=n; j>1; j=j/2)
            ++p;

        for (k=1; k<=p; k=k*2)
            ++q;
    }
    return (q);
}
```

For the function given,

we find order of value

returned by q .

$$q = O(?)$$

j loop runs $\log_2 n$ times
 $\Rightarrow P = \log_2 n$

k loop runs $\log_2 P$ times

$\Rightarrow$ i.e. $\log \log n$ times

$\Rightarrow$ for every iteration of outer loops iteration q value is incremented by $\log \log n$.

outer loop runs 'n' times.

$\therefore$ order of value q is $n \log \log n$

$\rightarrow$ int Run(int n)

{

int i, j, P, q = 0;

P = 0;

for (i = 1; i <= n; i++)

{ for (j = n; j > 1; j = j/2)

++P;

for (k = 1; k <= P; k = k*2)

++q;

}

return q;

}

Find order of value returned.

TC of the program

on the left side is

$O(n \log n)$

$\rightarrow$ solve it

for every 'j' loop iteration execution p is incremented by $\log n$

i=1	i=2	i=3	...	i=n
P = $\log n$	P = $2 \log n$	P = $3 \log n$	...	P = $n \log n$
q = $\log P$	q = $\log P$	q = $\log P$	...	q = $\log P$
= $\log \log n$	= $\log(2 \log n)$	= $\log(3 \log n)$	...	= $\log(n \log n)$

$\therefore q = \log(\log n) + \log(2 \log n) + \log(3 \log n) + \dots + \log(n \log n)$

= $\log(\log n) + \log 2 + \log \log n + \log 3 + \log \log n + \dots + \log n + \log \log n$

= $n \log(\log n) + \log n! \Rightarrow$ order $q = O(\log n!) = O(n \log n)$

→ Find TC of below code

```
for (i=1; i<=n; i++) ----- n
```

```
{
```

```
    j=1; ----- 1
```

```
    while (j<=n) ----- log n
```

```
        j=2*j;
```

```
        for (k=1; k<=n; k++) ----- n
```

```
            c=c+i;
```

```
}
```

Time = $n(1 + \log n + n)$

$= n + n \log n + n^2$

$\Rightarrow$ TC: $O(n^2)$

→ ~~$n = 2^k$~~ $n = 2^k$

```
for (i=1; i<=n; i++)
```

```
{
```

```
    j=2;
```

```
    while (j<=n)
```

```
    {
```

```
        j=j*j;
```

```
        print("*");
```

```
    }
```

```
}
```

for one iteration of outer loop -

$j = 2 \rightarrow 2^2 \rightarrow (2^2)^2 = 2^4 \rightarrow (2^4)^2 = 2^8 \rightarrow (2^8)^2 = 2^{16} \rightarrow \dots \rightarrow j = 2^{2^k} \geq n$

while loop runs $k+1$ times

$\Rightarrow$ $k+1$ '*'s are printed

$\therefore$ total no of stars printed $> n(k+1)$

$= n(\log \log n + 1)$

$n = 2^{2^k}$

$2^k \geq \log n$

$k \geq \log \log n$

101
→ A: Array $[1 \dots n]$ of binary

$f(m) = \Theta(m)$

count: integer;

count = 1;

for $i = 1$ to n

{

if $(A[i] == 1)$ count ++;

else

{

$f(\text{count})$;

count = 1; // line 1

}

}

Q: What is best case & worst case TC for above program?

Sol:

Case I:

all bits are 1

TC: $O(n)$

Case II: all bits are 0

TC: $O(n)$

Case III: $A[1 \text{ to } n-1] = 1$ & $A[n] = 0$

TC: $(n-1) + O(n) = O(n)$

Case IV: First $\frac{n}{2}$ bits are 1 & $A[\frac{n}{2}+1 \text{ to } n] = 0$

Time: $i = 1 \text{ to } \frac{n}{2}$ $i = \frac{n}{2} + 1$ $i = \frac{n}{2} + 2 \text{ to } n$

line: $\frac{n}{2}$ $\frac{n}{2} + 1$ $O(\frac{n}{2} - 1)$

$\Rightarrow$ TC: $O(n)$

Case V:

Alternative 1's & 0's

TC: $O(n)$

Here we can observe that irrespective of I/P sequence the TC is $O(n)$.

$\therefore$ Best case: $O(n)$

Worst case: $O(n)$

(ii) ~~First~~ Deleting line 1, determine the best & worst case TC

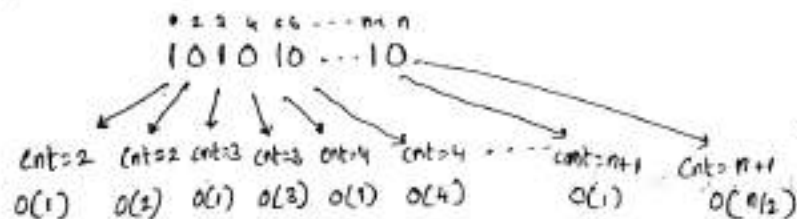
Sol:

If all inputs are 1 then

$O(n)$ --- Best case.

worst case:

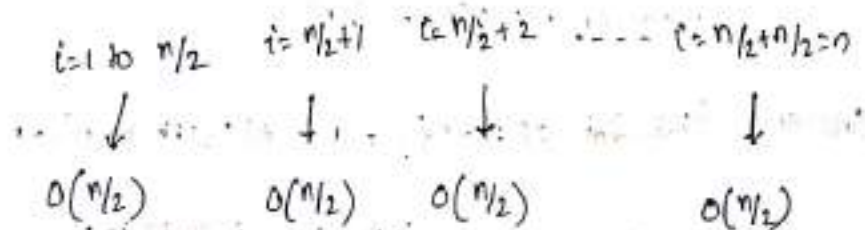
i, Consider case with alternate 1's & 0's



$$\Rightarrow O(1) \frac{n}{2} + O\left[2+3+4+\dots+n/2\right]$$

$\Rightarrow O(n^2)$ --- worst case

(ii) Consider case with first half are 1's and next half are 0's

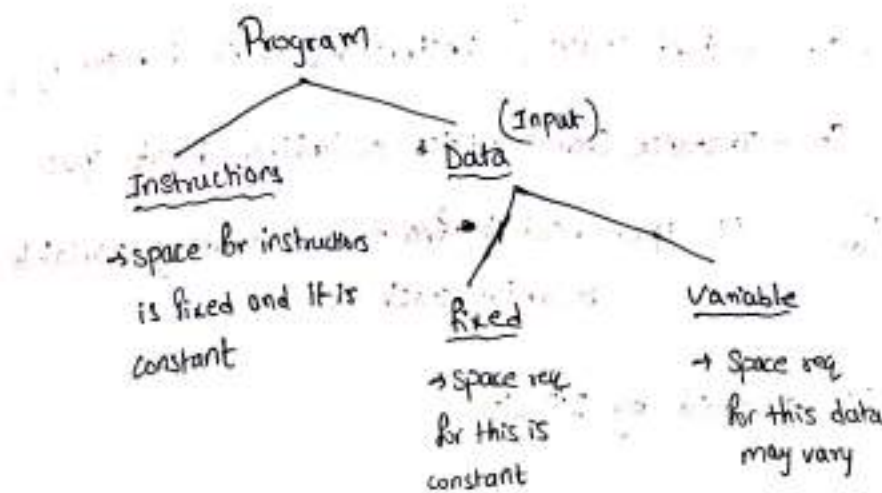


$$\therefore \text{time} = O(n/2) + \frac{n}{2} \cdot O(n/2)$$

$= O(n^2)$ --- worst case

Space Complexity

103



→ Space complexity is not for program instructions or not for data.

Defo:

The space used by an algorithm is defined as the number of memory words needed to carry out the computational steps required to solve an instance of the problem, excluding the space allocated to hold the input. In other words, it is only workspace required by the algorithm.

→ All definitions of order of growth & asymptotic bounds pertaining to time complexity equally applies to space complexity.

→ The workspace (space complexity) cannot exceed the running time of an algorithm, since writing into each memory cell requires atleast a constant amount of time.

i.e., if $T(n)$ = time complexity &

$S(n)$ = space complexity then

$$S(n) = O(T(n))$$

$$\text{Space Complexity (P)} = C + S_p$$

determined
prior to the
execution

← C — Constant Space (Instructions & Fixed memory)

depends ← S_p — Instance Characteristics excluding input space.

on ~~variable~~ input
and determined at
runtime

i.e., space needed for temporary variable,
recursion, stack etc.

→ S_p is governed by I/P size

Eg: void swap(int a, int b)
{
 int t;
 t = a;
 a = b;
 b = t;
}

input

additional
variable

TC: $O(1)$

now 't' contributes to space complexity

size(t) : workspace = time complexity

size(t) : 2 bytes = $O(1)$

$$S = C + S_p$$

$$= 2 + 0$$

$$= O(1)$$

①

Eg: Algorithm sum(A, n)

A[1...n], n: int

TC: $O(n)$

{

int i, sum = 0;

for i ← 1 to n

sum = sum + A[i];

}

A & n are inputs

∴ A & n doesn't contribute to space complexity

variables i, sum contribute to space complexity

$$S = C + S_p = 4 \text{ bytes} + 0$$

$$= O(1)$$

Eg: Algo Rsum(A, n)

```
II {  
    if (n == 1)  
        return A[1];  
    else  
        return (Rsum(A, n-1) + A[n]);  
}
```

Since recursion is used we need consider size of stack used.

Stack contains activation records.

Activation record has formal parameters, local variables, return address etc.

$\therefore$ size of activation record is constant.

Here we need $O(n)$ activation records.

$\therefore$ amount of space used in stack

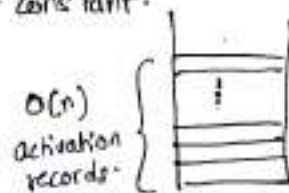
$$= O(n) * C = O(n)$$

$\therefore$ Space complexity = $C + Sp$

$$= O + O(n)$$

$$= O(n)$$

Time complexity = $O(n)$



↓
workspace
of this algorithm

Observation:

Both algo I & algo II, does the same job: of computing sum of array elements.

Algo I $\left\{ \begin{array}{l} TC: O(n) \\ SC: O(1) \end{array} \right.$

Algo II $\left\{ \begin{array}{l} TC: O(n) \\ SC: O(n) \end{array} \right.$

$\therefore$ Algo I is space efficient.

Algo II is space inefficient.

General rule:

An algorithm is said to be space efficient, if its space requirement is $O(1)$ or atmost $O(\log n)$ in case of recursive algorithm.

Eg: Algo Test (A, n)

$A[1..n, 1..n]$, $n, \text{int} \leftarrow \text{input}$

{
 integer $B[1..n], i$;

 for $i \leftarrow 1$ to n

$B[i] = A[i, i]$

}

Tc: $O(n)$

B, i are considered workspace.

$$\begin{aligned} S &= C + Sp \\ &= \downarrow \quad \downarrow \\ &= \text{size}(C) + \text{size}(B) \\ &= k + O(n) \end{aligned}$$

$$\therefore SC = O(n)$$

Eg: Algo $A(n)$

{
 if ($n=1$) return;
 else

Tc: $O(\log n)$

$A(n/2)$;

}

}

Space complexity is calculated by amount of space used by stack.

$\Rightarrow$ no of recursive calls = $\log n$

$\Rightarrow$ space = $(\log n) \times (\text{size of activation record})$

$$S = C + Sp = 0 + \log n * c = O(\log n)$$

Eg: $A(n)$

```
{ if(n=2) return;  
  else  
    return (A(n));  
}
```

Space complexity in this case is order of no of rec. calls.

let $S(n)$ denote no of recursive calls.

$$\Rightarrow S(n) = 1 + S(n/2)$$

$$= 1 + S(n^{1/2})$$

$$= 1 + 1 + S(n^{1/2^2})$$

$$= 2 + S(n^{1/2^2})$$

$$\vdots$$
$$= k + S(n^{1/2^k})$$

$$\frac{1}{n^{1/2^k}} \approx 2$$

if k is no of recursive calls.

$$\text{then } n^{1/2^k} = 2$$

$$\Rightarrow 2^k = \log_2 n$$

$$k = \log \log n$$

$$\therefore \text{Space Complexity} = O(\log \log n)$$

Eg: $\text{recur}(n)$

```
{  
  if(n=1) return;  
  else  
  {  
    recur(n/2);  
    recur(n/2);  
    B(n);  
  }  
}
```

let $S(n)$ denote no of recursive calls.

~~$T(n)$ = order of no of recursive calls~~

$$= 1 + 2S(n/2)$$

$$= 2[2 \cdot S(n/2) + 1] + 1$$

$$= 2^2 S(n/2^2) + 2^2 - 1$$

$$= 2^k S(n/2^k) + 2^k - 1$$

~~If k is no of recursive calls then~~

$$\frac{n}{2^k} = 1$$

$$\Rightarrow k = \log_2 n$$

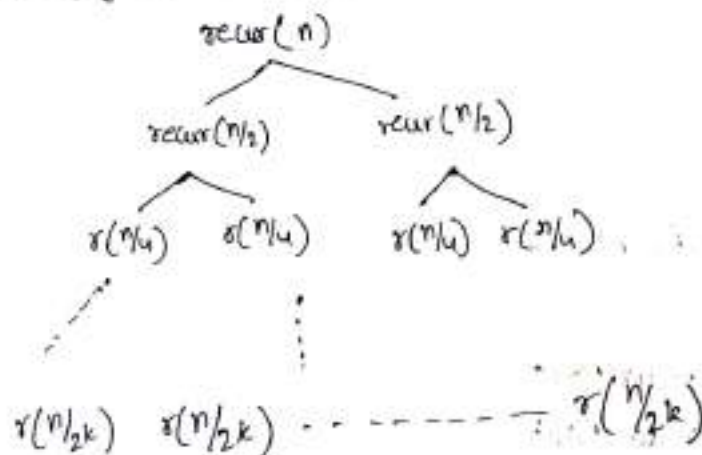
$$S(n) = 2^{\log_2 n} S(1) + 2^k - 1$$

$$= n + n - 1$$

$$= 2n - 1$$

$\therefore$ no of recursive calls = $2n - 1$

However observing the recursion stack, the depth of recursion stack is only k .



$\therefore$ space complexity = depth of recursion stack

$$= O(k) = O(\log n)$$

Design Strategies

Divide & Conquer

→ Find value of C , & time complexity for below code segment.

```
for i ← 1 to n-1
  for j ← i+1 to n
    for k ← 1 to j
      C = C + 1;
```

TC: $O(n^3)$

$$C = \frac{(n-1)(n)(n+1)}{3}$$

→ Find value of C & TC for below code segment

```
for (i=1; i<=n; i++)
  for (j=1; j<=i*i; j++)
    if ((j%i) == 0)
      for (k=1; k<=j; k++)
        C = C + 1;
```

TC: $O(n^4)$

$$C = \frac{n(n+1)(3n^2+7n+2)}{24}$$

04/10/20

Divide & Conquer

→ This strategy is used when the problem (its input) is large / complex.

→ So we divide it into subproblems ~~and~~ until the subproblem is small.

→ A subproblem is said to be small, if it can be solved in one/two fundamental (basic) operations.

→ Combine (conquer) the results of smaller problems to get the result of the original ~~prob~~ problem.

This step of combining is optional. It is done only if it is needed.

Eg: quicksort uses only dividing but doesn't perform combining.

→ Division can be done into 2 or more problems and each ~~sub~~ subproblem can be of different sizes.

Control Abstraction:

→ A procedure whose flow of control is clear but the inner details are abstracted.

Algorithm DandC(P)

```
{  
  if (SMALL(P)) then  
    return(S(P)); // S(P) is soln of smaller problem.  
  else  
  {  
    Divide P into smaller problems  $P_1, P_2, \dots, P_k$  ( $k \geq 1$ ) // Mandatory  
    Apply DandC to each subproblems  
    return(COMBINE(DandC( $P_1$ ), ..., DandC( $P_k$ ))); // optional  
  }  
}
```

→ The below procedure is a general procedure for dividing problem into 2 subproblems.

Procedure DandC(P, l, h)

{

if (SMALL(l, h))

return $G(P, l, h)$;

else

$m \leftarrow \text{Divide}(l, h)$;

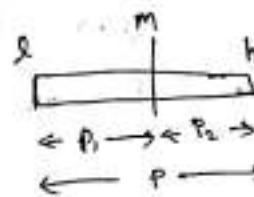
$\text{Soln1} \leftarrow \text{DandC}(P, l, m)$;

$\text{Soln2} \leftarrow \text{DandC}(P, m+1, h)$;

return COMBINE($\text{soln1}, \text{soln2}$);

}

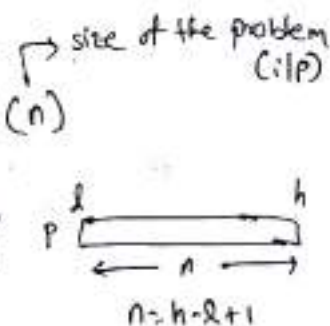
}



→ The TC of D&C problems is expressed in the form of DandC recurrence equations.

let $T(n)$ represent time complexity of DandC(n)

Consider we divide the problem into 2 subproblems



$$T(n) \begin{cases} f(n) & , n \text{ is small} \\ 2T(n/2) + g(n) & , n \text{ is large} \end{cases}$$

$f(n) \rightarrow$ time for SMALL & function G (in most of the cases this is $O(1)$)

$g(n) \rightarrow$ time for SMALL, dividing & combining

The general form of DandC Recurrence: (for symmetric ~~no~~ DandC)
 i.e., equal size subproblems

$$T(n) \begin{cases} f(n) & , n \text{ is small} \\ a \cdot T(n/b) + g(n) & , n \text{ is large} \end{cases}$$

$a \rightarrow$ no. of subproblems $\therefore a \geq 1$

$\frac{n}{b} \rightarrow$ size of each subproblem $\therefore b > 1$ ($\because$ problem size has to reduce when it is divided)

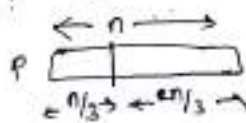
$g(n) \rightarrow$ time for ~~do~~ divide, combine

$$\therefore g(n) > 0$$

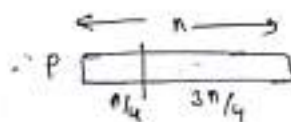
$$\therefore a \geq 1; b > 1; g(n) > 0$$

Variation of DandC recurrence: (asymmetric)

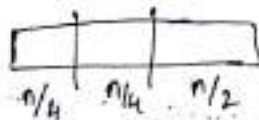
unequal sized subproblems



$$\Rightarrow T(n) = T(n/3) + T(2n/3) + g(n)$$



$$\Rightarrow T(n) = T(n/4) + T(3n/4) + g(n)$$



$$\Rightarrow T(n) = 2 \cdot T(n/4) + T(n/2) + g(n)$$

~~Sum of sizes of subproblems~~

~~= size of original problem.~~

→ Sometimes it's possible that sum of size of subproblems not equal to size of original ~~program~~ problem.

$$\text{Ex: } T(n) = T(n/2) + T(n/4) + g(n)$$

$$\text{Ex: } T(n) = T(n/2) + g(n)$$

$$\text{general form: } T(n) = T(\alpha n) + T(\beta n) + T(\gamma n) + \dots + g(n)$$

where $0 < \alpha, \beta, \gamma, \dots < 1$

Note:

$$T(n) = 8T(n/2) + g(n)$$

The above eq is valid even ~~for~~ though size of subproblem is $n/2$ and no of subproblems are 8.

Reason: It could be the case that a subproblem is solved multiple times

(or)

It is possible when the input data ~~store~~ structure is non-linear

For example, for input being $n \times n$ matrix

& dividing it ~~into~~ into 4 subproblems where

each is $\frac{n}{2} \times \frac{n}{2}$ matrix.

i) Max-Min:

The problem is finding minimum and maximum elements in an array of size n elements.

Naive approach (Non-DC):

Procedure Non-DC (A, n, f_{max}, f_{min})

{

1. $f_{max} \leftarrow f_{min} \leftarrow A[1];$

2. For $i \leftarrow 2$ to n

{ if ($A[i] > f_{max}$)

$f_{max} \leftarrow A[i];$

if ($A[i] < f_{min}$) // line 1

$f_{min} \leftarrow A[i];$

}

}

TC: $O(n)$

→ Here the metric based on which we measure the time is comparison. (Also we consider comparisons involving array element but we ignore comparison of index in for loop)

Here total no of comparisons = $2(n-1)$

↳ (best, avg, worst cases)

→ Here by replacing line 1 with "else", for certain elements we can reduce no of comparisons.

best case → $n-1$ (increasing order)

worst case → $2(n-1)$ (decreasing order)

Avg. case $\rightarrow$ on an average assume that the first comparison fails for $1/2$ of given elements.

It is given that comparison fails for $1/2$ elements

$\Rightarrow$ for $n/2$ elements comparison fails

$\Rightarrow n/2$ comparisons out of $n-1$ comparisons fails

In this case no of comparisons

$$\text{req} = n/2 * 2 = n$$

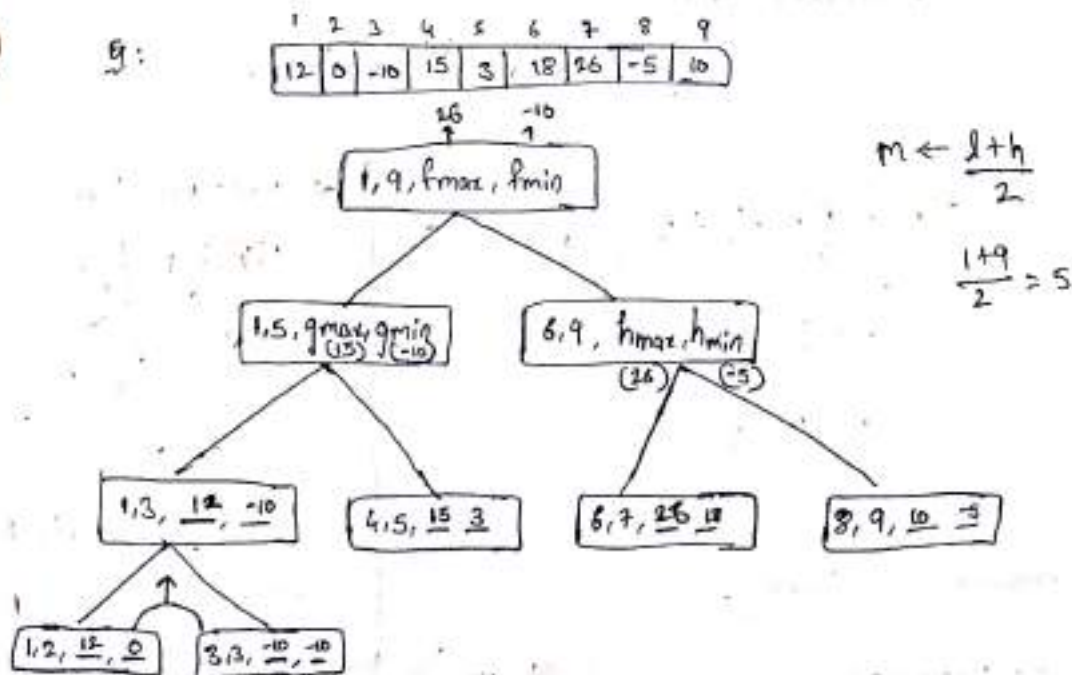
$\Rightarrow$ for $n-1-n/2$ comparison are success

$\Rightarrow$ total comparisons $\rightarrow n + n-1-n/2$

$$\frac{3n}{2} - 1$$

Divide and Conquer approach:

If $n=1$ or $n=2$ then we say the problem is small. Since in that case no of comparisons will be either 0 (or) 1, respectively.



Let $T(n)$ represent the no. of comparisons made in DandC MergeSort

$$T(n) = \begin{cases} 0 & , n=1 \\ 1 & , n=2 \\ 2T(n/2) + 2 & , n > 2 \end{cases}$$

$$T(n) = 2T(n/2) + 2$$

$$= 2[2T(n/4) + 2] + 2$$

$$= 2^2 T(n/4) + 2^2 + 2$$

$$= 2^3 T(n/8) + 2^3 + 2^2 + 2$$

$$= 2^k T(n/2^k) + 2^k + 2^{k-1} + \dots + 2$$

$$= 2^k T(n/2^k) + 2 \frac{(2^k - 1)}{2 - 1}$$

$$= 2^k T(n/2^k) + 2 \cdot 2^k - 2$$

$$= \cancel{n T(1)} + 2n - 2$$

terminating condition

$$\frac{n}{2^k} = 2 \Rightarrow n = 2^{k+1} \Rightarrow 2^k = n/2$$

$$T(n) = \frac{n}{2} T(2) + 2 \cdot \frac{n}{2} - 2$$

$$= \frac{n}{2} (1) + n - 2$$

$$= \frac{3n}{2} - 2$$

→ no. of comparison in DandC = $\frac{3n}{2} - 2$

→ Time complexity: $O(n)$

↳ (All cases)

we should not take

$$T(n/2^k) = 1 \quad \frac{n}{2^k} = 1$$

$$\therefore \frac{n}{2^k} = 1$$

$$\Rightarrow \frac{n}{2^{k-1}} = 2$$

which was already a terminating condition

In most of the cases it terminates with $n=2$.

Performance Comparison

I/P class	Non-DC	DC
Inc order	$(n-1)$	$\frac{3n}{2} - 2$
Dec order	$2(n-1)$	$\frac{3n}{2} - 2$
random order	$\frac{3n}{2} - 1$	$\frac{3n}{2} - 2$
TC	$O(n)$	$O(n)$
space-c	$O(1)$	$O(\log n)$

} naive approach is better

} Dand C is better here

Space Complexity

Non-DC: $O(1)$ --- memory for i

DC:

Here we need consider the space for stack.

The stack grows upto a depth of $\log n$

$\therefore O(\log n)$

Merge Sort

	1	2	3	4	5	6	7	8	9	10
A:	319	285	179	652	351	423	861	254	450	520

Merging Process (conquer)

→ the below is about 82-way merging i.e., we merge 2 sorted lists

Problem stmt: Given two sorted lists L_1 & L_2 , it is req to merge them into single sorted list.

Lists

$$L_1: \langle 4, 5, 8, 12, 20 \rangle \quad L_2: \langle 3, 7, 11, 18, 25, 30 \rangle$$

Assume $\text{size}(L_1) \leq \text{size}(L_2)$

let combined list be L .

let i, j be indices pointing to the start of the list.

$$L_1: \langle 4, 5, 8, 12, 20 \rangle \quad L_2: \langle 3, 7, 11, 18, 25, 30 \rangle$$

$\uparrow$
 i

$\uparrow$
 j

$$L_1[i] < L_2[j]$$

$$\therefore L[i] = L_2[j]; j++$$

$$L_1: \langle 4, 5, 8, 12, 20 \rangle \quad L_2: \langle 3, 7, 11, 18, 25, 30 \rangle$$

$\uparrow$
 i

$\uparrow$
 j

$$L: \langle 3, \dots \rangle$$

$$L_1[i] < L_2[j]$$

$$\therefore L[2] = L_1[i]; i++$$

$$L_1: \langle 4, 5, 8, 12, 20 \rangle \quad L_2: \langle 3, 7, 11, 18, 25, 30 \rangle$$

$\uparrow$
 i

$\uparrow$
 j

$$L: \langle 3, 4, \dots \rangle$$

Continuing this process, we obtain

$$L: \langle 3, 4, 5, 7, 8, 11, 12, 18, 20, 25, 30 \rangle$$

one of the two array will be completely traversed.

Now append the elements of array that is not completely traversed to the end of L .

Finally we obtain

$$L: \langle 3, 4, 5, 7, 8, 11, 12, 18, 20, 25, 30 \rangle$$

~~let n be size of L_1 & m be size of L_2~~

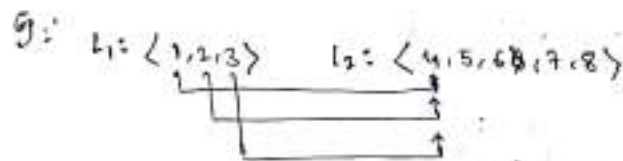
Analysis of Merge process:

Finding no of comparisons:

→ Here we consider comparison only b/w array elements
let n be size(L_1) & m be size of(L_2)

Case I: (best case)

every element of L_1 < starting element of L_2



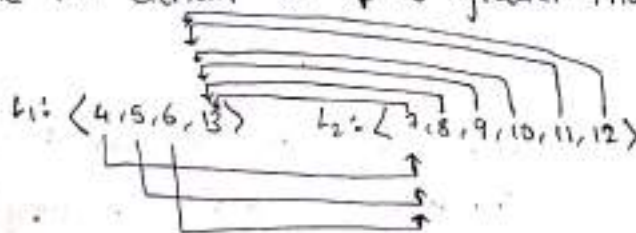
$\therefore$ no of comparison req = ~~size~~ 3 = size of L_1

Case II: (worst case)

~~every L_1 except last element of~~

$n-1$ of list L_1 are less than 1st element of L_2

and n th element of L_1 is greater than all elements of L_2



$\therefore$ no of comparisons = $(n-1) + m$

$$= n + m - 1$$

$\therefore$ ~~Time complexity~~

no of comparisons in best case = $\min(n, m)$

no of comparisons in worst case = $n + m - 1$

$\therefore$ TC: $O(n+m)$

Now we see how mergesort works

	1	2	3	4	5	6	7	8	9	10
A:	310	285	179	652	351	423	861	254	450	520

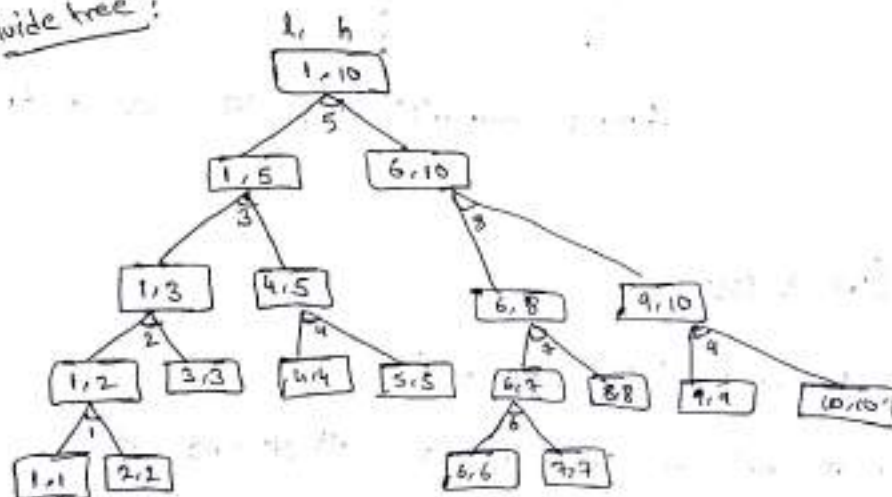
Merge sort uses a temporary array which is of the same size of A. let this temporary array be B.

B:

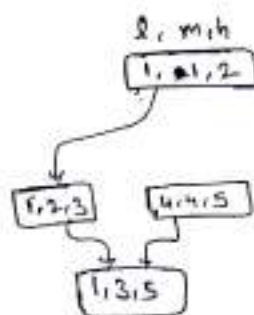
→ we say the problem is small when there is 1 element in the list.

$$\text{Division: mid} \leftarrow \frac{l+h}{2}$$

Divide tree:



Conquer tree:



while combining we store the elements in B.

And after every combine, we copy those combined elements from B to A.

B[1] = 285 B[2] = 310

Now copy them into A

A: <285, 310, 179, 652, ...>

combine(1, 2, 3):

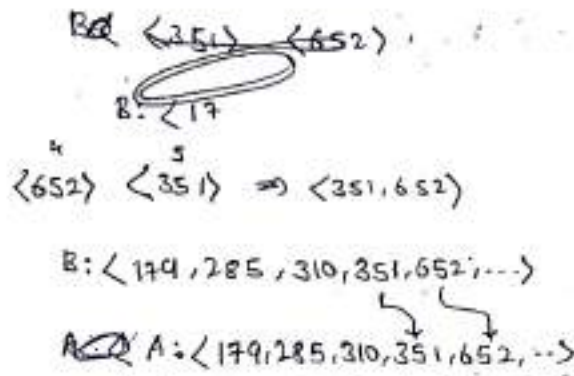
~~B[1] = 179~~

<285, 310> <179>

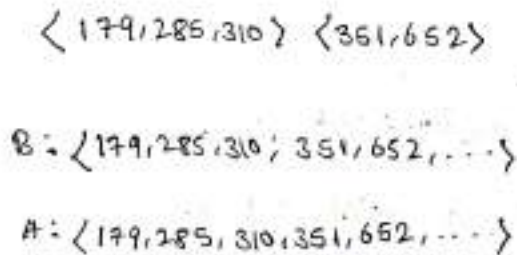
B: <179, 285, 310, ...>

A: <179, 285, 310, ...>

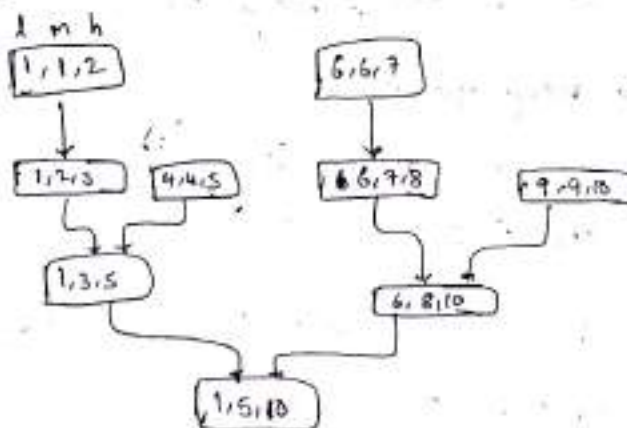
Combine (4,4,5):



Combine (1,3,5):



Combine tree:



Performance Analysis:

Let $T(n)$ represent time complexity of Merge Sort

$$T(n) = \begin{cases} c, & n=1 \\ 2T(n/2) + bn, & n>1 \quad (b>0) \end{cases}$$

We can also have

$$T(n) = 2T(n/2) + O(n)$$

for the process of merging, checking if the problem is small etc.

bn in best case $\rightarrow n/2$
 worst case $\rightarrow n/2 + n/2 - 1 = n-1$

∴ time complexity of merge sort

$$T(n) = O(n \log n)$$

space complexity:

$$S = C + S_p$$

C is some ~~constant~~ constants and

$S_p \rightarrow$ sum of stack size + additional ~~array~~ array B

$$\rightarrow O(\log n) + O(n)$$

$$\therefore S = O(n)$$

∴ Merge Sort is not ^{space} efficient. ~~with space~~

Merge Sort is not inplace sorting.

2-way / bottom-up Merge Sort (variation of merge sort)

→ This sorting uses only merge operations.

→ Here we consider array is already divided and hence we perform only merge operations

A: $\langle 310, 285, 179, 652, 351, 423, 861, 254 \rangle$

no of merge operations

Pass 3 → $[179, 254, 285, 310, 351, 423, 652, 861]$

Pass 2 → $[179, 285, 310, 652]$ $[254, 351, 423, 861]$

Pass 1 →

Pass 1 → $[285, 310]$ $[179, 652]$ $[351, 423]$ $[254, 861]$

A: $[310]$ $[285]$ $[179]$ $[652]$ $[351]$ $[423]$ $[861]$ $[254]$

$$\frac{8}{2^3} = \frac{2}{2} = 1$$

$$\frac{8}{2^2} = \frac{4}{2} = 2$$

$$8/2 = 4$$

Time complexity : $O(n \log n)$

∴ $\log n$ passes & every pass has comparisons less than n.

W/35

Time complexity of mergesort = $n \log n$

$$n = 64 \begin{cases} \text{Apriori time} = 30s \\ \text{Apriori time} = 64 \log_2 64 = 64 \times 6 \text{ units} \end{cases}$$

$$\Rightarrow 64 \times 6 \text{ units} \rightarrow 30s$$

$$1 \text{ unit} \rightarrow \frac{30}{64 \times 6} s$$

$$\Rightarrow 360s \rightarrow 64$$

$$\Rightarrow 1s \rightarrow \frac{64 \times 4}{30}$$

$$6 \text{ min} \Rightarrow 360s \rightarrow \frac{64 \times 4 \times 360}{30} = 4608$$

Let n be no of elements sorted in 360s

$$\text{then } n \log n = 4608$$

$$\text{let } n = 2^k$$

$$\Rightarrow 2^k \cdot k = 4608$$

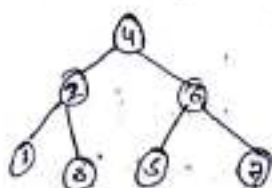
$k = 9$ satisfies the equation

$$\therefore n = 2^k = 512 \text{ elements.}$$

Binary Search:

* $\rightarrow$ Binary search requires that the list is sorted

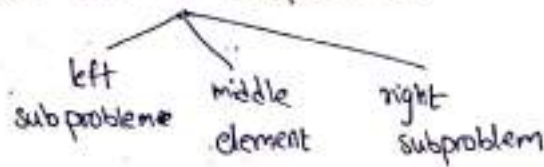
Consider a list of size 7



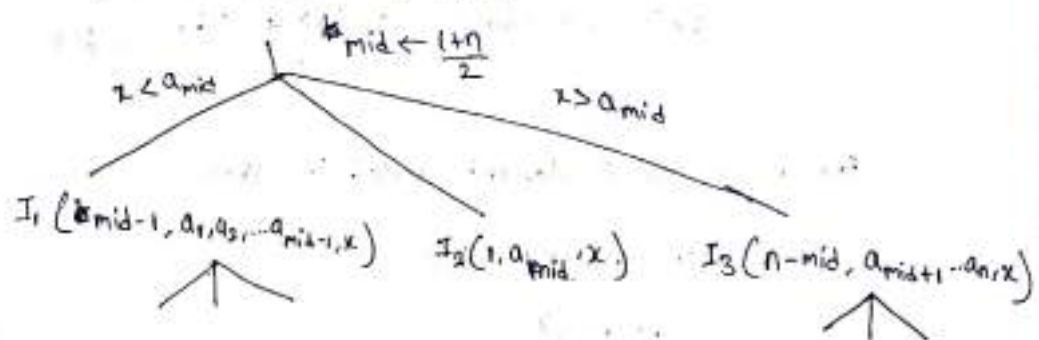
Search path ~~is~~ using binary search corresponds to one of the path from root to leaf in the above tree.

DandC-BS(n, x)

- we say a problem is small when there is only one element
- In binary search, ~~the binary~~ when problem is large ($n > 1$), the problem is divide into 3 subproblems



$$I: \langle n, a_1, a_2, a_3, \dots, a_n, x \rangle$$



Out of these 3 subproblems, we solve only 2

i.e., I_2 and/or (I_1 or I_3)

→ Binary search ~~don~~ doesn't perform combine operation.

Performance Analysis:

time complexity: Let $T(n)$ represent time complexity of DandC-BS(n)

$$T(n) = \begin{cases} c, & n=1 \\ T(n/2) + a + b + c, & n > 1 \end{cases}$$

solving I_2

checking if problem is large

time for divide

$$T(n) = T(n/2) + k, n > 1, k > 0$$

$$\therefore T(n) = O(\log n)$$

→ Best case Tc of binary search = $O(1)$

→ Space Complexity:

$$S = C + Sp$$

↘

$$= C_1 + \text{size of stack used}$$

$$= C_1 + \log n$$

$$S = O(\log n)$$

However for non-recursive implementation of binary search
space complexity, $S = O(1)$

Linear Search:

→ Best case $TC = O(1)$

→ worst case & avg case $TC = O(n)$

→ The avg no of key comparisons involved in a successful sequential search with n -elements is $\frac{n+1}{2}$

$$\text{i.e., } \frac{1+2+3+\dots+n}{n} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Matrix Multiplication [square matrices]

$$\rightarrow C = A \times B$$

naive approach:

```

for i ← 1 to n
  for j ← 1 to n
    {
      C[i,j] = 0;
      for k ← 1 to n
        C[i,j] = A[i,k] + B[k,j];
    }
  
```

③ Calculate the total no of minimum & maximum no of element comparisons involved in 2-way / bottom up merge-sort with $n=2^k$ elements.

Sol:

Minimum Comparisons:

Diagram illustrating the minimum comparisons: A tree structure showing the merging process. The root node is labeled $\frac{2^k}{2} \times 1$ with an arrow pointing to it labeled "no. of lists elements". Below the root, an arrow points to the text "no. of merge operations". The tree branches down to $\frac{2^{k-1}}{2} \times 2$, then to $\frac{2^{k-2}}{2} \times 2^2$, and so on, ending at $\frac{2^{k-i}}{2} \times 2^i$. An arrow points from the tree to the text "min no of comparisons".

$$\left(\frac{2^k}{2} \times 1\right) + \left(\frac{2^{k-1}}{2} \times 2\right) + \left(\frac{2^{k-2}}{2} \times 2^2\right) + \dots + \left(\frac{2^{k-i}}{2} \times 2^i\right)$$

At the termination, no of ~~element~~ merge operations will be one.

$$\Rightarrow \text{no of lists} = 2$$

$$\text{i.e., } 2^{k-i} = 2$$

$$\Rightarrow k-i=1$$

$$\Rightarrow i=k-1$$

$$\Rightarrow \frac{2^k + 2^k + \dots + 2^k \text{ (k times)}}{2} = \frac{k \cdot 2^k}{2} = k \cdot 2^{k-1}$$

$$\therefore \text{Min no of comparisons} = \frac{n \log n}{2}$$

Maximum Comparisons:

$$\left(\frac{2^k}{2} \times 1\right) + \left[\frac{2^{k-1}}{2} \times (2^2 - 1)\right] + \left[\frac{2^{k-2}}{2} \times (2^3 - 1)\right] + \dots + \left[\frac{2^{k-i}}{2} \times (2^{i+1} - 1)\right]$$

$$\Rightarrow \left[\frac{2^k}{2} \times 2^1 - 1\right] + \left[\frac{2^{k-1}}{2} \times (2^2 - 1)\right] + \dots + \left[\frac{2^1}{2} \times (2^k - 1)\right] \quad \left\{ \begin{array}{l} 2^{k-i} = 2 \\ k-i=1 \\ i=k-1 \end{array} \right.$$

$$\frac{1}{2} \left[\underbrace{(2^{k+1} + 2^{k+1} + \dots + 2^{k+1})}_{k \text{ times}} - (2^k + 2^{k-1} + \dots + 1) \right]$$

$$= \frac{1}{2} [k \cdot 2^{k+1} - 2(2^k - 1)] = 2^k (k-1) + 1$$

$$= n \log n - n + 1$$

$\therefore$ Max no of Comparison = $n \log n - n + 1$

05/10/20

Consider

$$A = \begin{array}{c|c} A_{11} & A_{12} \\ \hline \begin{matrix} 4 & 5 \\ 8 & 9 \end{matrix} & \begin{matrix} 6 & 7 \\ 1 & 2 \end{matrix} \\ \hline A_{21} & A_{22} \\ \hline \begin{matrix} 3 & 9 \\ 6 & 7 \end{matrix} & \begin{matrix} 1 & 5 \\ 2 & 3 \end{matrix} \end{array} \cdot B = \begin{array}{c|c} B_{11} & B_{12} \\ \hline \begin{matrix} 9 & 6 \\ 4 & 1 \end{matrix} & \begin{matrix} 5 & 2 \\ 3 & 9 \end{matrix} \\ \hline B_{21} & B_{22} \\ \hline \begin{matrix} 8 & 6 \\ 4 & 1 \end{matrix} & \begin{matrix} 3 & 2 \\ 3 & 5 \end{matrix} \end{array} = C = \begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array}$$

Here

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$A, B, C \rightarrow n \times n$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$A_{ij}, B_{ij}, C_{ij} \rightarrow \frac{n}{2} \times \frac{n}{2}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

$\rightarrow$ Here we say problem is small when the matrix is 2×2 or 1×1

$$T(n) = \begin{cases} c, & n \leq 2, (c > 0) \\ 8T(n/2) + bn^2, & n > 2 \end{cases}$$

8 multiplications
where each multiplication
is b/w two $n/2 \times n/2$
matrices

time for
addition

$$T(n) = 8T(n/2) + bn^2$$

$$= 8 \left[8T(n/2^2) + b(n/2)^2 \right] + bn^2$$

$$= 8^2 T(n/2^2) + 8 \frac{bn^2}{4} + bn^2$$

$$= \cancel{8^2 T(n/2^2)} + \cancel{\frac{8^2}{4}} bn^2$$

$$= \cancel{8^k T(n/2^k)} + \cancel{\frac{8^k}{4}} bn^2$$

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow 8^k = n^3$$

$$= \cancel{n^3 T(1)} + \cancel{n^3 - 1} bn^2$$

$$\therefore \sim 8^2 \left[8T(n/2^2) + b(n/2)^2 \right] + 2bn^2 + bn^2$$

$$= 8^3 T(n/2^3) + 2^2 bn^2 + 2bn^2 + bn^2$$

$$= 8^3 T(n/2^3) + (2^3 - 1) bn^2$$

$$= 8^k T(n/2^k) + (2^k - 1) bn^2$$

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow 8^k = n^3$$

$$= n^3 T(1) + (n-1) bn^2$$

$$= cn^3 + bn^3 - bn^2$$

$$\therefore T(n) = O(n^3)$$

$$\text{Space Complexity} = O(\log n)$$

$\therefore$ So this approach of matrix multiplication didn't reduce the time complexity.

Strassen's Matrix Multiplication

→ The idea behind strassen's matrix multiplication is reducing no of multiplication ^{by} ~~is~~ increasing addition & subtraction of θ operations.

let $A, B, C \rightarrow \theta \times \theta$

$$A_{ij}, B_{ij}, C_{ij} \rightarrow \frac{\theta}{2} \times \frac{\theta}{2}$$

$$P, Q, R, S, T, U, V \rightarrow \frac{\theta}{2} \times \frac{\theta}{2}$$

$$P = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) \cdot B_{11}$$

$$R = (A_{11}) \cdot (B_{12} - B_{22})$$

$$S = (A_{22}) \cdot (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) \cdot (B_{22})$$

$$U = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$$

$$V = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

Here to compute $C_{11}, C_{12}, C_{21}, C_{22}$ we perform

7 multiplications & 18 add/sub

$$T(n) = \begin{cases} c, & n \leq 2 \quad (c > 0) \\ 7T(n/2) + bn^2, & n > 2 \end{cases}$$

$$T(n) = 7T(n/2) + bn^2$$

$$= 7(7T(n/2^2) + b(n/2)^2) + bn^2$$

$$= 7^2 T(n/2^2) + 7b(n/2)^2 + bn^2$$

$$= 7^2 [7T(n/2^3) + b(n/2^2)^2] + 7b(n/2)^2 + bn^2$$

$$= 7^3 T(n/2^3) + 7^2 (n/2^2)^2 + 7b(n/2)^2 + bn^2$$

$$\vdots$$

$$= 7^k T(n/2^k) + 7^{k-1} (n/2^{k-1})^2 + 7^{k-2} b(n/2)^2 + 7^0 b(n/2)^2$$

$$= 7^k T(n/2^k) + n^2 \left(\left(\frac{7}{2^2} \right)^{k-1} + \left(\frac{7}{2^2} \right)^{k-2} + \dots + 1 \right)$$

$$= 7^k T(n/2^k) + n^2 \left[1 + \frac{(7/4)^k - 1}{7/4 - 1} \right]$$

$$n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$

$$\Rightarrow 7^k = 7^{\log_2 n}$$

$$= 7^{\log_2 n} T(1) + \frac{4}{3} n^2 \left[(7/4)^{\log_2 n} - 1 \right]$$

$$= n^{\log_2 7} T(1) + \frac{4}{3} n^2 \left[n^{\log_2 7 - \log_2 4} - 1 \right]$$

$$= n^{\log_2 7} \cdot c + \frac{4}{3} n^2 \left[n^{\log_2 7 - 2} - 1 \right]$$

$$= cn^{\log_2 7} + \frac{4}{3} n^{\log_2 7} - 1$$

$$\therefore T(n) = O(\log n) \cdot O(n^{\log_2 7})$$

$$\Rightarrow T(n) = O(n^{2.81})$$

$$\therefore \text{Space Complexity, } S = O(n^2)$$

↳ space for matrices

P, B, R, S, T, U, V

Quick Sort [Partition - Exchange Sort]

→ Quick sort is based on the principle of partitioning (pivoting)

A:

1	2	3	4	5	6	7	8	9
65	70	75	80	85	60	55	50	45

Partitioning:

(i) Select a pivot (partitioning element)

(ii) Pivot the partitioning element at its correct place (fix)

(iii) All elements that are less than pivot has to be on left of the pivot.

(iv) All elements that are greater than pivot has to be on right of the pivot.

```
int PARTITION (A, l, h)
```

```
{
```

```
    i ← l; j ← h; // h is n+1
```

```
    v ← A[l]; // v contains partitioning element.
```

```
    loop
```

```
    { loop i ← i+1 until (A[i] ≥ v);
```

```
      loop j ← j-1 until (A[j] ≤ v);
```

```

if (i < j)
    SWAP(A[i], A[j]);

```

```

else
    break;
} while (false);
A[i] = A[j]; A[j] = v; return j;
} until (false); // position of pivot

```

(1)
 A: [65 70 75 80 85 60 55 50 45] ∞ (2)
 $\uparrow$ $\uparrow$
 i j

Having infinity here helps when pivot is the largest element.

```

v ← A[i]
v ← 65

```

$i \leftarrow i+1 \Rightarrow i=2$

$A[i] \geq v \Rightarrow A[2] \geq 65$

$\downarrow$
true $\therefore$ exit loop

Sometimes, if we don't use ∞ the program may go to infinite loop

$j \leftarrow j-1 \Rightarrow j=9$

$A[j] \leq v \Rightarrow A[9] \leq 65$

$\downarrow$
true $\therefore$ exit loop

if (i < j) .. true

SWAP(A[i], A[j])

$\therefore$ SWAP(A[2], A[9])

A: [65 45 75 80 85 60 55 50 70]

$\uparrow$
i

$\uparrow$
j

$i \leftarrow i+1 \Rightarrow i=3$

$A[3] \geq 65 \checkmark$

$j \leftarrow j-1 \Rightarrow j=8$

$A[8] \leq 65 \checkmark$

if (i < j) .. true

$\therefore$ SWAP(A[3], A[8])

next iteration

12/68 70 250 60 80 80 55 40 40

A: [65 45 60 80 85 60 55 75 70]

next iteration:

A: [65 45 50 55 85 60 80 75 70]
 ↑ ↑
 i j

next iteration:

A: [65 45 50 55 60 85 80 75 70]

$i \rightarrow j$ $j \rightarrow i$
 $i \rightarrow 5$ $j \rightarrow 6$

$$i \leq i+1 \Rightarrow i \boxed{6}$$
$$A[i] \geq 65 \quad \checkmark$$
$$j \leftarrow j - 1 \quad j \boxed{5}$$
$$A[j] \leq 465$$
$$60 \leq 65 \quad \checkmark$$

if ($i < j$) ... false

Now we need to put partitioning element in its position.

This position is present in j.

$A: [(60 \ 45 \ 50 \ 55) \boxed{65} (85 \ 80 \ 75 \ 70)]$

→ In quick sort, problem is said to be small if low is greater than or equal to high (i.e., no of elements are 1)

void QuickSort (A, l, h)

{
 if (l < h) // Problem is large

{

 m ← PARTITION(A, l, h);

 QuickSort (A, l, m-1);

 QuickSort (A, m+1, h);

}

}

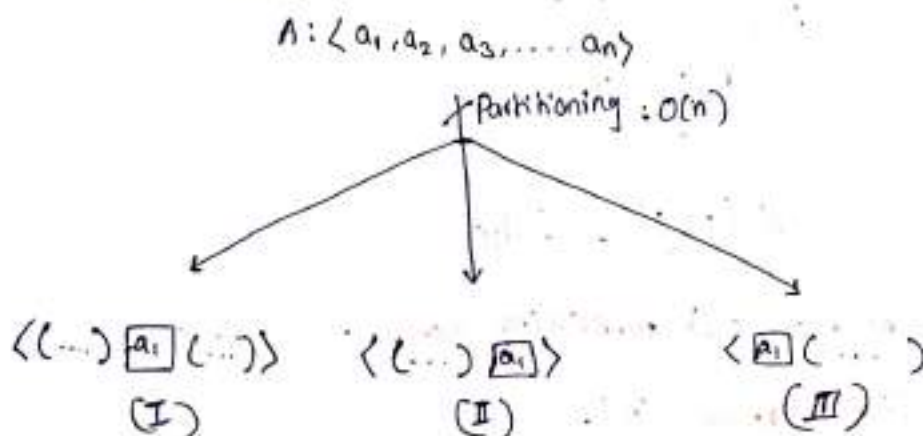
Analysis:

TC of PARTITION:

→ In both loops which are inside loop-until loop, we perform a total $n+1$ comparison b/w array elements and $v(\text{pivot})$.

∴ TC of partition = $O(n)$

TC of quicksort:



Case I:

$$T(n) = \begin{cases} c, & n=1 \\ 2T(n/2) + O(n), & n>1 \end{cases}$$

$$\Rightarrow T(n) = O(n \log n)$$

Case II & III:

$$T(n) = \begin{cases} c, & n=1 \\ T(n-1) + n, & n>1 \end{cases}$$

$$T(n) = O(n^2)$$

Note:

→ avg case TC of Quick sort = $O(n \log n)$

∴ for QS,

best case TC = avg case TC = $O(n \log n)$

worst case TC = $O(n^2)$

→ worst case of TC happens when elements are already sorted (i.e., either increasing or decreasing)

Space Complexity:

best case space complexity → $O(\log n)$

worst case space complexity → $O(n)$

↳ sorted list

H/76

Partitioning has to start choosing pivot element from one of the end elements of array.

So we swap central element with any other end elements of array & finally the worst case TC is $O(n^2)$

H/77

time to find median $O(n)$

~~to~~ time to partition w.r.to median $O(n)$

$\therefore$ total time for partitioning $O(n)$

Here every time pivot goes to middle of list and hence forms best case.

$\therefore O(n \log n)$

H/78

$$T(n) = O(n) + O(n) + T(n/4) + T(3n/4)$$

$$\Rightarrow T(n) = O(n \log n)$$

Note:

$$T(n) = T(\alpha n) + T((1-\alpha)n) + O(n), \quad 0 < \alpha < 1$$

$$\Rightarrow T(n) = O(n \log n)$$

H/79

~~T(n)~~

$$T(n) \leq T(n/5) + T(4n/5) + n$$

H/85

both are worst case

the question talks abt realistic time. ($\therefore$ the word program is given)
 $\therefore t_1 < t_2$

H/86

$t_1 \rightarrow$ worst case

$t_2 \rightarrow$ best case

$\therefore t_1 > t_2$

H/89

Both are worst case & they are symmetric.

$\therefore c_1 = c_2$

Master Theorem for solving Divide & Conquer Recurrences:

$$T(n) = aT(n/b) + f(n), \quad n > d \quad a \geq 1; b > 1, f(n) \text{ is true} \\ = c, \quad n \leq d$$

Case I:

If $f(n)$ is $O(n^{\log_b a - \epsilon})$, $\epsilon > 0$ then

$$T(n) = \Theta(n^{\log_b a})$$

Case II:

If $f(n)$ is $\Theta(n^{\log_b a} \cdot \log^k n)$ for some 'k'.

a) $k \geq 0$ then $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

b) $k = -1$ then $T(n) = \Theta(n^{\log_b a} \cdot \log \log n)$

c) $k < -1$ then $T(n) = \Theta(n^{\log_b a})$

Case III:

If $f(n)$ is $\Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$ and
 $a \cdot f(n/b) \leq \delta \cdot f(n)$ whenever $\delta < 1$ then

$$T(n) = \Theta(f(n))$$

Examples on Master theorem:

$$\rightarrow T(n) = 4T(n/2) + n$$

$$a=4; b=2; f(n)=n;$$

$$\log_2 4 = 2$$

Case (i): $n = O(n^{2-\epsilon})$

for $\epsilon > 0$

Say $\epsilon = 0.5$

the equation satisfies

$$\therefore T(n) = O(n^{\log_2 4})$$

$$= O(n^2)$$

$$\rightarrow T(n) = 2T(n/2) + n \log n$$

$$a=2 \quad b=2 \quad f(n)=n \log n$$

$$\log_a b = 1$$

Case (i): $n \log n = O(n^{\log_2 2 - \epsilon})$

$$n \log n = O(n^{1-\epsilon})$$

$\therefore$ for $\epsilon > 0$

the above equation is false

Case (ii):

$$n \log n = \theta(n \log^k n)$$

for $k=1$, the above equation is satisfied.

also $k \geq 0$

$$\therefore T(n) = \theta(n^{\log_a b} \cdot \log^{k+r} n)$$

$$= \theta(n^1 \cdot \log^2 n)$$

$$= \theta(n \cdot \log n \cdot \log n)$$

$$\rightarrow T(n) = T(n/3) + n$$

$$a=1, b=3; f(n)=n$$

$$\log_a b = \log_1 3 = \log_3 1 = 0$$

Case I: $n = O(n^{0-\epsilon})$

false

Case II: $n = \theta(n^{\log_a b} \log^k n)$

$$n = \theta(n^{\log_a b} \cdot \log^k n)$$

$$n = \theta(n^0 \cdot \log^k n)$$

$$n = \theta((\log n)^k) \quad \alpha$$

Case III: $n = \Omega(n^{\log_a b + \epsilon})$

$$n = \Omega(n^\epsilon), \quad \epsilon > 0$$

$$\forall \epsilon, 0 < \epsilon \leq 1$$

above eqn satisfies

$$\text{Now } af(n/b) \leq 8f(n), \quad 8 < 1$$

$$1. n/3 \leq 8 \cdot n$$

$$\frac{n}{3} \leq 8n \quad \text{⑥}$$

for ~~$\frac{1}{3} \leq 8$~~

for $\frac{1}{3} \leq 8 < 1$ the above inequality holds

$$\Rightarrow T(n) = \Theta(f(n))$$

$$\Rightarrow T(n) = \Theta(n)$$

$$\rightarrow T(n) = 9T(n/3) + n^{2.5}$$

$$a=9 \quad b=3 \quad f(n)=n^{2.5}$$

$$\log_b a = 2$$

Case I:

$$f(n) = n^{2.5} = O(n^{\log_b a + \epsilon})$$

$$n^{2.5} = O(n^{2+\epsilon}) \quad \checkmark$$

Case II:

$$f(n) = n^{2.5} = \Theta(n^{\log_b a} \log^k n)$$

$$n^{2.5} = \Theta(n^2 \log^k n)$$

for no k , the above
eqn will be satisfied ($\because \log^k n = O(n^\epsilon)$)
as 0)

Case III:

$$n^{2.5} = \Omega(n^{\log_b a + \epsilon})$$

$$= \Omega(n^{2+\epsilon}) \quad \checkmark$$

now consider

$$a \cdot f(n/b) \leq 8 \cdot f(n)$$

$$9 f(n/3) \leq 8 n^{2.5}$$

$$9 \cdot (n/3)^{2.5} \leq 8 n^{2.5}$$

$$\frac{n^{2.5}}{\sqrt{3}} \leq 8 n^{2.5}$$

$$\Rightarrow \frac{1}{\sqrt{3}} \leq 8$$

$$\Rightarrow 8 \geq \frac{1}{1.732}$$

$$8 \geq 0.577$$

$\therefore$ The above eqn satisfies for $8 < 1$

$$\therefore T(n) = \Theta(f(n))$$

$$= \Theta(n^{2.5})$$

$\rightarrow$ MaxMin :

$$T(n) = 2T(n/2) + 2$$

$$\text{Case (i)} : 2 = O(n^{\log_2 2 - \epsilon})$$

$$2 = O(n^{1-\epsilon})$$

true for $\epsilon = 1$

$$\therefore T(n) = O(n^{\log_2 2}) = O(n)$$

$\rightarrow$ MergeSort :

$$T(n) = 2T(n/2) + bn$$

$$\text{Case (i)} : bn = O(n^{1-\epsilon})$$

$$\text{Case (ii)} : bn = \Theta(n^k \log^k n)$$

true for $k > 0$

$$k \geq 0 \quad \therefore T(n) = \Theta(n^{\log_2 k} \cdot \log^{k+1} n) \dots$$

$$T(n) = \Theta(n \log n) \dots$$

→ Matrix multiplication:

$$T(n) = 8T(n/2) + n^2$$

$$n^2 = O(n^{3-\epsilon}) \checkmark$$

$$\therefore T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

Strassen's Matrix mul:

$$T(n) = 7T(n/2) + bn^2$$

$$bn^2 = O(n^{2.81-\epsilon}) \checkmark$$

$$\therefore T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

→ Binary Search:

$$T(n) = T(n/2) + c$$

$$i) \quad c = O(n^{\log_2 1 - \epsilon}) \dots$$

$$c = O(n^{-\epsilon}) \alpha$$

$$ii) \quad c = \Theta(n^0 \log^k n)$$

for $k=0$ eqn is satisfied

now $k \geq 0$

$$\Rightarrow T(n) = \Theta(n^{\log_2 1} \log^{k+1} n)$$

$$= \Theta(n^0 \log n)$$

$$\therefore T(n) = \Theta(\log n)$$

$$* \rightarrow T(n) = 2 \cdot T(\sqrt{n}) + \log n$$

Here we cannot directly apply master theorem.
so we perform appropriate transformations

$$\text{let } n = 2^k$$

$$\Rightarrow T(2^k) = 2 \cdot T(2^{k/2}) + k$$

$$\text{let } T(2^k) \rightarrow S(k)$$

$$S(k) = 2 \cdot S(k/2) + k$$

while transform only T
has to be changed to S .
Remaining has to be
kept same.

~~Part 2~~

$$\Rightarrow S(k) = \Theta(k \log k)$$

$$\Rightarrow T(n) = \Theta(\log n \cdot \log \log n)$$

*
*
*

$$T(n) = 2T(n/2) + \frac{n}{\log n}$$

$$\text{Case (i)}: \frac{n}{\log n} = O(n^{1-\epsilon})$$

$$\text{Case (ii)}: \frac{n}{\log n} = \Theta(n \cdot \log^k n)$$

$$\Rightarrow k > -1$$

$$\therefore T(n) = \Theta(n^{\log_a b} \cdot \log \log n)$$

$$\Rightarrow T(n) = \Theta(n \cdot \log \log n)$$

$$* \rightarrow T(n) = T(n/2) + T(n/4) + n$$

**

$$\therefore T(n/4) < T(n/2)$$

Recursion tree method gives $O(n)$

$$\Rightarrow T(n) < 2T(n/2) + n$$

$$\Rightarrow T(n) = O(n \log n)$$

Long Integer Multiplication:

We represent long integers using 1-D array

~~But~~ In general time for addition or multiplication is $O(1)$.

However representing with array ~~use~~ time complexity for addition or multiplication will not be $O(1)$.

$\rightarrow$ TC for add/sub of two long integers = $O(n)$

$\rightarrow$ TC for mul of two long integers = $O(n^2)$ (naive approach)

$\therefore$ for every digit of one number we multiply it with the other number.

DnC approach:

Let u, v be n digit numbers

$$m \leftarrow \left\lfloor \frac{n}{2} \right\rfloor$$

Now we divide u as follows into w & x ($2(n/2)$ integers)

$$u = w * 10^m + x$$

$$\text{where } w = u / 10^m$$

$$x = u \% 10^m$$

silly we divide v into 2 integers

$$v = y \cdot 10^m + z$$

$$y = v / 10^m$$

$$z = v \% 10^m$$

w, z, y, z are $n/2$ sized integers

$$\Rightarrow \text{Now } uv = (w \cdot 10^m + x)(y \cdot 10^m + z)$$

$$= (wy \cdot 10^{2m}) + (wz + zy) \cdot 10^m + xz$$



4 multiplications of size $n/2$

$$T(n) = \begin{cases} c & , n=1 \\ 4T(n/2) + bn & , n>1 \end{cases}$$

↓
additions

$$T(n) = 4T(n/2) + bn$$

Case i): $bn = O(n^{2-\epsilon})$ ✓

$$\therefore T(n) = O(n^2)$$

$\therefore$ time complexity $\rightarrow O(n^2)$

~~space complexity $O(n)$~~

Anatolii Karatsuba's Optimization for LIM:

$\rightarrow$ The ideology behind the optimization is reducing no. of multiplication & operations

$$uv = (wy \times 10^{2m}) + (wz + zy) \times 10^m + xz$$

Let t be an $n/2$ sized integer

$$t = (w+x)(y+z)$$

$$t = wy + \underline{wz + zy} + xz$$

$$\Rightarrow wz + zy = t - (wy + xz)$$

from this we define

$$P_1 = wy$$

$$P_2 = xz$$

$$P_3 = (w+x)(y+z)$$

$$\Rightarrow wz + zy = P_3 - (P_1 + P_2)$$

now we write

$$uv = (P_1 \times 10^{2m}) + (P_3 - (P_1 + P_2)) \times 10^m + P_2$$

This approach requires only 3 multiplication

However no of add/sub operations are increased.

$$\therefore T(n) = 3T(n/2) + O(n) \quad , n > 1$$

$$= C \quad , n = 1$$

$$\Rightarrow T(n) = \left(n^{\log_2 3} \right) = O(n^{1.58})$$

Toom & Cook Optimization for LIM:

→ This approach uses multi-way split.

3-way split:

$$m \leftarrow \lfloor n/3 \rfloor$$

~~$$u = a \times 10^{2m}$$~~

$$u = a \times 10^{2m} + b \times 10^m + c$$

$$v = d \times 10^{2m} + e \times 10^m + f$$

$$uv = (a \times 10^{2m} + b \times 10^m + c)(d \times 10^{2m} + e \times 10^m + f)$$

$$\Rightarrow \text{no. of multiplication req.} = 3^2 = 9$$

each on $n/3$ sized integers

$$\Rightarrow T(n) = 9T(n/3) + O(n), \quad n > 1$$
$$= c, \quad n \leq 1$$

$$\Rightarrow T(n) = O(n^2)$$

using anadolli's optimization we get

$$T(n) = 8T(n/3) + O(n)$$

In anadolli's optimization value of 'd' is always reduced by 1.

$$\Rightarrow T(n) = O(n^{1.89})$$

Ideology behind Toom & Cook optimization

let x be no of subproblems solved

$$\Rightarrow T(n) = xT(n/3) + O(n)$$

Now target is to have $T(n) < O(n^{1.58})$

$$\text{Here } T(n) = \cancel{O(n)} = O(n^{\log_3 x}) < O(n^{1.58})$$

$$\Rightarrow \log_3 x < 1.58$$

$$\Rightarrow x = 5$$

Now the conclusion is that if we solve only 5 subproblems then we can have TC which is less than $O(n^{1.58})$.

→ Toom & Cook were successful in devising equation in which we require to ~~solve only 5~~ perform only 5 multiplication operations.

$$\Rightarrow T(n) = 5T(n/3) + O(n)$$

$$\Rightarrow T(n) = O(n^{\log_3 5})$$

$$\Rightarrow T(n) = O(n^{1.46})$$

4-way split:

normal D&C approach

$$T(n) = 16T(n/4) + O(n)$$

Amortolli's optimization

$$T(n) = 15T(n/4) + O(n)$$

$$\Rightarrow T(n) = O(n^{1.9})$$

Master's Optimization:

$$\Rightarrow T(n) = xT(n/4) + O(n) = O(n^{\log_4 x}) < O(n^{1.46})$$

$$\Rightarrow T(n) =$$

$$\Rightarrow x = 7$$

$$\Rightarrow T(n) = 7T(n/4) + O(n)$$

$$\Rightarrow T(n) = O(n^{1.49})$$

k-way split:

$$\underline{DC}: T(n) = k^2 T(n/k) + O(n) : O(n^2)$$

$$\underline{AK}: T(n) = (k^2 - 1) T(n/k) + O(n) : O(n^{\log_k (k^2 - 1)})$$

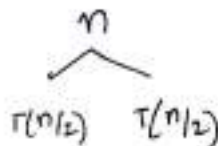
$$\underline{TKL}: T(n) = (2k - 1) T(n/k) + O(n) : O(n^{\log_k (2k - 1)})$$

07/10/20

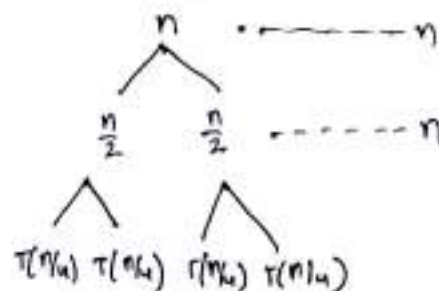
Solving Dand C Recurrences using Recursion tree method:

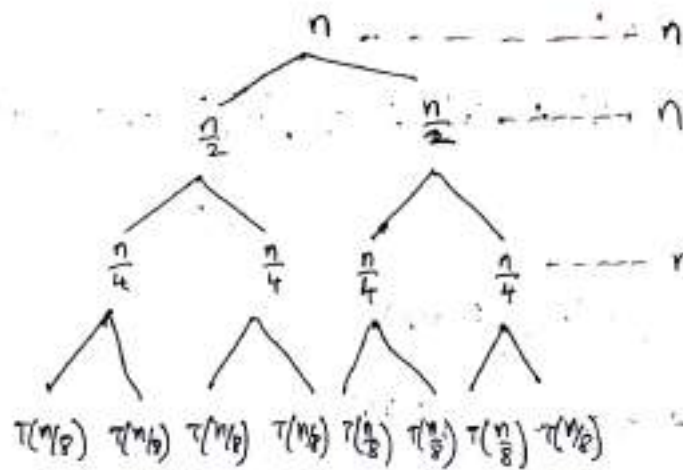
→ Recursion tree can be used to solve asymmetric recurrences.

$$E: T(n) = 2T(n/2) + n$$

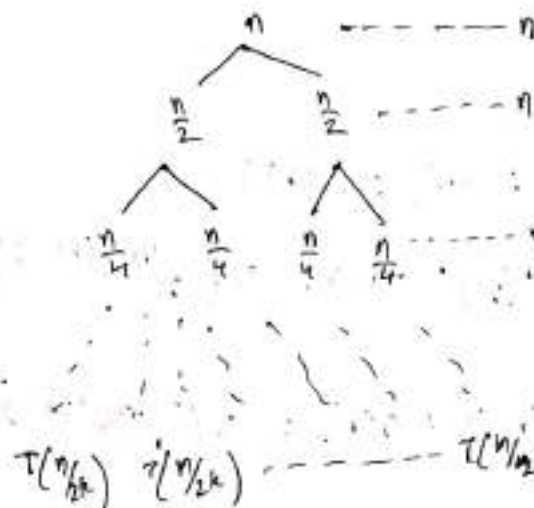


$$\Rightarrow T(n) = 2T(n/2) + n$$





At every level it is n .
So we continue to
max possible depth



This last level containing
 2^k subproblems.

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log n$$

no of ~~the~~ levels ~~above~~ are $\frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \dots, \frac{n}{2^{k-1}}, \frac{n}{2^k}$

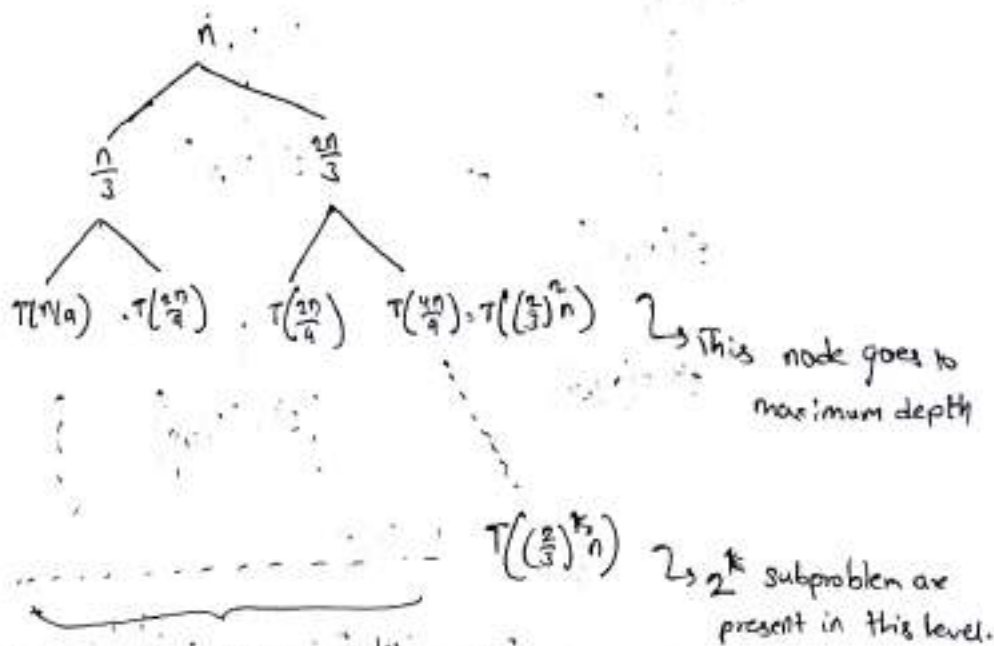
~~therefore~~ $\therefore k+1$ level

$$\therefore T(n) = n(k+1)$$

$$= n(\log n + 1)$$

$$\Rightarrow T(n) = O(n \log n)$$

eg: $T(n) = T(n/3) + T(2n/3) + n$; $T(1) = 1$



we assume all nodes have reach this depth and get an approximate answer

$$\therefore \left(\frac{2}{3}\right)^k n = 1$$

$$\log_{3/2} n = k$$

$$k = \log_{3/2} n$$

total of $k+1$ levels.

$$\therefore T(n) = n(k+1)$$

$$= n(\log_{3/2} n + 1)$$

$$\therefore T(n) = O(n \log_{3/2} n) = O(n \log n)$$

eg: $T(n) = 3T(n/4) + n^2$

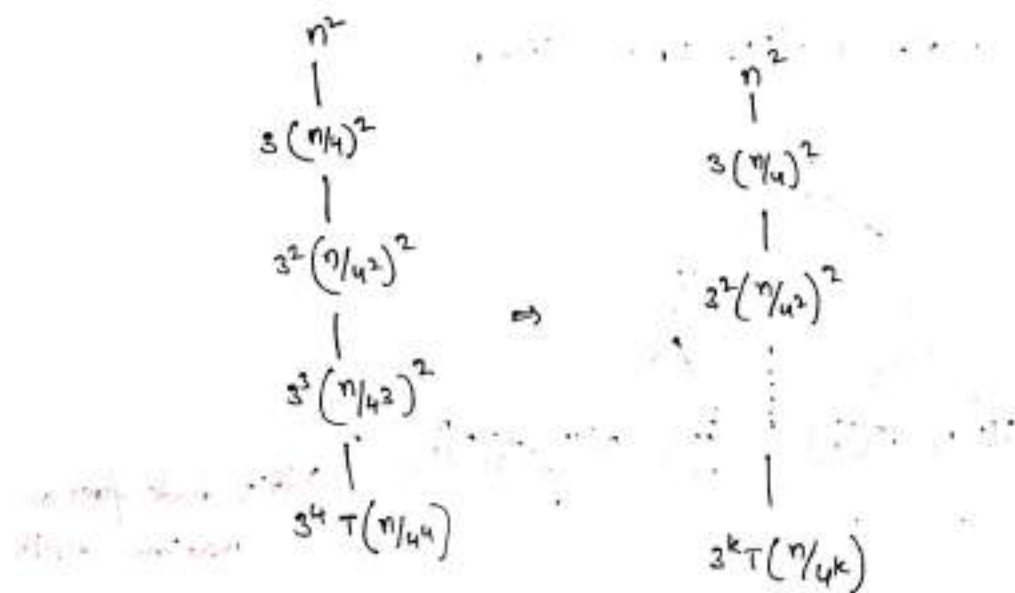
$$\begin{array}{c} n^2 \\ | \\ 3T(n/4) \end{array}$$

$\Rightarrow$

$$\begin{array}{c} n^2 \\ | \\ 3(n/4)^2 \\ | \\ 3^2 T(n/4^2) \end{array}$$

$\Rightarrow$

$$\begin{array}{c} T(n) = 3T(n/4) + (n/4)^2 \\ n^2 \\ | \\ 3(n/4)^2 \\ | \\ 3^2 (n/4^2)^2 \\ | \\ 3^3 T(n/4^3) \end{array}$$



$$\frac{n}{4^k} = 1 \Rightarrow k = \log_4 n$$

$$\therefore n^2 + \frac{3}{4^2} n^2 + \left(\frac{3}{4^2}\right)^2 n^2 + \left(\frac{3}{4^2}\right)^3 n^2 + \dots + \left(\frac{3}{4^2}\right)^{k-1} n^2 + 3^k$$

$$= n^2 \left(1 + \frac{3}{16} + \left(\frac{3}{16}\right)^2 + \dots + \left(\frac{3}{16}\right)^{k-1} \right) + 3^k$$

$$= n^2 \left(\frac{1}{1 - 3/16} \right) + 3^{\log_4 n}$$

$$= n^2 \left(\frac{16}{13} \right) + n^{\frac{\log_4 3}{4}}$$

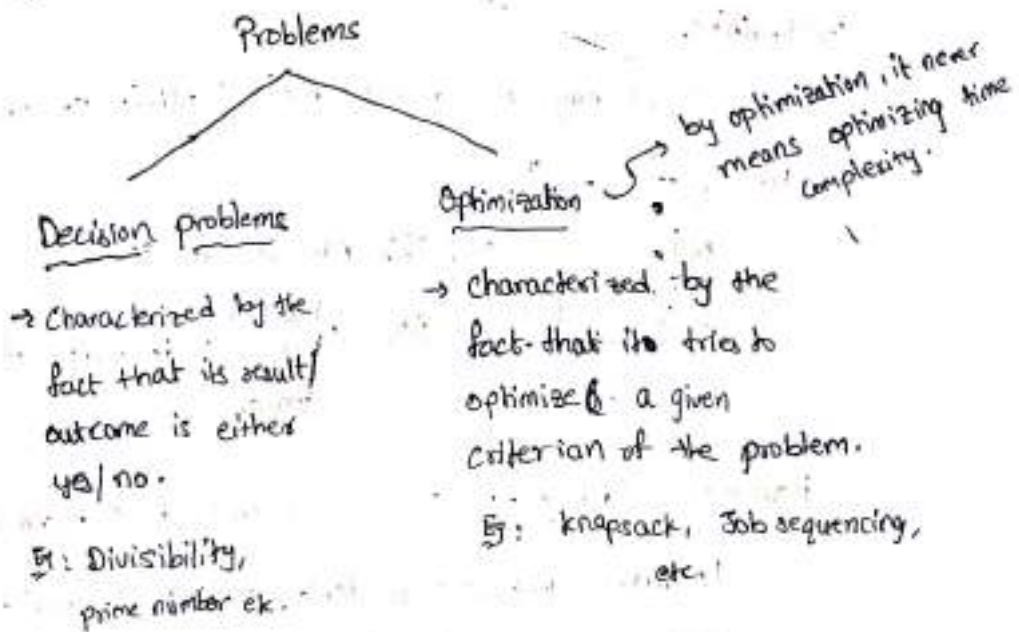
$$\Rightarrow T(n) = O(n^2)$$

$$\rightarrow T(n) = T(n/2) + T(n/4) + n$$

$$\underline{\text{Ans}}: O(n) \rightarrow \text{solve}$$

Greedy Method:

→ generally used for optimization problem.



Ex: Graph coloring → optimization problem

Travelling salesman → optimization problem

shortest path → optimization problem

n-Queens → Decision (whether queens can be placed in non-attacking position or not)

Sorting → Decision

Banker's algo → Decision

CPU scheduling → Optimization

Memory allocation → Optimization.

Terminology:

i) Problem Definition: Defining problem. Ex: n-queens
↳ we define what the problem is.

ii) Constraints

- Implicit (local constraints to the problem. Feasibility Criteria)
Ex: no two queens should in attacking positions
- Explicit (depends on instance of problem.
For example solving 4-queens, so array size is 4)

3. (iii) Solution Space:

All possible ways of organizing inputs but satisfying explicit constraints.

i.e., It may or may not meet implicit constraint.

Ex:

	1	2	3	4
q ₁	•			
q ₂	•			
q ₃	•			
q ₄	•			

⇒ This part of soln space

i.e., $\langle 1, 2, 3, 4 \rangle$ is part of soln space.

However $\langle 1, 3, 5, 2 \rangle$ is not a part of soln space

because 4-queens problem's explicit constraints is that queens could be placed in locations from 1 to 4.

∴ ~~5 is in~~ Having 5 doesn't meet explicit constraint.

(iv) So for n-queens problem's size of soln space = ~~n!~~ $n!$

If problem is constructing binary search tree with 'n' nodes,

then size of soln space = $\frac{1}{n+1} 2^{n+1} n$

→ Working with entire soln space is called brute force method.

(iv) Feasible solution:

The solutions in solutions space, that satisfy the implicit constraints of the problem are called feasible solutions.

Ex:

	1	2	3	4
q ₁		•		
q ₂				•
q ₃	•			
q ₄			•	

4-queens has 2 feasible solns

(v) Objective Function:

Refers to min/max of given criterion of the problem.

Problems may have objective function or may not have.

↳ n-queens problem
doesn't have objective
function.

Eg: Consider shortest path problem

→ soln space has all paths b/w any two vertices.

→ feasible soln has only those paths that connect source and destination vertex.

→ Here objective function is to minimize the cost of path.

(vi) Optimal Solution:

It is the feasible solution that satisfies objective

function.

→ optimal soln is always unique.

Note:

→ If a problem doesn't have objective function then we find only feasible soln.

→ Decision problems don't have objective function, hence for decision problems we find only feasible solutions.

→ Greedy method is an algorithm design strategy, used for solving problems, whose outcomes are seen as a result of making a sequence of decisions.

→ Greedy method makes these decision in a stepwise manner by applying the principle of local optimality (greedy choice property)

→ Principle of local optimality says that whatever the initial state & available options, greedily select that option that satisfies the objectivity function (this give feasible soln)

Eg: SJF algorithm, Shortest seek time First

i) knapsack Problem:

→ given a knapsack of capacity : M

→ given n -objects (O_i) & each object is associated with weight (w_i) and profit (p_i)

→ Maximizing the profit such that the total weight that put into knapsack should not exceed its capacity (M).

→ For every ^{object} i we make decision to find x_i (fraction of O_i put into knapsack)

$$\downarrow$$
$$0 \leq x_i \leq 1$$

for x_i the weight put into knapsack = $w_i x_i$
the profit object = $p_i x_i$

∴ we need to maximize $\sum_{i=1}^n p_i \cdot x_i$ such that

$$\sum_{i=1}^n w_i \cdot x_i \leq M$$

→ x_i can be real value

This type of knapsack problem is called

~~real knapsack~~
fractional knapsack
greedy knapsack

→ Size of soln space of real knapsack problem infinite.

→ for 0/1 knapsack the size of soln space is 2^n .

Consider below instance of knapsack problem.

$$n=3; m=20; \langle p_1, p_2, p_3 \rangle = \langle 25; 24; 15 \rangle$$

$$\langle w_1, w_2, w_3 \rangle = \langle 18; 15; 10 \rangle$$

$$\langle x_1, x_2, x_3 \rangle = ?$$

Explicit constraint
of knapsack problem is

$$\sum_{i=1}^n w_i > M$$

(i) Greedy about profit:

p_1 has highest profit

$$\Rightarrow x_1 = 1$$

$$\text{remaining capacity} = 20 - 18 = 2$$

p_2 has next highest profit

$$\Rightarrow x_2 = \frac{2}{15}$$

$$\text{here profit } P = p_1 x_1 + p_2 x_2 + p_3 x_3$$

$$= 25(1) + 24\left(\frac{2}{15}\right) = 28.8$$

(ii) Greedy about weight:

w_3 is least.

$$\therefore x_3 = 1$$

$$\text{remaining capacity} = 20 - 10 = 10$$

w_2 has next least weight

$$\Rightarrow x_2 = \frac{10}{15} = \frac{2}{3}$$

$$\therefore \text{profit} = p_1 x_1 + p_2 x_2 + p_3 x_3$$

$$= 0 + \frac{2}{3}(24) + 0(15)$$

$$= 31$$

(ii) Greedy about object that has highest profit per weight ratio:

$$\frac{P_1}{\omega_1} = \frac{25}{18} = 1.3 \Rightarrow K_1 > 0$$

$$\frac{p_2}{w_2} = 24/15 = 1.6 \Rightarrow x_2 = 1$$

$$\frac{p_3}{w_3 p_3} = 15/10 = 1.5 \Rightarrow r_3 = \frac{5}{10}$$

Profit $P = P_1x_1 + P_2x_2 + P_3x_3$

$$= 24 \cancel{P} 0 + 24(1) + 15\left(\frac{5}{20}\right)$$

$$= 24 + 7.5 = 31.5$$

→ 3rd approach is optimal. ϕ

→ So, arrange the objects in decreasing order of the ratio & keep on including the objects, until a k^{th} object cannot be placed and for this object we take proportionate fraction.

Time complexity after sorting $\rightarrow O(n)$

Time Complexity including sorting $\rightarrow O(n \log n)$

Ex: $n=7$ $m=15$; $\langle P_i - P_j \rangle = \langle 10, 5, 15, 7, 6, 18, 3 \rangle$

$$\langle w_r, w_q \rangle = \langle 2, 3, 5, 7, 1, 4, 1 \rangle$$

$$\langle \frac{P_1}{w_1}, \frac{P_2}{w_2} \rangle = \langle 5, 1.66, 2, 1, 6, 4.5, 3 \rangle$$

7) 8

Diagram showing five vertical lines with arrows pointing down. The lines are labeled with values: $x_1=1$, $x_3=1$, $x_5=1$, $x_7=1$, and $x_9=1$. Below the first line, there is a calculation: $x_2 = 7/3$.

$$P = 10 + \frac{2}{3}(5) + 15 + 0 + 6 + 18 + 3$$

$$\approx 55.33$$

(H/86)

It is clear the optimal profit is 60.

$$\langle w_i \rangle : \langle 10, 7, 4, 2 \rangle$$

$$\langle P_i \rangle : \langle 60, 28, 20, 24 \rangle$$

$$\langle \frac{P_i}{w_i} \rangle : \langle 6, 4, 5, 12 \rangle$$

② ④ ③ ①

Select $O_1 \Rightarrow$ remaining weight = $11 - 2 = 9$ $P = 24$ Select $O_2 \Rightarrow w_2 > 9$ $\therefore$ not possible.Select $O_3 \Rightarrow$ remaining weight = $9 - 4 = 5 \Rightarrow P = 24 + 20 = 44$ Select $O_4 \Rightarrow w_4 > 5$ $\therefore$ not possible.

$$\therefore V_{opt} = 60$$

$$V_{greedy} = 44$$

 $\therefore$ Ans: 16

Job Sequencing with Deadlines (JSD)

$\rightarrow$ n-jobs / programs / Process

$\rightarrow$ Each job (J_i) arrives at time 0 and needs one unit time.

$\rightarrow$ Every job is associated with a deadline d_i & profit P_i . The profit is obtained only if the job is completed within its deadline.

Problem stmt:

select a subset of given n-jobs such that the jobs in the subset are completable within their deadlines and generate maximum profit.

→ Here the trivial case is when all the jobs can be scheduled.

→ size of soln space is 2^n .

It means working out this problem using brute force, the time complexity is $O(2^n)$.

Eg: $n=4$; (J_1, J_4) ;

$$(P_1, P_4) = \langle 100, 15, 10, 40 \rangle$$

$$\langle d_1, d_2, d_3, d_4 \rangle = \langle 2, 1, 2, 1 \rangle$$

Let J be soln subset. Initially $J = \emptyset$
↳ feasible soln

I. $|J|=0$

feasible soln

profit, $P=0$

II: $|J|=1$

$$J = \{J_1\}$$

feasible soln; $P=100$

$$J = \{J_2\}$$

feasible soln; $P=15$

III: $|J|=2$

$$J = \{J_1, J_2\}$$

J_2	J_1
-------	-------

feasible, soln

profit $\rightarrow 100+15=115$

schedule: J_2, J_1

$$J = \{J_1, J_3\}$$

this is not a feasible soln

Profit = 110

schedule: J_1, J_3 (or) J_3, J_1

feasible soln for this problem are those subsets having all the jobs which can be scheduled within the deadline (implicit constraint)

$$J = \{J_2, J_4\}$$

This is not a feasible soln.

$\therefore$ we cannot schedule J_2 & J_4 within their deadlines.

$$|J| = 3$$

no three jobs can be scheduled.

$\therefore$ there is not subset feasible soln with $|J| = 3$.

Note:

The maximum no of jobs that a feasible subset can have = maximum deadline.

(This rule applies only when each job takes exactly 1 unit of time)

Greedy approach for JSD:

$$n = 4; \langle J_1, J_4 \rangle; \langle P_1, P_4 \rangle = \langle 100, 15, 10, 40 \rangle; \langle d_1, d_4 \rangle = \langle 2, 1, 2, 1 \rangle$$

Let J be the soln subset

$\rightarrow$ The job with highest profit will always be part of optimal soln

Step 1: Arrange the jobs in decreasing order of profits.

$$\text{ie, } j_1, j_4, j_2, j_3$$

one by one

schedule the first jobs of sorted order such that the job goes into maximum possible slot below its deadline.

$\therefore$ Pick J_1 :

deadline is 2

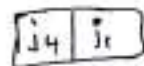
slot 2 is free



Pick j_1 :

slot deadline $\rightarrow 1$

slot '1' is empty.



pick j_2 :

slot 1 is not empty.

pick j_3 :

slot 2 is not empty. Now check slot 1.

slot 1 is not empty.

$\therefore$ final schedule:

j_4	j_3
0	1 2

$$\therefore \text{profit} = 100 + 40 = 140$$

Eg: $n=7$

$\langle s_1-s_7 \rangle$:

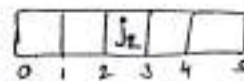
$\langle d_1-d_7 \rangle = \langle 2, 3, 4, 5, 2, 4, 4 \rangle$

$\langle p_1-p_7 \rangle = \langle 54, 92, 30, 10, 15, 60, 45 \rangle$

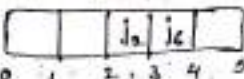
decreasing order of profits

$j_2, j_6, j_1, j_7, j_3, j_5, j_4$

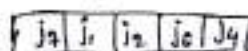
pick j_2 :



pick j_6 :



pick j_1 :



$$\text{profit} = 54 + 92 + 10 + 60 + 45$$

$$= 261$$

Q: $n=6$; $\langle j_1, j_6 \rangle$;

$$\langle d_1, d_6 \rangle = \langle 4, 4, 3, 4, 2, 3 \rangle$$

$$\langle p_1, p_6 \rangle = \langle 105, 38, 46, 150, 20, 19 \rangle$$

j_2	j_3	j_1	j_4	
0	1	2	3	4

$$P = 105 + 38 + 46 + 150 = 339$$

H/37

a)

T_2	T_4	T_9	T_5	T_3	T_1	T_8	
0	1	2	3	4	5	6	7

∴ opt (d)

b) $P = 15 + 20 + 30 + 18 + 23 + 16 + 25 = 147$

Control Abstraction of Greedy method

Procedure GREEDY(A, n)

{ $\hookrightarrow$ if p size

1. solution $\leftarrow \emptyset$

2. for $i \leftarrow 1$ to n

{ $x \leftarrow \text{SELECT}(A);$

if (FEASIBLE(x, solution)) // checks whether including x in soln is feasible or not.
ADD(x, solution);

}

3. return (solution);

}

From this control abstraction,

we say minimum time complexity is $O(n)$

i.e., when, $SELECT()$, $FEASIBLE(S)$, $ADD()$ are $O(1)$

High-level Pseudocode for JSD:

Algorithm JSD(d, p, J, n)

// we assume that jobs are arranged in decreasing order of profits

// $p_1 \geq p_2 \geq p_3 \dots \geq p_n$

{

1. $J \leftarrow \{1\};$

2. for $i \leftarrow 2$ to $n \rightarrow O(n)$

{

if (all job in $J \cup \{i\}$ can be completed by their $\rightarrow O(n)$
deadlines) then

$J \leftarrow J \cup \{i\};$

}

}

Time Complexity of JSD: $O(n^2)$

→ The working of JSD is similar to the worst case behaviour of insertion sort.

Optimal Merge Patterns

- Merging of files: Files have records in sorted order.
- Given n files, it is required to merge them using 2-way merging to get a single sorted file with an objective of minimizing the total no of record movements.

Ex: Consider merging two file A, B with 5, 3 records respectively

~~A = <1, 2, 3, 4, 5>~~

A = <1, 3, 4, 6, 8>

B = <2, 5, 7>

<1, 2, 3, 4, 5, 6, 7, 8>

5+3 = 8

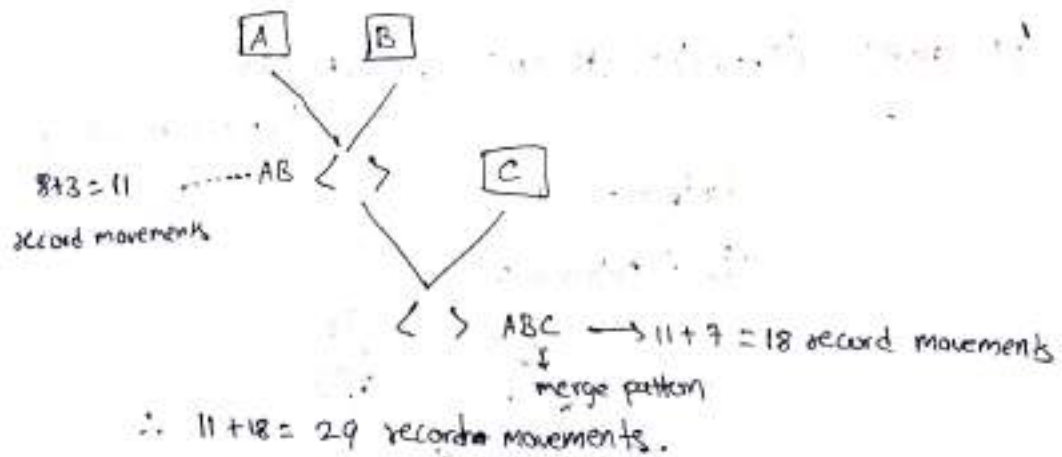
record movements

Ex: $n=3$

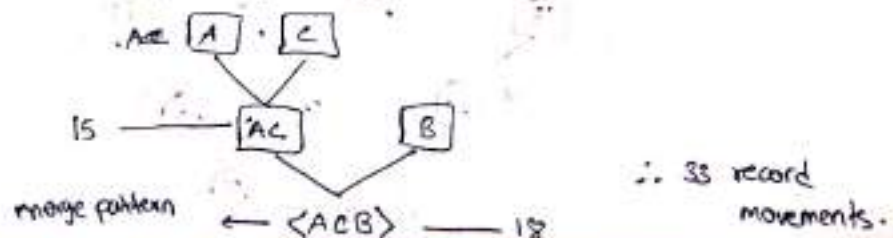
$\langle A, B, C \rangle = \langle 8, 3, 7 \rangle$

↳ no of records

→ Consider we merge A, B first



→ Now consider we merge A, C first



→ Here size of soln space = $n!$

∴ TC with brute force = $O(n!)$

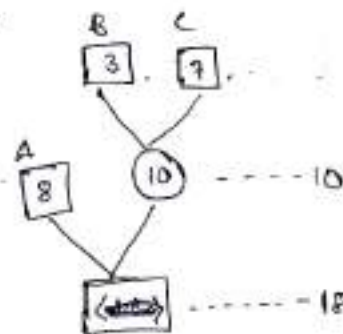
→ Also here every soln in the soln space is a feasible soln.

Greedy strategy for OHP

At each step (of every $(n-1)$ steps), select the two files with

least no of records, merge them and add to the list.

$n=3$; $\langle A, B, C \rangle = \langle 8, 3, 7 \rangle$



2-way
→ Optimal binary merge tree
(reverse of the figure)

∴ Min record movements = $10 + 18 = 28$

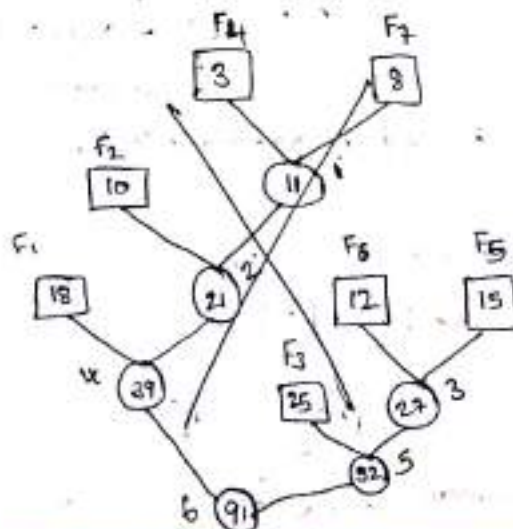
optimal merge pattern = $\langle B, C, A \rangle$

Ex: $n=7$; $\langle F_1 - F_7 \rangle = \langle 18, 10, 25, 3, 15, 12, 8 \rangle$

↳ no. of records per each file

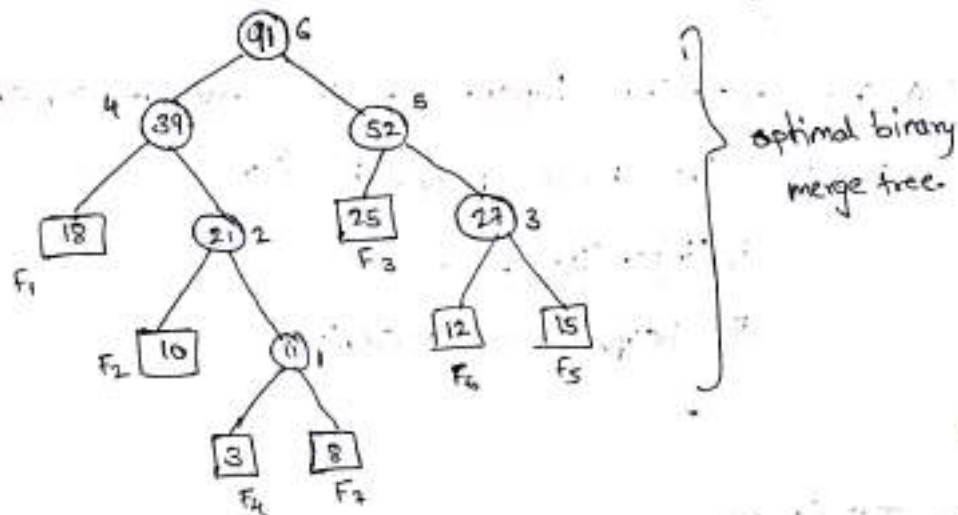
~~$\langle E_4, E_7 \rangle$~~ 11

~~$\langle \langle F_4, F_7 \rangle, F_2 \rangle$~~ 21



$$\therefore \text{no of record movements} = 11 + 21 + 39 + 27 + 52 + 91$$

$$= 241$$



→ Sum of internal nodes gives min. no. of record movements.

note:

Total no. of record movements involved during OMP is given by weighted external path length.

$$\text{i.e., } \sum_{i=1}^n q_i * d_i$$

$d_i \rightarrow$ distance from root to F_i

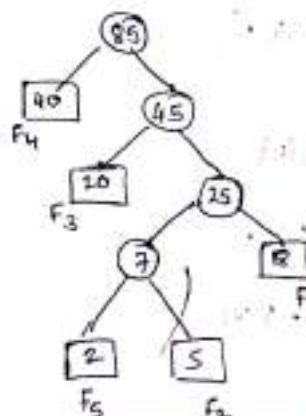
$q_i \rightarrow$ size of F_i

$$= (2 \times 18) + (3 \times 10) + (2 \times 25) + (4 \times 3) + (3 \times 15) + (3 \times 12) + (4 \times 8)$$

$$= 36 + 30 + 50 + 12 + 45 + 36 + 32$$

$$= 241$$

Ex: $n=5; \langle F_1, F_2 \rangle = \langle 8, 5, 20, 40, 2 \rangle$



Min record movement

$$= 85 F_{45} + 25 + 7$$

$$= 162$$

Performance Analysis:

Time Complexity:

→ Time complexity depends on the way we implement or store the no of record in files

(i) linked list $\rightarrow O(n^2)$
(or array)

(ii) Heap $\rightarrow O(n \log n)$

11/10/20

Algo JSD(L, n)

// L is a list of n single node binary tree as described below

```
{
  1. For  $i \leftarrow 1$  to  $n-1$  do
  2. call GETNODE( $T$ ) // creates a node  $T$ 
  3.  $LCHILD(T) \leftarrow LEAST(L)$ 
  4.  $RCHILD(T) \leftarrow LEAST(L)$ 
  5.  $WEIGHT(T) \leftarrow WEIGHT(LCHILD(T)) + WEIGHT(RCHILD(T))$ 
  6. call INSERT( $L, T$ )
  7. repeat
  8. return ( $LEAST(L)$ )
}
```

Here time complexity depends on the way we implement list L .

(i) If L is ordered linear list

$LEAST(L) \rightarrow O(1)$

$INSERT(L, T) \rightarrow O(n)$

(ii) If L is unordered linear list

$LEAST(L) \rightarrow O(n)$

$INSERT(L, T) \rightarrow O(1)$

Thus if the algorithm is implemented with a list

time complexity $\rightarrow O(n^2)$

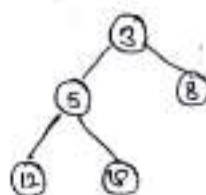
Space Complexity $\rightarrow O(n)$

$\hookrightarrow$ for tree created by nodes T

(iii) If L is implemented as a heap (min heap)

first we build the heap with $O(n)$ time

$\langle F_1, F_5 \rangle = \langle 5, 12, 8, 3, 18 \rangle$



LEAST(L) takes $O(\log n)$ time

INSERT(L, T) takes $O(\log n)$ time

$\Rightarrow$ Time complexity $\rightarrow O(n \log n)$

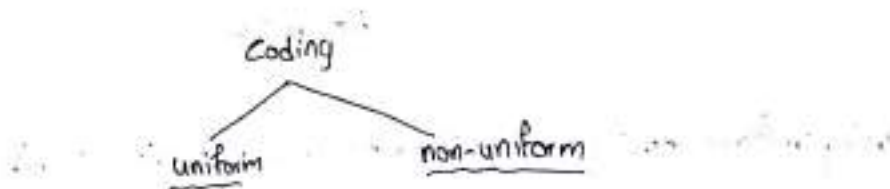
Space Complexity $\rightarrow O(n)$

$\hookrightarrow$ space for heap

Huffman Coding

$\rightarrow$ It is an application of optimal merge pattern

$\rightarrow$ Huffman coding is a data encoding technique.



$\rightarrow$ Each character is represented with equal no of bits

Ex: ASCII

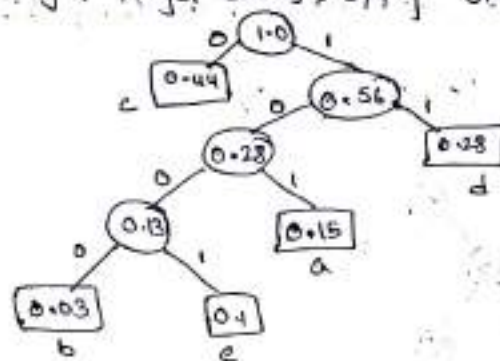
$\rightarrow$ Characters may be represented with different no of bits.

→ Huffman coding is a non-uniform coding.

→ Characters/elements having higher frequencies/probabilities of occurrences must be encoded with less no of bits.

Eg : $L = \langle a, b, c, d, e \rangle = \langle 0.15; 0.03; 0.44; 0.28; 0.1 \rangle$

Huffman coding : to get codes, apply OMP algorithm:



Mark every left edge as 0 and right edge as 1
(reverse can also be followed)

⇒ a → 101

b → 1000

c → 0

d → 11

e → 1001

Now consider the text ccdcccdade

then corresponding binary stream is 00110001110111001

i.e., 18 bits

using uniform coding every character would require 3 bits
and hence the text would require 30 bits.

Ex: Consider binary stream 101010111

Find text considering previous example

Sol:

for i/p traverse the tree until you reach the leaf.

After reach leaf, print that character and traverse again

from the root using remaining i/p.

In above stream after reading 101 we arrive at leaf 'a'

after reading 0 we arrive at leaf 'e'.

:

$\therefore$ 101010111 $\rightarrow$ a c d e d d

$\rightarrow$ Find avg no of bits used per character in Huffman coding.

$$(0.15)(3) + (0.8)(4) + (0.44)(1) + (0.28)(2) + (0.1)(4) = 1.97 \text{ bits}$$

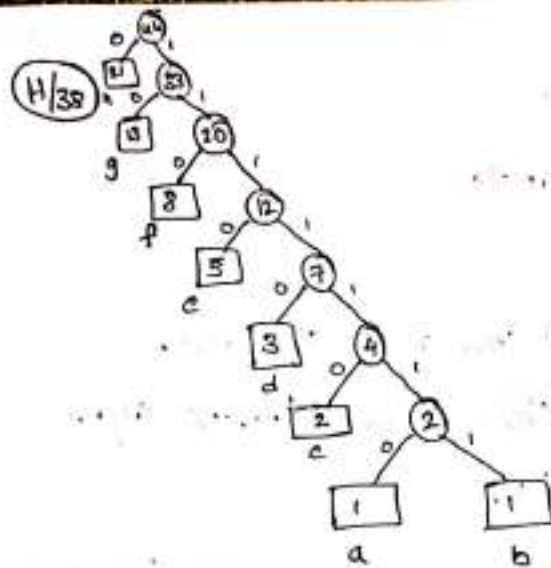
$$\text{i.e., } \sum_{i=1}^n d_i q_i$$

i.e., avg no of bits is given by ~~external~~ weighted external path

length = sum of internal nodes.

$\rightarrow$ Disadvantage of Huffman coding is that even a single bit error can make the whole text corrupted.

$\rightarrow$ If there are 2^n characters and if each character has equal probability (i.e., $\frac{1}{2^n}$) then ^{avg} no of bits req. in Huffman coding is n . Also the tree formed will be a complete binary tree with all the ^{Character} nodes at leaf position.



1101110011010
f d h e j

$\therefore f d h e j$

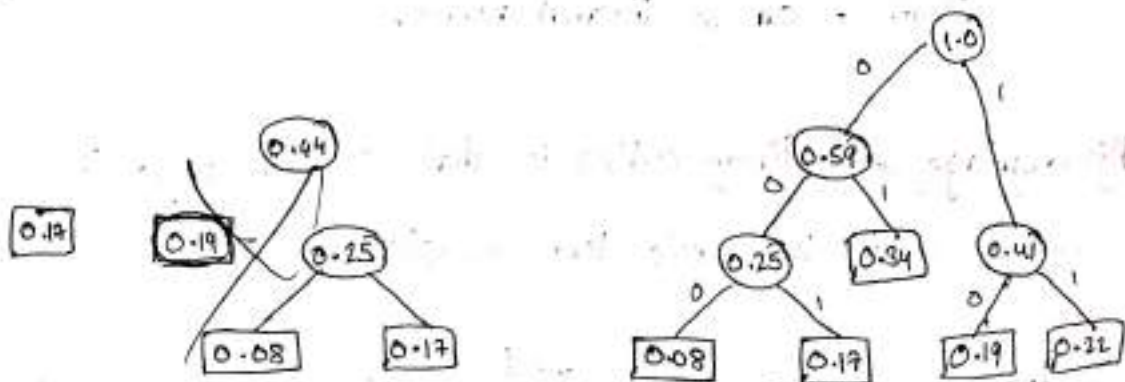
Find avg no of characters bits per characters.

$$= \frac{\sum l_i d_i}{\sum l_i \rightarrow \text{frequencies}}$$

$$= \frac{\text{sum of internal nodes}}{1+1+2+3+5+8+13+21}$$

$$= \frac{122}{54}$$

H/39



$$\text{avg no of bits} = 3(0.08) + 3(0.17) + 2[0.34 + 0.19 + 0.22]$$

$$= 0.75 + 2(0.75)$$

$$= 1.5 \text{ bits} = 2.25 \text{ bits}$$

$\therefore 2.25 \text{ bits}$

Spanning Trees:

$$G = (V, E); \quad |V| = n; \quad |E| = e;$$

G is weight undirected graph

Spanning tree: A subgraph $T(V, E')$ of a given graph $G = (V, E)$

where $E' \subseteq E$ is a spanning tree iff T is

connected acyclic graph (tree)

solution space:

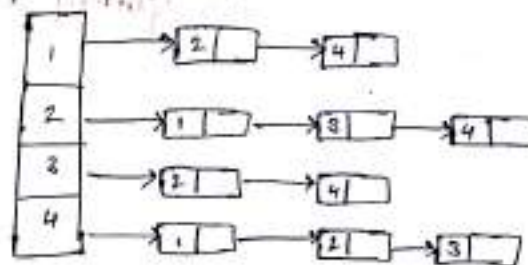
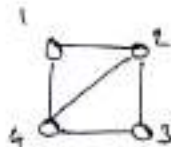
i.e., Maximum no of spanning trees = n^{n-2} Kaley's formula
for a complete graph of n vertices.

~~→ No of spanning trees possible for any graph is given by cofactor of any element of the matrix representing the graph~~

Note:

→ Any graph based problem that use adjacency matrix or Cost adjacency matrix has time complexity no less than $O(n^2)$

→



undirected graph $\rightarrow n+2e$ nodes.

directed graph $\rightarrow n+e$ nodes

→ Thus any graph problem which represent graph with adjacency list has atleast $O(n+e)$

→ Max value possible for e is $O(n^2)$

↳ In some cases $O(n^2) = O(n^2)$

∴ For complete graphs/dense graphs, adjacency matrix representation is desirable.

For sparse graphs 'adj. list representation' is desirable.

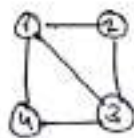
Calculating no of spanning trees for a non-complete graph:

i) Create adjacency matrix for given graph.

ii) Replace all diagonal elements with degrees of node and replace all non-diagonal 1's (edges) with -1.

iii) Now calculate co-factor of any element of the matrix and this gives no of spanning trees.

Ex:

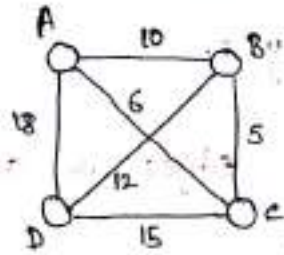


$$\rightarrow \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}$$

$$\text{Cofactor of 3} = \begin{vmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{vmatrix} = 8$$

∴ 8 spanning trees.

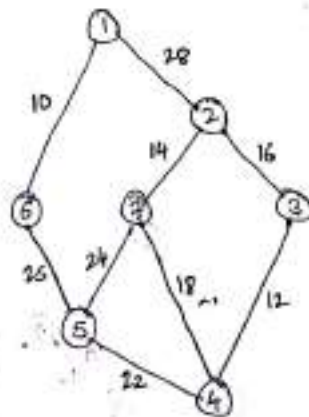
Minimum cost spanning Tree (MCST):



Applications of spanning tree:

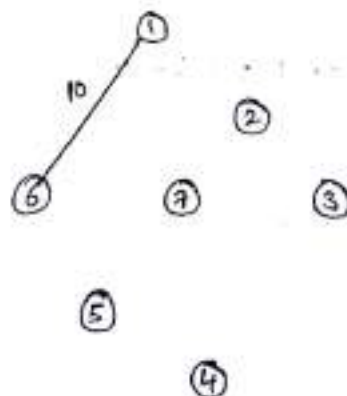
- Used in multicast & broadcast
- MCST is used in electronic & electrical applications (in circuits)

Construction of MCST:



Prims Algorithm:

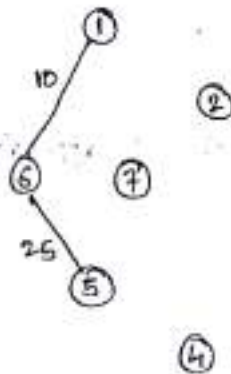
- 1) choose the least cost edge and add to the solution.



(ii) The next edge to be added, must be adjacent from to the vertices included so far such that it has least cost and it does not form a cycle.

Repeat step (ii) until a total $(n-1)$ edges are added

So add edge 1-6

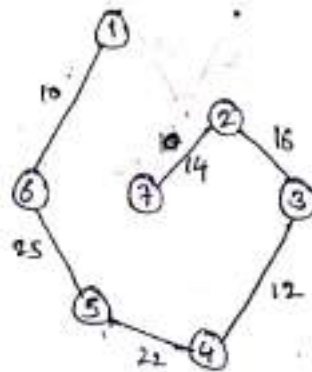


Add 5-4 ... 22

Add 4-3 ... 12

Add 2-3 ... 16

Add 2-7 ... 14



6 edges are added and we finish the process here

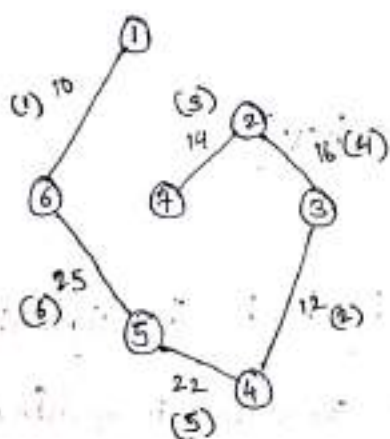
$\therefore$ ~~Min cost~~ =

$$\begin{aligned}\text{Cost of MCST} &= 10 + 25 + 22 + 12 + 16 + 14 \\ &= 99\end{aligned}$$

Kruskal's algorithm

1. Arrange the edges of the graph in increasing order. (min heap)
2. Delete the least cost edge from the list. (delete root of min heap)
3. Add the edge to the soln iff it does not lead to cycle.
4. Repeat (2) & (3) until a total of $(n-1)$ edges are added.

edges $\rightarrow 10, 12, 14, 16, 18, 22, 24, 25, 28$



Add 10

Add 12

Add 14

Add 16

Adding 18 leads to a cycle

Add 22

Add 24 forms a cycle

Add 25

The numbering shows the order in which the edges are added.

$\rightarrow$ At the time of construction ^{edges} ~~elements~~ 18, 24 are discarded, whereas 28 is not considered.

Difference b/w Prim's & Kruskal's algorithms

(i) Prim's method always maintains tree structured property at every step (correct connectedness) whereas Kruskal's algorithm may or may not.

(ii) Time complexities:

Prim's algorithm:

- adjacency matrix & linear list : $O(n^2)$
- adjacency list & heap : $O((n+e)\log n)$

Kruskal's algorithm

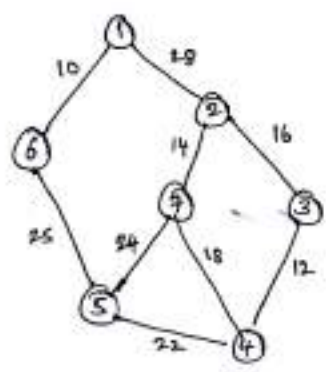
$O(e \log e)$ [using heap]

(iii) The cost of MST generated by both the algorithms is same. However, the tree structure may differ.

* The tree structure may differ when there are multiple edges of same weight. (or) ~~then~~
 ~~there are two or more spanning trees with same minimum cost~~

* Tree structure generated by Prim's and Kruskal will be same if all the edge costs are distinct. ~~and there is only one spanning tree with minimum cost.~~

Dijkstra's algorithm:

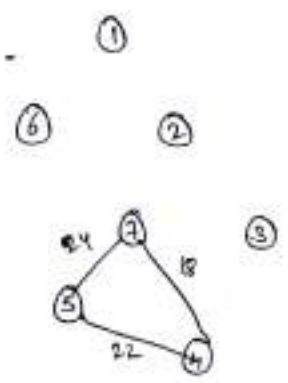


(This algorithm doesn't come totally under greedy)

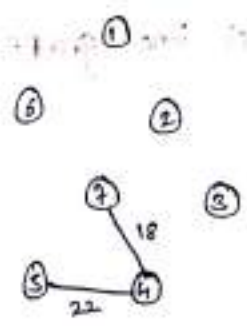
Let's

1. Starting adding any edge, and continue adding ^{any} edges one by one until a cycle is formed. Once a cycle is formed remove the maximum cost edge from the cycle. Repeat the above step until $n-1$ edges are added or all the edges are exhausted. (note that no edge should be added 2nd time after its removal)

Eg: Consider adding 24, 22, 18.



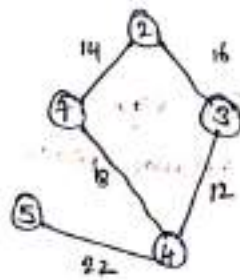
Remove 24 from cycle.



Add 12, 16, 14

①

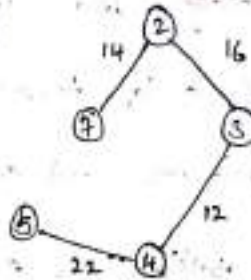
⑥



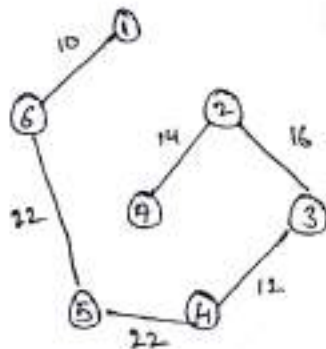
Remove 18

①

⑥



Add 25, 10



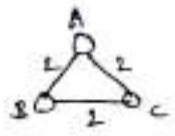
6 edges are added

∴ stop the process

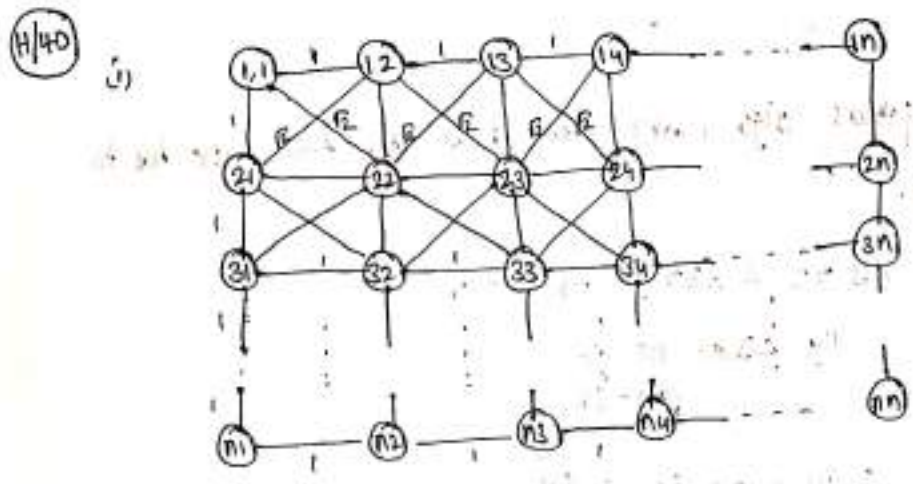
Calculating no of MCSTs:

→ If all edges costs are distinct the graph will have a unique MCST.

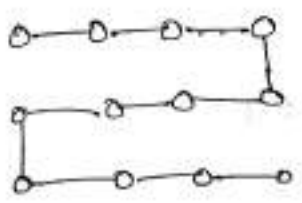
→ If there exist multiple edges of same cost then there may exist more than one MCST with same cost.

eg:  No of MCSTs possible = 3

Note: There is no standard formula for calculating no of MCSTs. we just need to calculate from the graph given using permutations & combinations.

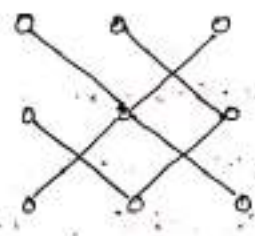


So the min cost spanning tree is



∴ Cost is $(n-1)(n) + (n-1) = (n+1)(n-1) = n^2 - 1$

(ii) Find cost of max cost spanning tree.
take $n=3$



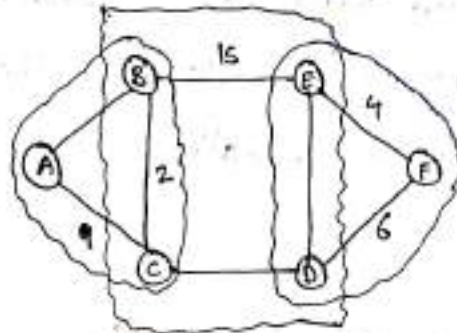
we have added 7 ~~add~~ edges to each of cost R_2 .

Now we can no more add any edge of ~~cost~~ R_2 .

$\therefore$ we add one edge of cost 1.

$\therefore$ Cost is $(n-2)(R_2) + 1$

(H/41)



using Dijkstra's algo, wkt that max wt. from cycle has to be removed.

$\therefore AB > 2$ & $AB > 9 \Rightarrow AB = 10$

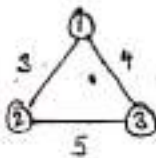
slly ~~CD~~ $DE = 7$

$ED = 16$

$\therefore 36 + 10 + 16 + 7 = 69$

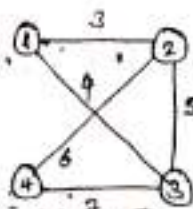
(H/42)

$n=3$



i.e., $3+4=7$

$n=4$



i.e., $3+4+6$

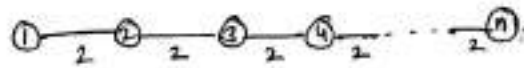
$\therefore$ the sequence is $3+4+6+8+10+\dots$

i.e., $3+2(2)+2(3)+\dots+2(n-1)$

$\Rightarrow 3-2+2[1+\dots+(n-1)] = 1+2\frac{n(n-1)}{2} = n^2-n+1$

H/43

MST is

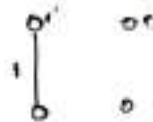


$$\therefore 2(n-1)$$

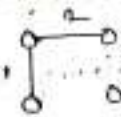
H/44

{1, 2, 3, 4, 5, 6}

'1' is always part of MST



Using Kruskal's algorithm, next we add 2 and this doesn't lead to cycle. No matter where the edge is.



Choosing '3' in first case it could be forming a cycle. So '4' will be part of MST.



Min cost

$$\text{Max possible cost of MST} = 1 + 2 + 4 = 7$$

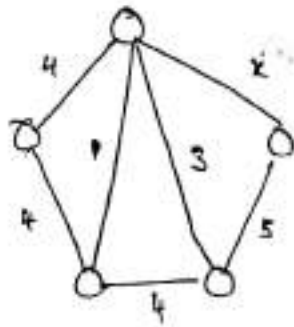
H/45

After modification,

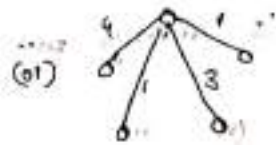
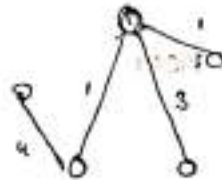
MST's structure remains same with weights increased.

$$\therefore 500 + 99(5) = 995$$

H/46



$x=1 \Rightarrow$



Silly for $x=2$

for $x=2$, 2 MCSTs.

for $x=3$, 2 MCSTs

for $x=4$, 2 MCSTs

for $x=5$



$\therefore 2 \times 2 = 4$ ways

For this kind of problems use dijkstra's algo.

Having $x=5$ will add 2 possibilities of removing max. cost edge from its corresponding cycle

H/47

i) ~~false~~ for w -regular graph.

ii) ~~etc~~ true

iii) True, since we can start construction of MCST for that ^{edge} (kruskal's algo)

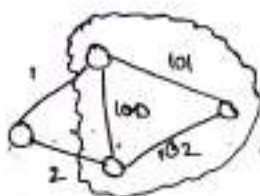
iv) true

v) False in the case of w -regular graph

vi) true

H/48

I.



'100' is lightest, still it is excluded cuz it is heaviest of another cycle

∴ false

II. true from Dijkstra's algo

∴ true

12/10/20

Algo PRIM ($E, \text{cost}, n, T, \text{mincost}$)

// E is set of edges in G

// $\text{cost}(n,n)$ is cost adjacency matrix such that $\text{cost}(i,j)$ is either positive or ∞ if there is no edge b/w i,j

// $T[1..n-1, 1..2]$ is store set of edges of MST

{

1. real $\text{cost}(n,n), \text{mincost};$

2. integer $\text{NEAR}(n), n, i, j, k, l, T(n-1, 2);$

3. $(k, l) \leftarrow \text{edge with min cost} \dots \dots \dots O(e)$

4. $\text{mincost} \leftarrow \text{cost}(k, l)$

5. $(T(1,1), T(1,2)) \leftarrow (k, l)$

6. for $i \leftarrow 1$ to n do $\dots \dots \dots O(n)$

if $\text{cost}(i, l) < \text{cost}(i, k)$ then

$\text{NEAR}(i) \leftarrow l$

else

$\text{NEAR}(i) \leftarrow k$

7. for

7. $NEAR(k) \leftarrow NEAR(1) \leftarrow 0$

8. for $i \leftarrow 2$ to $n-1$ do ----- $O(n)$

{
a) let j be an index such that $NEAR(j) \neq 0$ and $COST(j, NEAR(j))$
is minimum. ----- $O(n)$

b) $(T(i,1), T(i,2)) \leftarrow (j, NEAR(j))$

c) $mincost \leftarrow mincost + COST(j, NEAR(j))$

d) $NEAR(j) \leftarrow 0$

e) for $k \leftarrow i$ to n do ----- $O(n)$

{
if $NEAR(k) \neq 0$ and $COST(k, NEAR(k)) > COST(k, j)$ then
 $NEAR(k) \leftarrow j$

}

}

9. if $mincost \geq \infty$ then

print ("no spanning tree")

}

Time complexity

using adjacency matrix: $O(n^2)$

using heap / Red black tree:

(i) Creating heap with e edges ----- $O(e)$

(ii) Step 3 ----- $O(\log e)$

(iii) Step 8 ----- $O(n)$

Step 8.a) ----- $O(\log n)$ using heap with $COST(j, NEAR(j))$ values.
near values.

Step 8.e) $NEAR(k) \leftarrow j$ corresponds to decrease key operation
and it takes $O(\log n)$

In total it occurs $n^2 \log n$ times (in worst case)
However in general it is $e \log n$

$\therefore TC = O(e) + O(e)$

$O(e) + O(\log e) + \underbrace{O(n \log n)}_{8.0} + \underbrace{O(\log n)}_{8.0}$

i.e., $O((n+e) \log n) \approx O(e \log n)$

$\hookrightarrow$ in worst case it is $O(n^2 \log n)$

Hence for dense graphs we use only adjacency matrix
but if the graph is sparse graph using heap is better.

Space Complexity:

without heap:

S.C = size(T) + size(heap)
 $= O(n)$

with heap:

S.C = size(heap for edges) + size(heap for heap)
 $S.C = O(e + n)$

Shortest Paths:

I. Single pair shortest path

II. Single Source Shortest path

Dijkstra's

- $\rightarrow$ Greedy
- $\rightarrow$ Always works good for +ve wt edges.
- $\rightarrow$ May or may not work with -ve edges

Bellman-Ford

- $\rightarrow$ Dynamic
- $\rightarrow$ Always works correctly with -ve wt. edges but not having -ve cycle reachable from source.

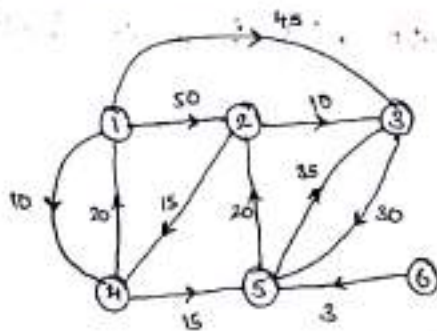
III. All Pair shortest Paths

→ one way is
 • we can use single source shortest paths for all vertices

→ Second approach is

Floyd-warshall's algorithm

Single Source ~~Set~~ Shortest Paths : Dijkstra's Algorithm:



This approach works for both directed & undirected graphs

Consider finding shortest path values from vertex '1'

i) Matrix Form:

$d(x)$: The value of vertex 'x' is ^{cost of} defined as shortest path known so far

vertex selected	d-values					
	1	2	3	4	5	6
{1}	-	50	45	(10)	(∞)	∞
{1,4}	-	50	45	(10)	(25)	∞
{1,4,5}	-	(45)	45	(10)	(25)	∞
{1,4,5,2}	-	(45)	(45)	(10)	(25)	∞
{1,4,5,2,3}	-	(45)	(45)	(10)	(25)	(∞)

→ This process of replacing distance is called relaxation.

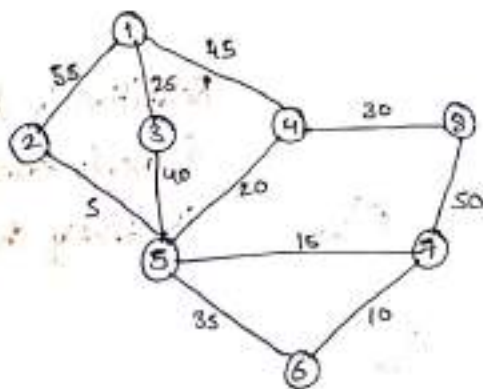
→ ~~find~~ cost of shortest paths.

Consider finding shortest path from vertex 6

Vertex Selected	d-values					
	1	2	3	4	5	6
{6}	∞	∞	∞	∞	(3)	-
{6,5}	∞	(23)	38	∞	(3)	-
{6,5,2}	∞	(23)	(33)	38	(3)	-
{6,5,2,3}	∞	(23)	(33)	(38)	(3)	-
{6,5,2,3,4}	58	23	33	28	3	-

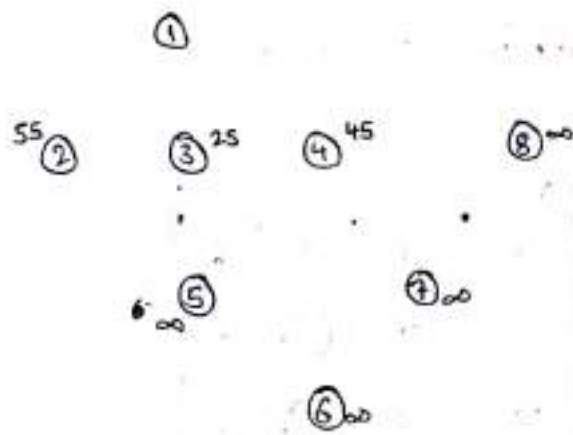
The limitation of this matrix form is that it shows only the cost of the shortest path but not the path.

(ii) Spanning tree approach:

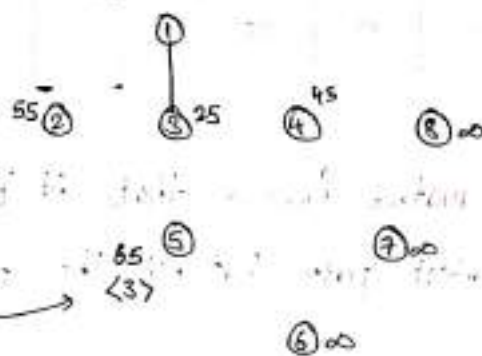


Consider finding shortest path from vertex 1.

→ write the edges of all nodes from node 1, as shown in the figure.

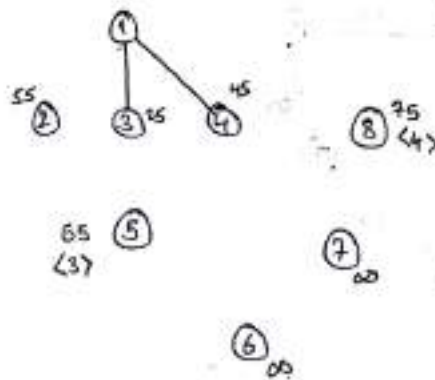


Since '3' is nearest so far, choose vertex 3



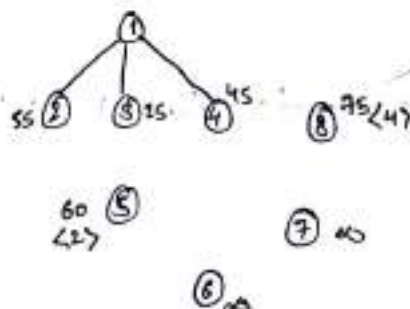
Here vertex 5 is relaxed.

Now choose vertex 4

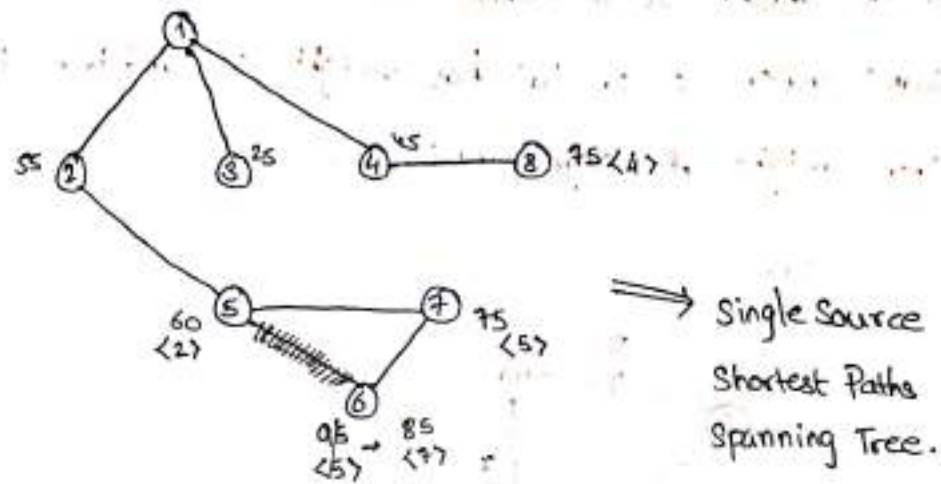


Relaxation is done if newly found path is strictly shorter.

Choose vertex '2'



choose vertex 5 (via 2)



∴ Final costs are

1	2	3	4	5	6	7	8
-	55	25	45	60	85	75	75

1-2

1-3

1-4

1-2-5

1-2-5-7-6

1-2-5-7

1-4-8

Note:

→ MST and single source shortest paths spanning tree need not to be same.

→ The working/implementation of dijkstra's algorithm is similarly to prim's algorithm.

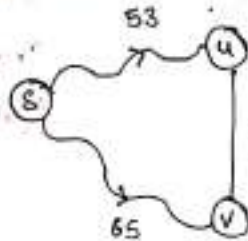
time complexity

without heap: $O(n^2)$

with heap: $O(n + \log n)$

- Q Consider weighted undirected graph. Let uv be an edge in the graph. It is known that the shortest path from the source vertex 's' to 'u' has weight 53 and to 'v' is 65. Then weight of edge uv is —

Sol:

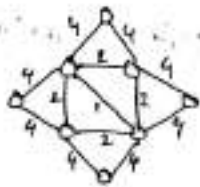


$$\Rightarrow 53 + x \geq 65$$

$$\Rightarrow x \geq 12$$

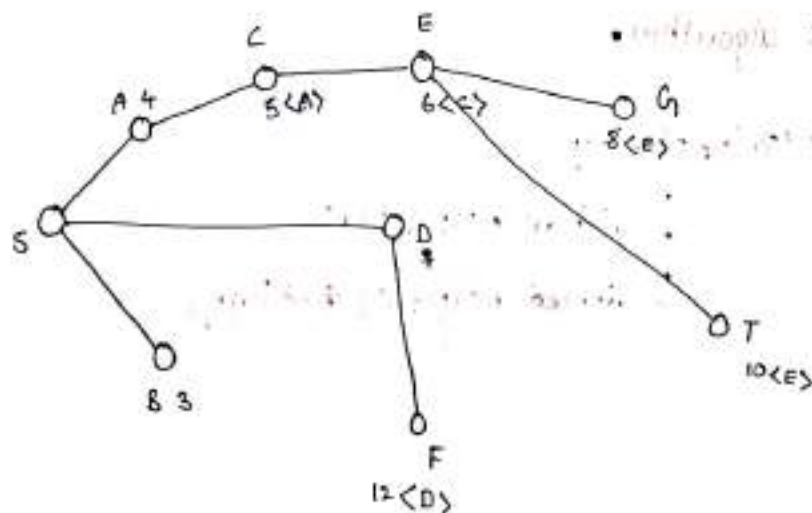
H/49

- Q Find no of MCST possible for below graph



Ans: 64

H/49

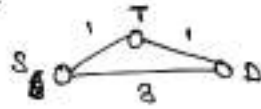


$\therefore$ SACET

H/50

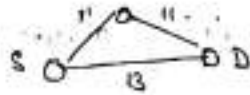
1. True ($\because$ doing so will inc cost of every spanning tree)
2. False by $(n-1)c \rightarrow$ increased cost.

Ex:



shortest path from S to D
is S-T-D

inc weight by 10

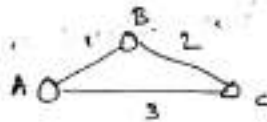


shortest path: S-D

H/52

I. True (concept)

II.



from A to C we have 2 shortest paths

i.e., A-B-C
A-C

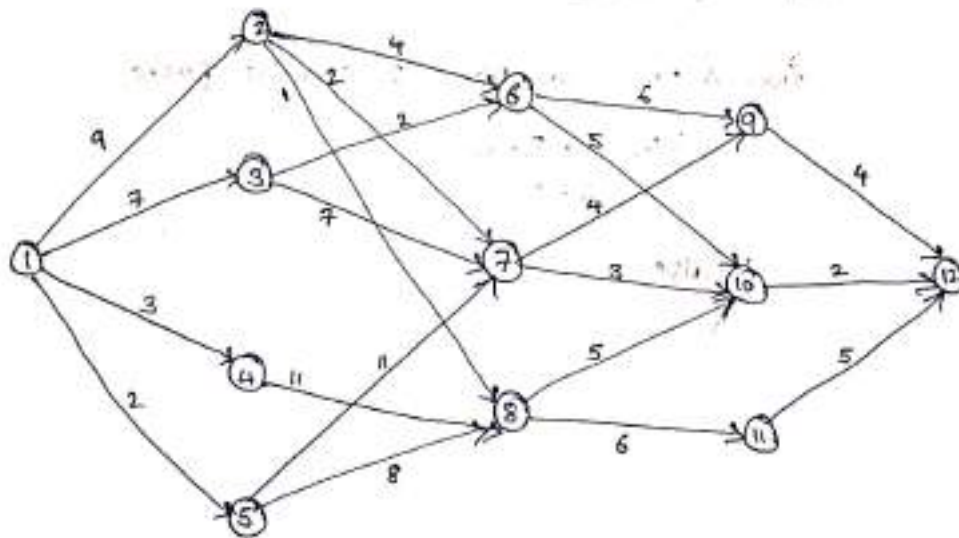
False

Dynamic Programming:

↳ Here programming means act of tabulating the results of subproblems.

Multistage Graph:

- A k -stage multistage graph has k stages where 1st & last stages have exactly one vertex and remaining stages have atleast one vertex.
- Every edge is from ~~stage i~~ a vertex of stage i to a vertex of stage $i+1$.
- The problem is to find distance b/w source & destination.



Applying greedy method (choosing least cost at every stage)

$1-5-8-10-12 \Rightarrow 17$ $\Rightarrow$ so using greedy method

Applying Dijkstra's algorithm for single pair shortest path may fail

$1-3-6-10-12 \Rightarrow 16$ $\Rightarrow$ greedy method works when we need to find shortest paths to all vertices.

→ DP is an algorithm design technique used for solving problems whose solutions are viewed as a result of making a set of decisions.

→ One way of solving these problems is by making decisions at every step based on local information available at that step. This is true for problems solved by greedy methods.

→ But for many problems, ~~optimal~~ like multistage graphs, optimal solutions cannot be achieved by making decisions based on local information in a stepwise manner.

→ One way of solving the problems for which optimal solutions cannot be made based on local information is to enumerate all possible decision sequences and pick up the best. This is known as brute force ^(enumeration strategy) approach. However for this strategy, the drawback is that it ~~has~~ has very large time & space complexities.

→ DP is based on enumeration, but it often tries to curtail the amount of enumeration by removing/avoiding those sequences that are not feasible or suboptimal.

Due to this there may be a significant improvement in time complexities.

→ DP makes the sequence of decisions by applying principle of optimality. (global optimality) (optimality substructure)

→ principle of optimality states that ^{whatever} the initial state and decision are, the remaining sequence of decision must be optimal with regard to state resulting from the current decision.

→ Any problem that ~~ob~~ obeys principle of optimality and overlapping subproblems can be solved by DP.

→ The fundamental difference b/w greedy method and DP is that GM generates only one decision sequence whereas DP may generate multiple decision sequences.

→ In DP, the results of subproblems can be tabulated and can be reused. (Memoization)

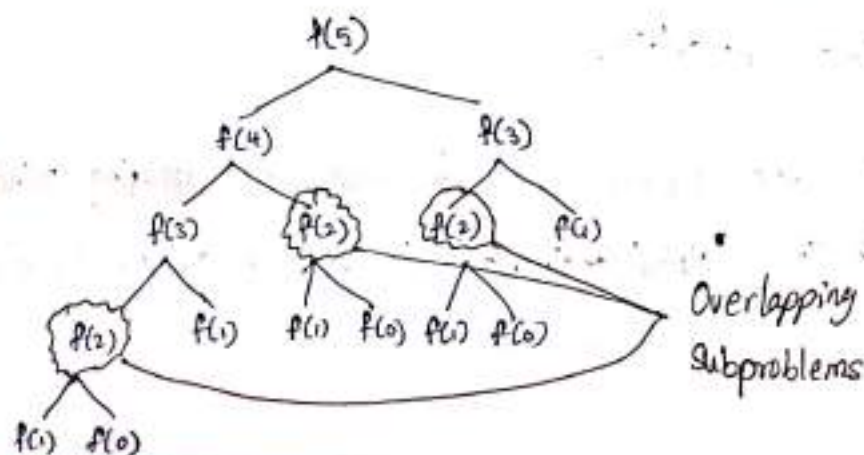
→ Thus DP optimizes time by curtailing decision that lead to non-feasible or suboptimal soln and by tabulated results of subproblems.

Eg: Consider computing n th fibonacci number

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$



→ Regular recursive implementation

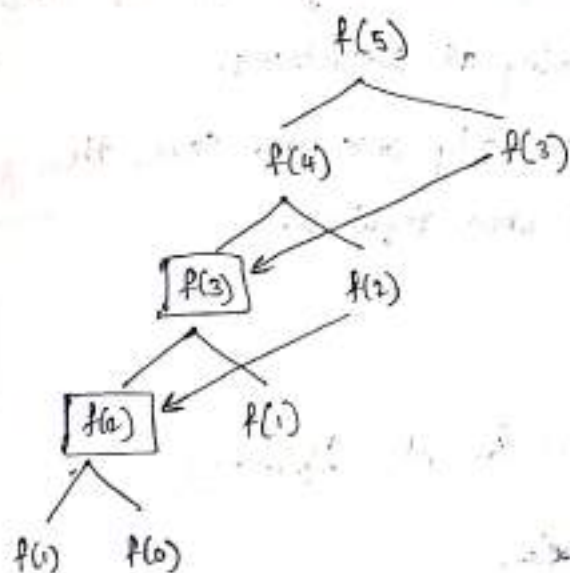
```

int fib(n)
{
    if (n ≤ 1) return n;
    else
        return fib(n-1) + fib(n-2);
}

```

TC: $O(2^n)$ Space Comp: $O(n)$

→ Top down DP with memoization:



Rather than tree, here
a closed graph is formed
without cycle.

→ DP problems can be solved in 2 ways:

Top down
approach with
memoization
(Implemented with
recursion)

Bottom up approach
with tabulation
(Non-recursive)
→ Generally this is
more efficient in
terms of T.C

Note:

Greedy: Builds up a solution incrementally by optimizing at each step some local criteria.

DTC: Break up a problem into separate subproblems and solve each subproblem independently and combining the solutions to subproblems to get soln of original problem.

DP: Break up a problem into a series of overlapping subproblems and build up solns to larger and larger subproblems.

Unlike ~~Greedy~~ DTC, DP involves solving all subproblems.

∴ DP solves each subproblem only once and stores the results & uses them later when required.

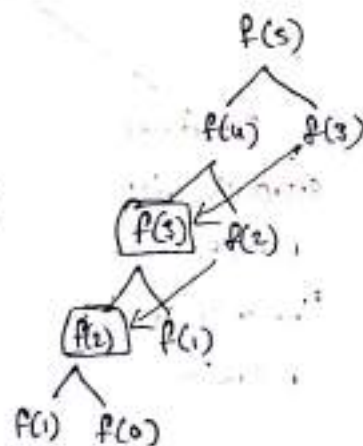
13/10/20

Top down approach (Memoization) for nth fibonacci

initialize arry $f[]$ as shown below

0	1	2	3	4	5
0	1	NIL	NIL	NIL	NIL

```
int fib(n)
{
    if (f[n] != NIL)
        f[n] = fib(n-1) + fib(n-2)
    return (f[n]);
}
```



T.C: $O(n)$

∴ every for every number $\text{fib}(n)$ is computed only once.

S.C: $O(n)$

i.e., size(array) + size(stack)

Bottom up approach (Tabulation) for nth Fibonacci number

```
f[1000]: int array;
```

```
f[0] = 0; f[1] = 1;
```

```
int fib(n)
```

```
{
```

```
    for i = 2 to n
```

```
        f[i] = f[i-1] + f[i-2];
```

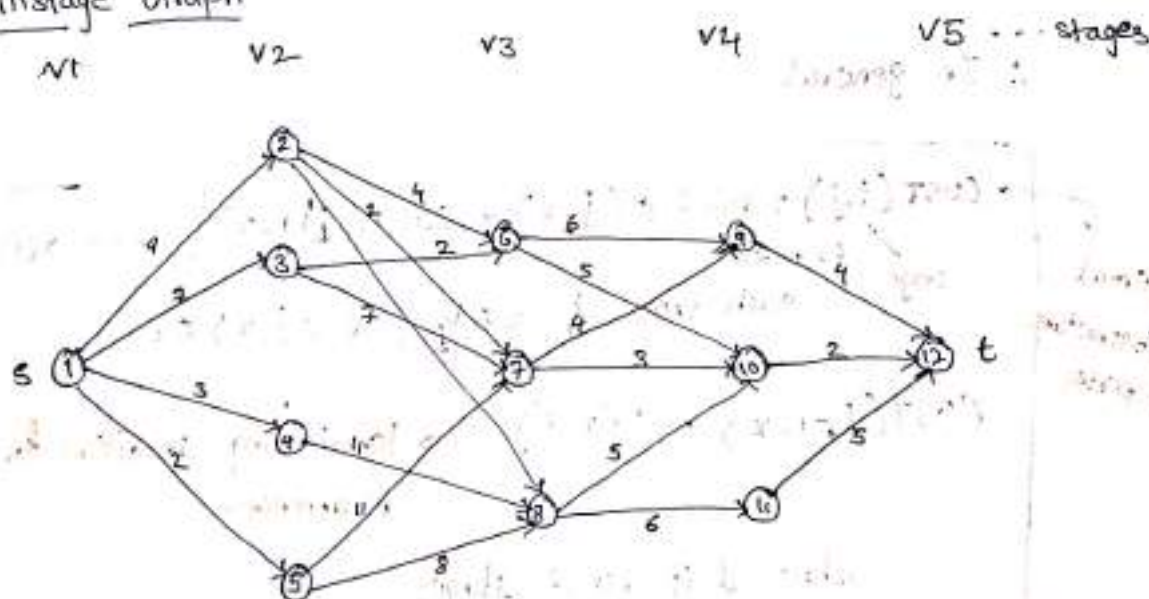
```
    return (f[i]);
```

```
}
```

TC: $O(n)$

SC: $O(n)$ [i.e., size of array]

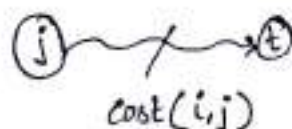
Multistage Graph



Let $C[i, n, 1 \dots n]$ represent cost adjacency matrix.

Let $cost(i, j)$ represent cost of path from vertex j that is present in stage i to the destination vertex t .

i.e., stage i



So the target is to find $\text{cost}(1, t)$.

from ① we go to vertex x such that $x \in V_2$ & $\langle 1, x \rangle \in E$
↓
i.e., there are edge
b/w 1 & x .

We choose this x such that $\text{cost}(1, x) + \text{remaining cost of path from } x \text{ to } t$ is minimum.

i.e., $C(1, x) + \text{cost}(x, t)$ is minimum

$$\therefore \text{cost}(1, t) = \min \{ C(1, x) + \text{cost}(x, t) \}$$

such that $x \in V_2$ & $\langle 1, x \rangle \in E$

$\therefore$ In general

optimal
substructure
property

$$\text{cost}(i, j) = \min \{ C(j, x) + \text{cost}(i+1, x) \} \quad \text{--- ①}$$

stage $\rightarrow$ vertex

such that $x \in V_{i+1}$ & $\langle j, x \rangle \in E$

$\text{cost}(l-1, x) = C(x, t) \rightarrow$ terminating condition for the recurrence.

where l is no of stages

$$D(i, j) = x'$$

such x' is the value which minimizes $\text{cost}(i, j)$ in eq ①

$\rightarrow$ used to obtain the vertices in the shortest path.

$$\text{cost}(1,1) = \min_{i \in \{2,3,4,5\}} \{ 9 + \text{cost}(2,2), 7 + \text{cost}(2,3), 3 + \text{cost}(2,4), 2 + \text{cost}(2,5) \}$$

so we start from back

$$\text{cost}(4,9) = c(9,t) = 4$$

$$\text{cost}(4,10) = c(10,t) = 2$$

$$\text{cost}(4,11) = c(11,t) = 5$$

now we compute costs of stage 3 vertices

$$\text{cost}(3,6) = \min \{ c(6,9) + \text{cost}(3+1,9), c(6,10) + \text{cost}(3+1,10) \}$$

$$= \min \{ 6 + \text{cost}(4,9), 5 + \text{cost}(4,10) \}$$

$$= \min \{ 6 + 4, 5 + 2 \} = 7$$

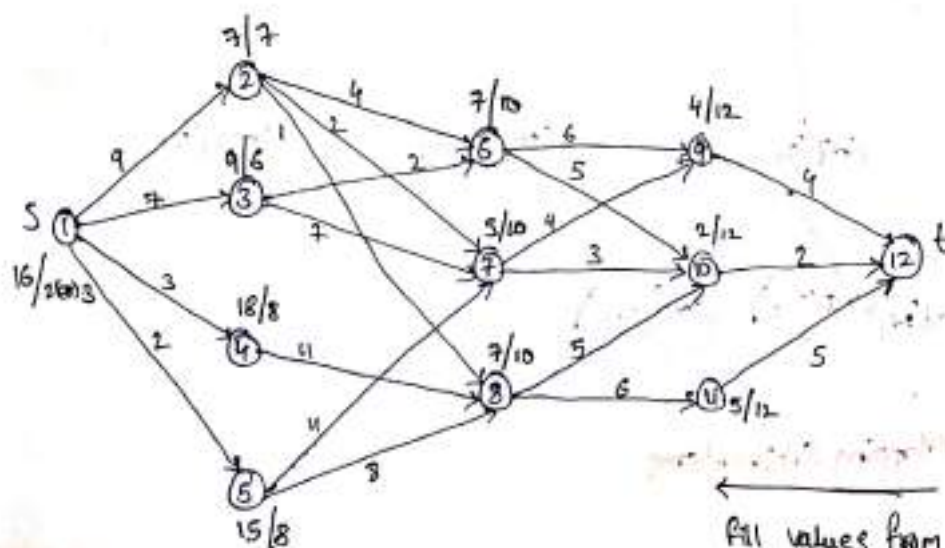
$$\text{also } D(3,6) = 10$$

↳ the vertex which gives shortest path from vertex 6.

$$\text{cost}(3,7) = \min \{ 5$$

$$D(3,7) = 10$$

} these values can be found directly from the graph



Fill values from backward.
At each node we write $\text{cost}(i,j) / D(i,j)$

$\therefore$ Min cost path's cost = 16

path can be found by using array D.

From graph we find path using the labels assigned to each vertex

$\therefore$ path: 1-2-7-10-12

1-3-6-10-12

Formal way for path construction:

For an k -stage graph we need to make $k-2$ decisions.

path: 1 -

$$D(1,1) = 2$$

path: 1-2-

↑
1st decision

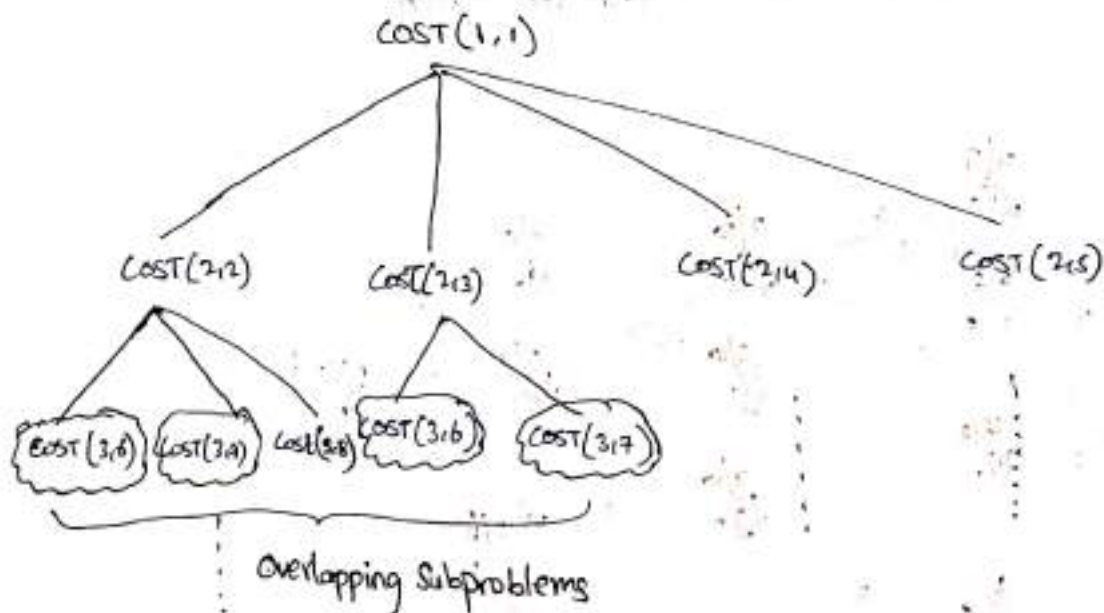
$$D(2, D(1,1)) = D(2,2) = 7$$

path: 1-2-7-

↑
2nd decision

$$D(3, D(2, D(1,1))) = D(3, D(2,2)) = D(3,7) = 10$$

path: 1-2-7-10-12



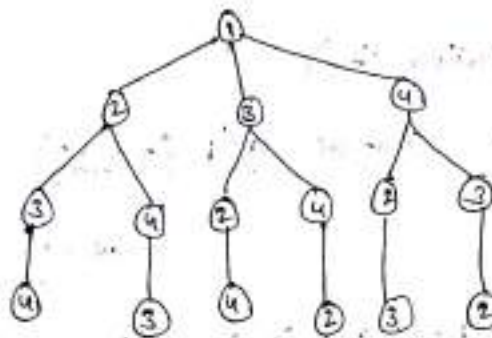
Travelling Salesman Problem

47

Problem Defn:

The TSP has to organize a tour in such a way that all the cities (except the starting city) must be visited exactly once and return to the home (starting city) with an objective of minimizing the cost of the tour.

① Thus if there are 4 cities (including starting city) brute force approach will be



→ $(n-1)!$

$$\therefore \text{TC of brute force} = (n-1)! = O(n^n)$$

The path is like $1 \xrightarrow{\quad} x \xrightarrow{\quad} 1$
optimality substructure.

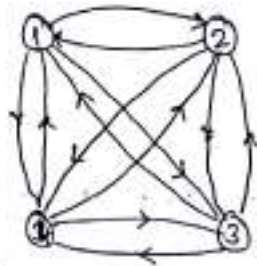
From tree, we can show overlapping subproblems.

Recurrence formula for TSP:

Let $c[i,j]$ be a cost adjacent matrix

Let $g(i,s)$ represent cost of the tour from vertex i , visiting all the vertices in the set s exactly once and terminating the tour at v_0 (home city)

Consider



Let cost adjacency matrix C be

C	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

For $v_0 = 1$ (starting city)

we need to compute $g(1, \{2, 3, 4\})$

↳ we need to minimize this.

$$\therefore g(1, \{2, 3, 4\}) = \min \{ C(1, 2) + g(2, \{3, 4\}), C(1, 3) + g(3, \{2, 4\}), C(1, 4) + g(4, \{2, 3\}) \}$$

$\therefore$ In general,

$$g(i, S) = \min \{ C(i, x) + g(x, S - \{x\}) \}$$

such that $x \in S$ & $\langle i, x \rangle \in E$

$$g(i, S) = C(i, v_0)$$

$\downarrow$
 $i \neq v_0$

if $S = \emptyset$

↳ termination condition for recursion.

we use $g(i, S)$ to compute the path

$$S(i, S) = x$$

Now let us calculate $g(1, \{2, 3, 4\})$

$|S|=0$:

$$g(2, \emptyset) = 5$$

$$g(3, \emptyset) = 6$$

$$g(4, \emptyset) = 8$$

$|S| \geq 1$:

$$g(2, \{3\}) = \min \{c(2, 3) + g(3, \emptyset)\} = 9 + 6 = 15, \quad J(2, \{3\}) = 3$$

$$g(2, \{4\}) = \min \{c(2, 4) + g(4, \emptyset)\} = 10 + 8 = 18, \quad J(2, \{4\}) = 4$$

$$g(3, \{2\}) = \min \{c(3, 2) + g(2, \emptyset)\} = 13 + 5 = 18, \quad J(3, \{2\}) = 2$$

$$g(3, \{4\}) = 12 + 8 = 20, \quad J(3, \{4\}) = 4$$

$$g(4, \{2\}) = 8 + 5 = 13, \quad J(4, \{2\}) = 2$$

$$g(4, \{3\}) = 9 + 6 = 15, \quad J(4, \{3\}) = 3$$

$|S| = 2$:

$$g(2, \{3, 4\}) = \min \{c(2, 3) + g(3, \{4\}), c(2, 4) + g(4, \{3\})\}$$

$$= \min \{9 + 20, 10 + 15\}$$

$$= 25$$

$$J(2, \{3, 4\}) = 4$$

$$g(3, \{2, 4\}) = \min \{13 + 18, 12 + 13\}$$

$$= 25$$

$$J(3, \{2, 4\}) = 4$$

$$g(4, \{2, 3\}) = \min \{8 + 15, 9 + 18\}$$

$$= 23$$

$$J(4, \{2, 3\}) = 2$$

$|S| = 1$:

$$g(1, \{1, 2, 4\})$$

$$= \min \{ c(1,2) + g(2, \{3,4\}), c(1,3) + g(3, \{2,4\}), c(1,4) + g(4, \{2,3\}) \}$$

$$= \min\{10+25, 15+25, 20+23\}$$

≈ 35

$$S(1, \{2, 3\}) = 2$$

Path Construction:

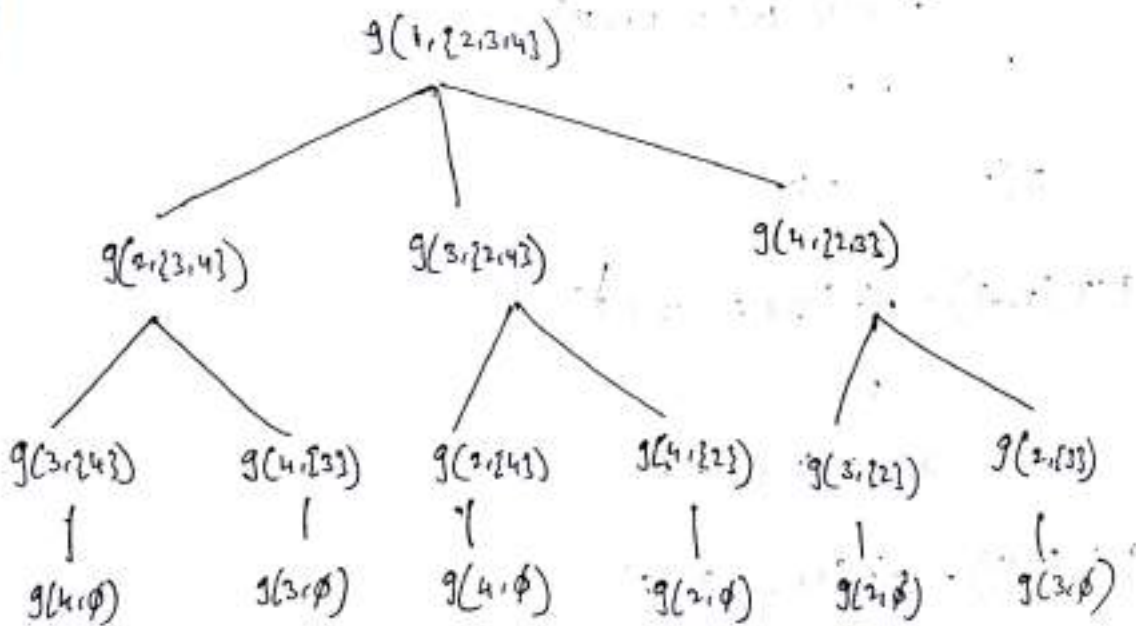
If $J(i, s) = x$ then from x we go to $J(x, s-x)$

$$J(1, \{2, 3, 4\}) = 2.$$

$$J(2, \{3, 4\}) = 4$$

$$T(4, \{3\}) = 3$$

$\therefore$ path : 1-2-4-3-1



For larger problems we can see overlapping subproblems.

→ The algorithm for TSP is Held & Karp Algorithm

Time Complexity: $O(n^2 \cdot 2^n)$

∵ we have 2^n subsets and for every subsets we need to check for possible edges (max possible edges = n^2)

Space Complexity: $O(n \cdot 2^n)$

↳ To store values of g

because for every possible subset s and for every vertex except start vertex we calculate and store

$g(v, s)$ ∴ $O(n \cdot 2^n)$
 $\downarrow \quad \downarrow$
 $n \quad 2^n$

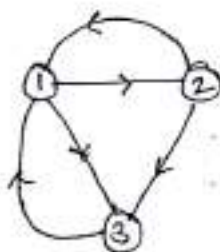
→ TSP is an intractable Problem

∵ it doesn't have a polynomial TC algorithm so far.

∴ TSP is an NP problem (NP-complete).

16/10/20

All Pairs Shortest Paths (Floyd Warshall's Algorithm)



c	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

$C(i,j) \rightarrow$ 'edge' cost

Let $A^k(i,j)$ represent the cost of the path from vertex 'i' to vertex 'j' NOT going through the intermediate vertex greater than k.

Assume vertices are sequentially numbered from 1 to n $(1, 2, 3, \dots, n)$

$$\therefore A^k(i,j) = \min \{ \text{path goes through } k, \text{ path doesn't go through } k \}$$

If path goes through k



hight now here highest intermediate vertex possible = $k-1$

$\therefore$ If path goes through k, it will be

$$A^{k-1}(i,k) + A^{k-1}(k,j)$$

$$\therefore A^{k+1}(i,j) = \min \{ A^{k-1}(i,k) + A^{k-1}(k,j), A^{k-1}(i,j) \}, k \neq 0$$

$$A^0(i,j) = C(i,j)$$

Ex:

A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

$$A^1(1,2) = \min \{ A^0(1,1) + A^0(1,2), A^0(1,2) \} = 4$$

$$A^1(1,3) = 11 \quad (1 \rightarrow 1 \rightarrow 2)$$

$$A^1(2,1) = 6 \quad (2 \rightarrow 1 \rightarrow 1)$$

$$A^1(2,3) = \min \{ A^0(2,1) + A^0(1,3), A^0(2,3) \} = \min \{ 1+2 \} = 2$$

$$A'(3,1) = 3$$

$$A'(3,2) = \min\{3+4, \infty\} = 7$$

$$A'(3,3) = 0$$

$$\therefore A' \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 11 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & 7 & 0 \end{array}$$
 $\Rightarrow$

$$A'' \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 6 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & 7 & 0 \end{array}$$

$$A^3 \begin{array}{c|ccc} & 1 & 2 & 3 \\ \hline 1 & 0 & 4 & 6 \\ 2 & 6 & 0 & 2 \\ 3 & 3 & 7 & 0 \end{array}$$

Algo Floyd-Warshall (G, n, e, C, A)

$\{C[1..n, 1..n]\}$... cost adjacency matrix

$\{A[1..n, 1..n]\}$... path cost adjacency matrix

{

1. for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to n

$A[i, j] = C[i, j];$

TC: $O(n^3)$

SC: $O(n^2)$

2. for $k \leftarrow 1$ to n

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to n

$A[i, j] = \min\{A[i, j], A[i, k] + A[k, j]\}$

}

→ The transi

Transitive closure:

→ Let A be the adjacency matrix of a directed/undirected graph. The transitive closure A^* of A be a matrix with the property that $A^*(i, j) = 1$, iff G has a directed path/path containing atleast atleast one edge from vertex i to vertex j , otherwise $A^*(i, j) = 0$.

→ Thus, transitive closure of a matrix, represent graph with n vertices, can be computed in $O(n^3)$ time.

Reflexive Transitive Closure:

Let A be adjacency matrix of graph G and matrix A^+ be reflexive transitive closure of graph G defined as.

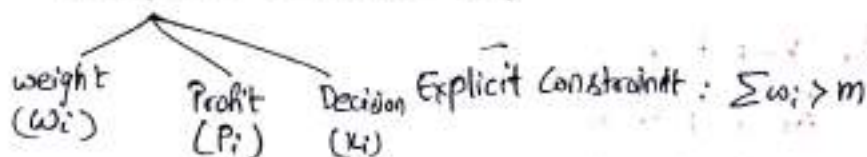
$A^+(i, j) = 1$ iff G has a path containing 0 or more edges from i to j

$A^+(i, j) = 0$, otherwise

0/1 knapsack:

knapsack capacity: m

n -object: $\langle o_1, o_2, \dots, o_n \rangle$



objective func: Maximize the total profit ~~that are~~ subjected to the condition that total weight being put into the knapsack, doesnot exceed its capacity.

i.e., Maximize $\sum_{i=1}^n P_i x_i$ subjected to $\sum_{i=1}^n w_i x_i \leq M$

where $x_i = 0/1$

Approach I:

Let $f_n(m)$ represent the profit with n -objects and a knapsack capacity of M .

$f_n(m) = \max \{ \text{profit by including the object, profit by excluding the object} \}$

$$\therefore f_n(m) = \max \left\{ \begin{array}{ll} (x_n=1) & (x_n=0) \\ P_n + f_{n-1}(m-w_n) & f_{n-1}(m) \end{array} \right\}$$

$f_0(x) = 0;$

Ex: $n=3; M=6; \langle P_1, P_2, P_3 \rangle = \langle 1, 2, 5 \rangle$

$\langle w_1, w_2, w_3 \rangle = \langle 2, 3, 4 \rangle$

$\langle x_1, x_2, x_3 \rangle = ?$

Tuple method:

Let f_n be represented by $S^n = \{(P, w)\}$

$P \rightarrow \text{Profit}$

$w \rightarrow \text{weight put into the knapsack.}$

$\therefore S^0 = \{(0, 0)\}$

S'_1 is obtained by adding P_1 & w_1 to S^0

$S'_1 = \{(1, 2)\}$

Now we merge S^0 & S'_1 to obtain S^1

$S^1 = \left\{ \begin{array}{ll} x_1=0 & x_1=1 \\ (0, 0) & (1, 2) \end{array} \right\}$

S_1^2 is obtained by adding (P_2, w_2) to S^1

$$S_1^2 = \{(2,3), (3,5)\}$$

$$S_1^2 \supset S^2 = \{(0,0), (1,2), (2,3), (3,5)\}$$

Now

$$S_1^3 = \{(5,4), (6,6)\}$$

not feasible to add $(5,4)$

not feasible to add $(5,4)$

$$S_1^3 \supset S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\}$$

Here we have discarded $(3,5)$ since

$(5,4)$ gives max profit with less weight.

This is called Purging rule.

$\therefore (3,5)$ will never lead to optimal soln.

Purging rule:

Let (P_i, w_i) & (P_j, w_j) be tuple to be merged

if $(P_i < P_j)$ and $(w_i > w_j)$ then the tuple (P_i, w_i) may be purged (removed).

Computing values of x_i 's:

we got $(6,6)$ as answer

$$(6,6) \in S^3 \text{ but } (6,6) \notin S^2$$

$\Rightarrow$ we added 0_3 i.e., $(5,4)$ is added

$$\therefore x_3 = 1$$

$$(6,6) - (5,4) = (1,2)$$

$$\text{Now } (1,2) \in S_1^2 \text{ \& } (1,2) \in S_1^1 \text{ \& } (1,2) \notin S^0$$

$$\Rightarrow x_1 = 1$$

$$x_2 = 0$$

$$\therefore (x_1, x_2, x_3) = (1, 0, 1)$$

57

$\therefore$ If $(p_i, w_i) \in S^i$ & $(p_i, w_i) \notin S^{i-1}$ then $x_i = 1$ & $(p_i, w_i) \leftarrow (p - p_i, w - w_i)$
 if $(p_i, w_i) \in S^i$ & $(p_i, w_i) \in S^{i-1}$ then $x_i = 0$

eg: $n=4$; $\langle p_1, p_2, p_3, p_4 \rangle = \langle 60, 28, 20, 24 \rangle$

$M=11$; $\langle w_1, w_2, w_3, w_4 \rangle = \langle 10, 7, 4, 2 \rangle$

I. Greedy fractional knapsack:

$$p_i/w_i: \langle 6, 4, 5, 12 \rangle$$

$$\therefore x_4 = 1 \Rightarrow m = 11 - 2 = 9$$

$$x_1 = \frac{9}{10} = 0.9$$

$$\therefore P = (0.9)(60) + (1)(24)$$

$$= 54 + 24 = 78$$

II. 0/1 knapsack with greedy approach

$$p_i/w_i: \langle 6, 4, 5, 12 \rangle$$

$$x_4 = 1 \Rightarrow m = 11 - 2 = 9$$

~~20~~ $w_1 = 10$ not possible to include

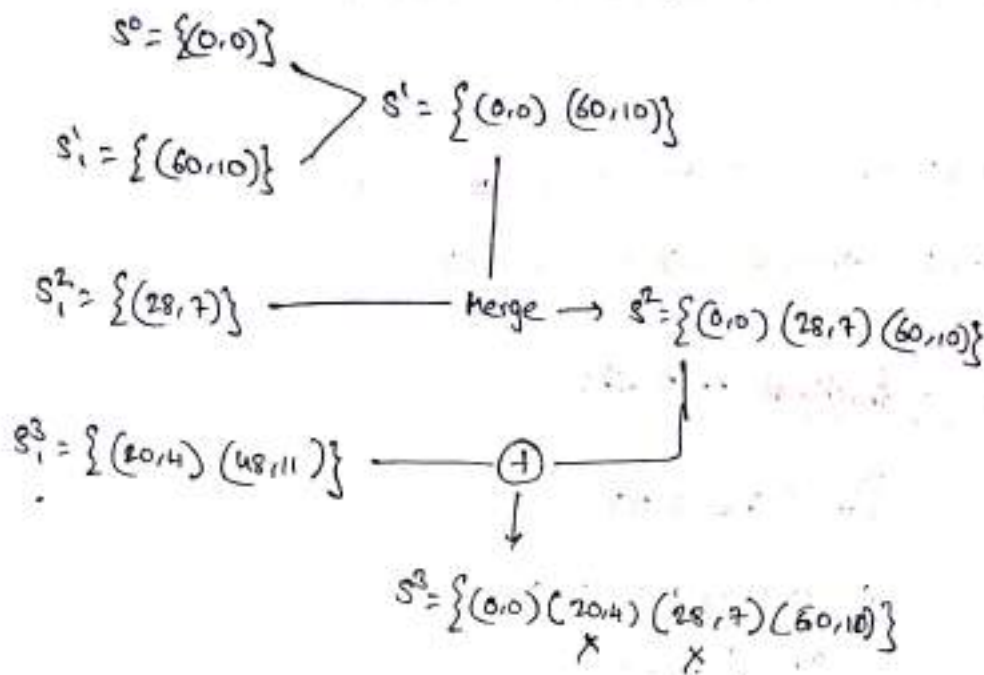
$$\therefore x_1 = 0$$

$$x_3 = 1 \Rightarrow m = 9 - 4 = 5$$

$$w_2 > m \therefore x_2 = 0$$

$$\therefore P = 20 + 24 = 44$$

III. 0/1 knapsack using DP



$$S^4 = \{(0,0), (24,2), (44,6), (52,9)\}$$

$$\Rightarrow S^4 = \{(0,0), (24,2), (44,6), (52,9), (60,10)\}$$

$\therefore (60,10)$ is optimal

$$\underbrace{(60,10) \in S^4; (60,10) \in S^3; (60,10) \in S^2; (60,10) \in S^1; (60,10) \in S^0}_{x_4=0, x_3=0, x_2=0, x_1=1}$$

$$\therefore P = (1)(60) = \underline{60}$$

Approach II: (tabulation / bottom up approach)

$$\text{KNAP} \rightarrow k(n, m)$$

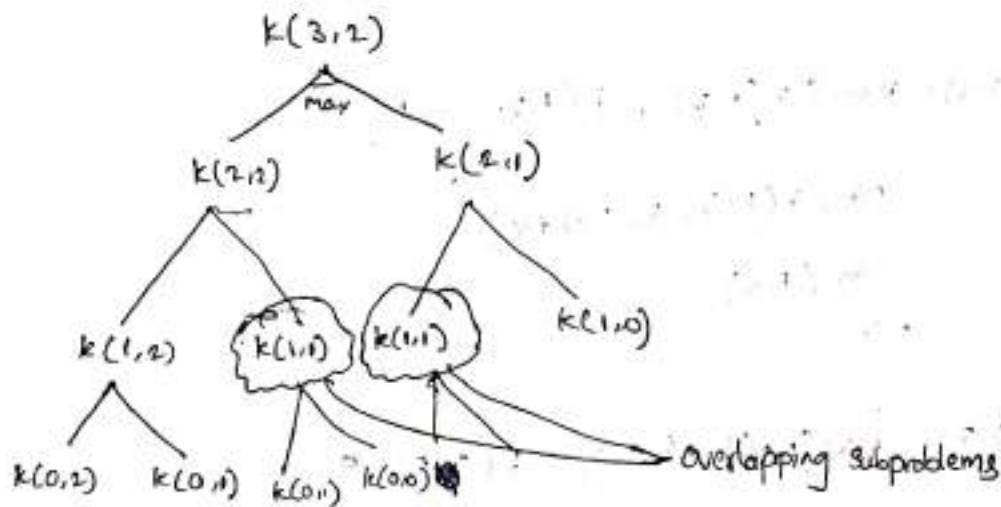
$\downarrow$
 no. of obj $\rightarrow$ knapsack capacity

$$k(n, m) = k(n-1, m), \quad w[n] > m$$

$$= \max \left\{ \underbrace{k(n-1, m)}_{(x_n=0)}, \underbrace{k(n-1, m-w_n) + p_n}_{(x_n=1)} \right\}, \quad w[n] \leq m$$

optimality substructure.

$g: n=3; m=2;$
 $w = \{1, 4\}$
 $P = \{5, 10, 15\}$



Boundary conditions:

$$k(0,x) = 0; \quad k(y,0) = 0;$$

$g: m=7; n=4; P = \{1, 4, 5, 7\}; w = \{1, 3, 4, 5\}$

we can solve this problem using a 2-dimensional matrix.

$$k[n+1, m+1]$$

$$m=7, n=4 \Rightarrow k[5, 8]$$

Indexing starts from

		H →							
	k	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1
2	0	1	1	4	5	5	5	5	5
3	0	1	1	4	5	6	6	9	9
4	0	1	1	4	5	7	8	9	9

The cell is $k[4,7]$
i.e., soln

$$k(1,1) = \max\{k(0,1) + k(0,0) + 1\} = 1$$

$$k(1,2) = 1$$

$$k(2,1) = \max \{ k(0,1), k(1,-2) \}$$

X
(not possible)

$$= k(1,1)$$

$$= 1$$

$$k(2,3) = \max \{ k(1,3), k(1,3-3) + 4 \}$$

$$= \max \{ k(1,3), k(1,0) + 4 \}$$

$$= \max(1, 4)$$

$$= 4$$

$$k(4,7) = \max \{ k(3,7), k(3,7-4) + 9 \}$$

$$= \max \{ 9, 9 \}$$

$$= 9$$

Calculating x_i values from table

$k(n,m)$ is obtained from either $k(n-1,m)$ cell or $k(n-1,m-w_n)$ cell.

If obtained from $k(n-1,m)$ then $k(n,m) = k(n-1,m)$

If obtained from $k(n-1,m-w_n)$ then $k(n,m) = k(n-1,m-w_n) + p_n$

$\therefore$ If $k(n,m) = k(n-1,m)$ then

$$x_i = 1 \text{ \& } (n,m) \leftarrow (n-1,m)$$

If $k(n,m) \neq k(n-1,m)$ then

$$x_i = 0 \text{ \& } (n,m) \leftarrow (n-1,m-w_n)$$

and we perform this recursively.

Algo KNAP ($n, m, w[], p[]$)

{ declare $k[n+1, m+1]$;

1. for $i \leftarrow 0$ to n

for $j \leftarrow 0$ to m

if ($i=0$ or $j=0$)

$k[i, j] = 0$

else

else if ($w[i] \leq j$) then

$k[i, j] = \max\{k[i-1, j], k[i-1, j-w[i]] + p_i\}$

else

$k[i, j] = k[i-1, j]$;

}

→ Time Complexity: $O(n \cdot m)$

→ Time complexity with brute force: $O(2^n)$

However for large values of m , $O(n \cdot m)$ is too high.

For example if $m = 2^n$

then TC of tabulation method = $O(n \cdot 2^n)$

and in this case brute force would be better.

Thus tabulation method is good for smaller values of n .

→ Space Complexity with tabulation method: $O(nm)$

17/10/20

Longest Common Subsequence:

Substring: grp of one or more characters taken from the string that are contiguous

Subsequence: group of one or more characters may not be contiguous nevertheless the relative order is same.

→ Every substring is a subsequence

→ For a string of length n ,

no of substrings $\rightarrow O(n^2)$

no of subsequences $\rightarrow O(2^n)$

→ A subsequence that is common to given two string is called common subsequence.

The longest of all these common subsequences is called longest common subsequence.

Applications of LCS:

i) Web search

ii) DNA matching

iii) Software matching

iv) Plagiarism

Solution:

Given two strings X & Y of length ' n ' & ' m ' respectively. It is required to determine a subsequence of longest length that is common to both X & Y

$$X = \langle x_1, x_2, \dots, x_n \rangle \quad Y = \langle y_1, y_2, y_3, \dots, y_m \rangle$$

→ Let ' i ' & ' j ' be indices into the strings X & Y as shown.

→ let $L[i, j]$ represent the length of common subsequence of string $X + Y$.

Case I: Last characters ^{are} same

Ex. $X = \langle G T T C C T A A T A \rangle$

$Y = \langle C G A T A A T T G A G A \rangle$

so we need to find $L[9, 11]$:

if $X[i] = Y[j]$ then

$$L[i, j] = 1 + L[i-1, j-1]$$

Case II: Last characters don't match

$X = \langle G T T C C T A A T A \rangle$

$Y = \langle C G A T A A T T G A G A \rangle$

If $X[i] \neq Y[j]$

∴ This means that LCS may terminate with last character of X or last character of Y or neither:

$$\text{i.e., } L[i, j] = \max \{ L[i-1, j], L[i, j-1] \}$$

∴ if $X[i] = Y[j]$

$$L[i, j] = 1 + L[i-1, j-1]$$

if $X[i] \neq Y[j]$

$$L[i, j] = \max \{ L[i-1, j], L[i, j-1] \}$$

$$L[-1, j] = 0$$

$$L[i, -1] = 0$$

Tree approach to solve LCS:

$X = \langle ABBAB \rangle$ $Y = \langle ACBAB \rangle$ $(ABAB)$
 $\uparrow u(\text{ABB})$
 $LCS("ABAB", "ACBAB")$

$(ABAB) \rightarrow (1+)(B)$

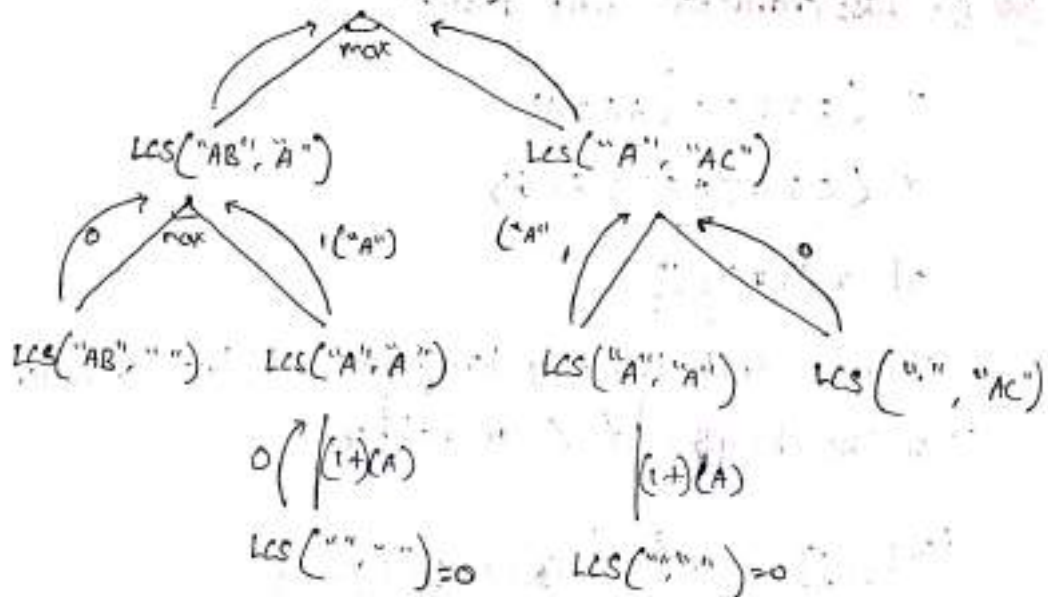
$LCS("ABBA", "ACBA")$

$(AB) \rightarrow (1+)(A)$

$LCS("ABB", "ACB")$

$(A) \rightarrow (1+)(B)$

$LCS("AB", "AC")$



Tabulation method (bottom up):

$X = \langle A B C B D A B \rangle$ $Y = \langle B D C A B A \rangle$

		0	1	2	3	4	5
		B	D	C	A	B	A
0	A	0	0	0	0	1	1
1	B	0	1	0	1	2	2
2	C	0	1	1	2	2	2
3	B	0	1	1	2	3	3
4	D	0	1	2	2	3	3
5	A	0	1	2	3	4	4
6	B	0	1	2	3	4	4

→ This is the soln
 i.e., $L[6,5]$

$$L[0,0] = \max\{L[-1,0], L[0,-1]\} = 0$$

$$L[0,1] = \max\{L[-1,1], L[0,0]\} = 0$$

If case II applies then we take $\max$ of {upper cell, left side cell}

If case I applies we take ~~1~~ 1 + upper left diagonal cell's value.

$$L[0,3] = 1 + L[-1,2] = 1 + 0 = 1$$

$$\therefore L[6,5] = 4$$

To get the subsequence we backtrack while backtracking add corresponding symbol whenever you move diagonally upward.

$\therefore$ LCS: B D A B

Algorithm LCS(x, y, n, m)

$L[n+1, m+1] := \text{array}$

1. for $i \leftarrow 0$ to $n-1$

~~for $j \leftarrow 0$ to $m-1$~~

$$L[i, -1] = 0$$

2. for $j \leftarrow 0$ to $m-1$

$$L[-1, j] = 0;$$

3. for $i \leftarrow 0$ to $n-1$

for $j \leftarrow 0$ to $m-1$

if $(x[i] = y[j])$

$$L[i, j] = 1 + L[i-1, j-1];$$

else

$$L[i, j] = \max\{L[i-1, j], L[i, j-1]\}$$

}

$$T.C: O(n \times m)$$

$$S.C: O(n \times m)$$

TC of LCS with brute force is $O(2^n \cdot m)$ or $O(n \cdot 2^m)$

Matrix Chain Product:

→ Given two ^{square} matrices of order n

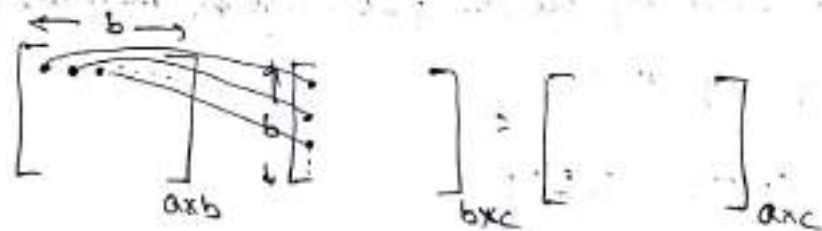
time to multiply: $O(n^3)$

operation: scalar multiplication

i.e., for product of two square matrices n^3 multiplications are required.

→ However to multiply two non-square matrices, the matrices have to be compatible.

i.e., no. of columns of 1st matrix = no. of rows of 2nd matrix



We need to compute $a \times c$ element and computation of each element requires $a \times b$ scalar multiplications.

⇒ we need to perform $a \times b \times c$ no. of multiplications

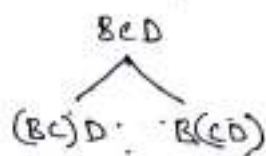
$$\therefore TC = O(a \times b \times c)$$

Problem Statement

Given a chain of matrices, it is required to multiply them to get a resultant product matrix.

• Eg: Consider $A = BCD$

$B: 2 \times 10; C: 10 \times 5; D: 5 \times 20$



$$(BC)D \left\{ \begin{array}{l} BC: 2 \times 10 \times 50 = 1000 \\ (BC)D: 2 \times 50 \times 20 = 2000 \\ \quad \quad \quad \downarrow \quad \downarrow \\ \quad 2 \times 50 \quad 50 \times 20 \end{array} \right\} 3000$$

$$B(CD) \left\{ \begin{array}{l} CD: 10 \times 50 \times 20 = 10000 \\ B(CD): 2 \times 10 \times 20 = 400 \end{array} \right\} 10400$$

Objective Function:

Minimizing the number of scalar multiplications.

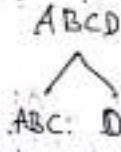
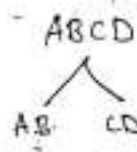
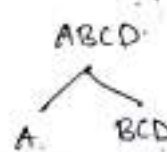
Problem statement:

Given a chain of matrices $\langle A_1 A_2 A_3 \dots A_n \rangle$ where matrix A_i is of order $P_{i-1} \times P_i$. It is required to fully parenthesize the given chain of matrices such that the no. of scalar multiplications is minimum.

ABC can be parenthesized in 2 ways

i.e., $(AB)C$ & $A(BC)$

ABCD can be parenthesized in 5 ways



$$(A(BCD))$$

$$((AB)(CD))$$

$$((AB)C)D$$

$$A((BC)D)$$

$$(A(BC))D$$

→ Observing this we can say: brute force method: will have exponential time complexity.

Applications

- i) In mathematics
- ii) Satellite communication
- iii) Physics

$$\text{Ex: } A_1 \rightarrow 3 \times 5 \rightarrow P_0 \times P_1$$

$$A_2 \rightarrow 5 \times 8 \rightarrow P_1 \times P_2$$

$$A_3 \rightarrow 8 \times 6 \rightarrow P_2 \times P_3$$

$$A_4 \rightarrow 6 \times 2 \rightarrow P_3 \times P_4$$

$$A_5 \rightarrow 2 \times 3 \rightarrow P_4 \times P_5$$

$$A_6 \rightarrow 3 \times 7 \rightarrow P_5 \times P_6$$

$$\langle A_1 A_2 \dots A_6 \rangle \rightarrow P_0 \times P_6$$

→ Let $A_{i:j}$ be the matrix that results from multiplication of matrices $\langle A_i A_{i+1} A_{i+2} \dots A_j \rangle$.

$$\text{Ex: } A_{1:6} = \langle A_1 A_2 A_3 A_4 A_5 A_6 \rangle$$

→ let $m[i,j]$ be the no of scalar multiplications needed to get the matrix $A_{i:j}$

→ Any optimal parenthesization must split the given chain of matrix about matrix A_k such that the no of scalar multiplications is minimum

$$(A_i A_{i+1} A_{i+2} \dots A_j) \rightarrow m[i,j]$$

$$= \underbrace{(A_i A_{i+1} A_{i+2} \dots A_k)}_{P_{i-1} \times P_k} \underbrace{(A_{k+1} A_{k+2} \dots A_j)}_{P_k \times P_j}$$

here k value ranges from i to $j-1$

i.e., $j-i$ splits are possible.

∴ If split at A_k

$$m[i,j] = m[i,k] + m[k+1,j] + (P_{i-1} * P_k * P_j)$$

$$\therefore m[i,j] = \min_{i \leq k < j} \{ m[i,k] + m[k+1,j] + (P_{i-1} * P_k * P_j) \}$$

$$m[i,i] = 0$$

Tabulation method: (bottom up)

69

Eg: $A_1 \rightarrow 3 \times 5$

$A_2 \rightarrow 5 \times 8$

$A_3 \rightarrow 8 \times 6$

$A_4 \rightarrow 6 \times 2$

So we need to compute $m[1,4]$

So in tabulation method we compute

$j-i=0$

$m[1,1] = m[2,2] = m[3,3] = m[4,4] = 0$

$j-i=1$

$m[1,2] = 3 \times 5 \times 8$

$m[2,3] = 5 \times 8 \times 6$

$m[3,4] = 8 \times 6 \times 2$

$j-i=2$

$m[1,3] = \min \{ m[1,2] + m[3,3] + 3 \times 8 \times 6, m[1,1] + m[2,3] + 3 \times 5 \times 6 \}$

still compute $m[2,4]$

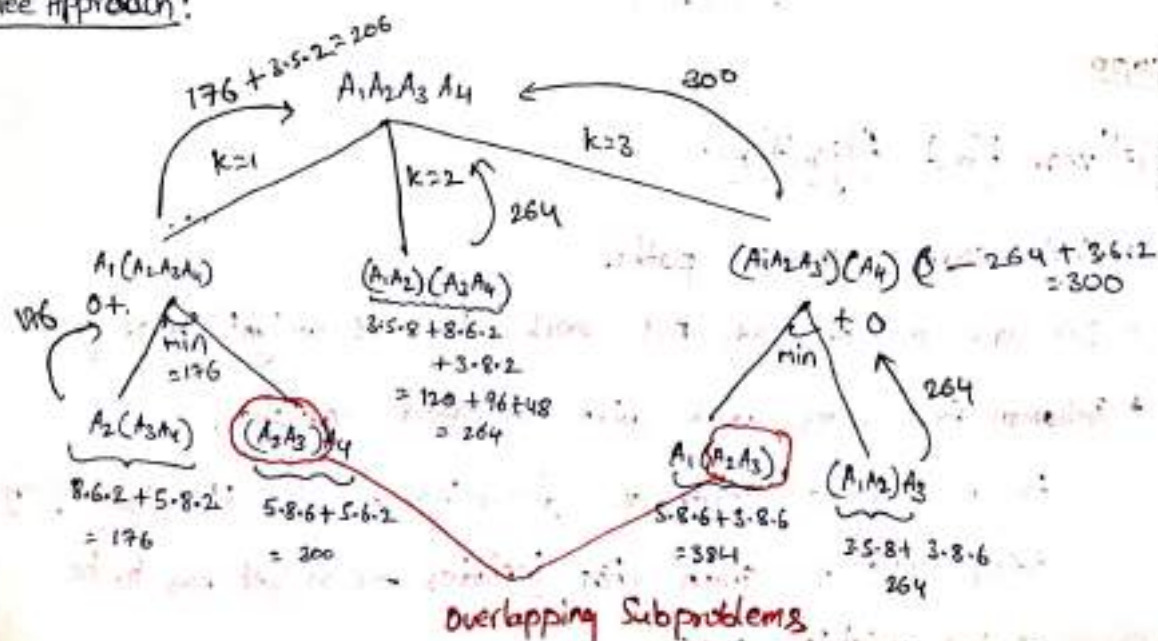
$j-i=3$

$m[1,4] = \min \{ m[1,1] + m[2,4] + 3 \times 5 \times 2,$

$m[1,2] + m[3,4] + 3 \times 8 \times 2,$

$m[1,3] + m[4,4] + 3 \times 6 \times 2 \}$

Tree Approach:



$\therefore$ Minimum cost = 206

optimal parenthesization = $(A_1 (A_2 (A_3 A_4)))$

Given a chain of n matrices $\langle A_1, A_2, \dots, A_n \rangle$

The time complexity to multiply them with optimal parenthesization
 $= O(n^3)$

Proof:

~~ii~~
In tabulation method

$j-i = n-1 \rightarrow$ no of comparisons = $n-1$
no of instances = 1

$j-i = n-2 \rightarrow$ no of comparisons = $n-2$
no of instances = 2

$j-i = n-3 \rightarrow$ no of comparisons = $n-3$
no of instances = 3

$\therefore$ no of comparisons =

$$(n-1)(1) + (n-2)(2) + \dots + (n-k)(k) + \dots + (n-(n-1))(n-1) \\ = O(n^3)$$

19/10/20

Bellman Ford Algorithm:

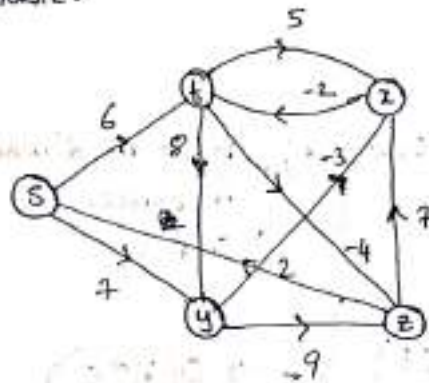
- * Single source shortest paths.
- * Dijkstra's may or may not work for -ve weight edged graph.
- * Bellman Ford does work with -ve edged graphs,

However, shortest path are determinable if ~~the cycle~~ every cycle of graph that includes -ve weight has to be a true weighted cycle.

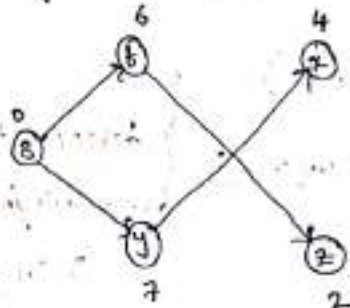
If the cycle containing negative edge is negative weighted then shortest path is not determinable.

In fact is not determinable with any algorithm.

→ The shortest path ~~at~~ of all the vertices that are reachable through negative weighted cycle is not determinable. ~~some~~ there could be vertices that are not part of negative weighted cycle but still have their shortest path not determinable.



Using Dijkstra's algorithm



Here shortest path from S to t is reported as 2.

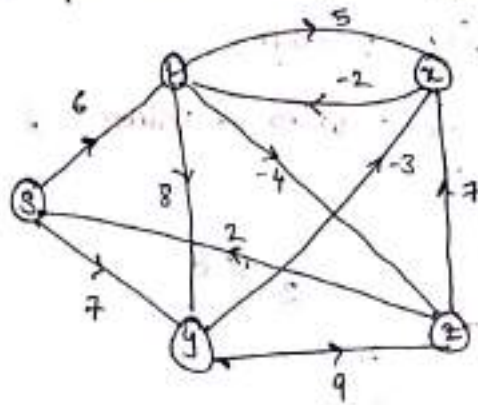
However the path S-y-x-t has a cost of 2.

Thus we say Dijkstra ~~doesn't work~~ may or may not have correct soln for -ve weight graph.

(∵ ~~Re~~ re-relaxation of path S-t (6) is never done in Dijkstra's algorithm)

Bellman ford approach:

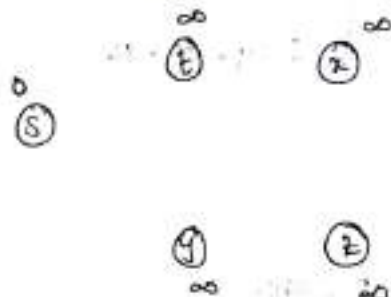
Consider the previous example again



Trace of point (2) in algo

Step 1:

Initialize d values:

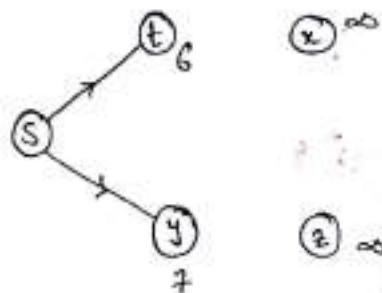


Refer Sahani for
optimization on
BellmanFord

Step 2: (point '2' in algo - 1st iteration) (edge paths with 1 edge)

→ no relaxation can be done using ~~value of~~ d values of
to x, y, z (∵ it is infinite)

→ In this step relaxation can be done only from vertex 1.

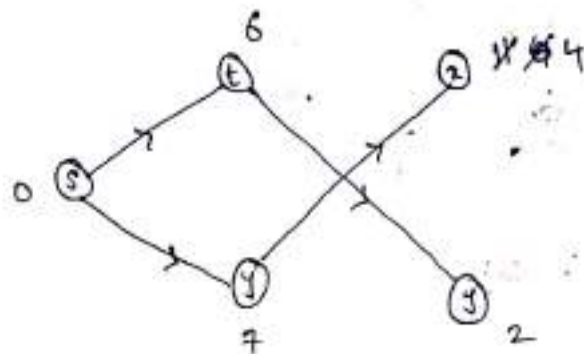


→ we perform this relaxation for every edge.

Step 3:

* Finding shortest paths with 2 edges

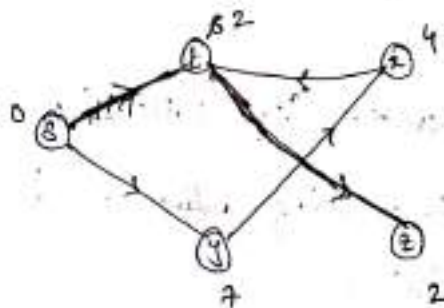
* Relaxation can be done using d' values of s, t, y



$$\begin{array}{l|l} S \rightarrow t \rightarrow x : 11 & S \rightarrow t \rightarrow y : 2 \\ S \rightarrow y \rightarrow x : 4 & S \rightarrow y \rightarrow z : 16 \end{array}$$

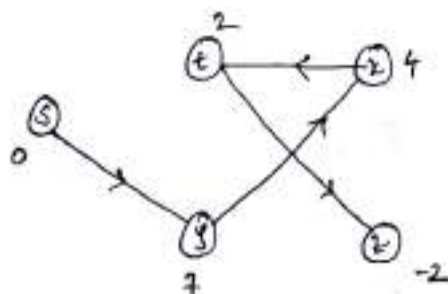
Step 4:

→ Finding shortest paths with 3 edges.



Step 5:

→ Find shortest paths with 4 edges



Derivation of formula:

let $c[i, j]$ be edge cost

$d^l[x]$ = cost of the path from source 's' to vertex 'x' with atmost 'l' edges.

$$d^l[x] = \min \left\{ d^{l-1}[x], \min_{k \in V} \{ d^{l-1}[k] + c[k, x] \} \right\}$$

without $l-1$ edges

$d^l[x] = c[s, x]$

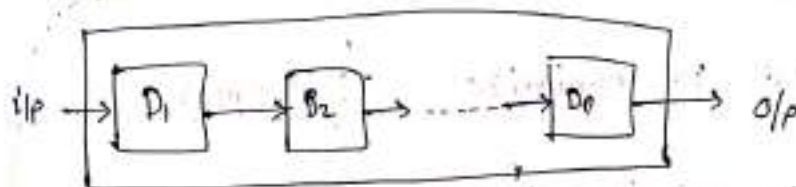
Reliability Design:

Reliable System Design

Design of an n-stage system

Here, design is in cascade

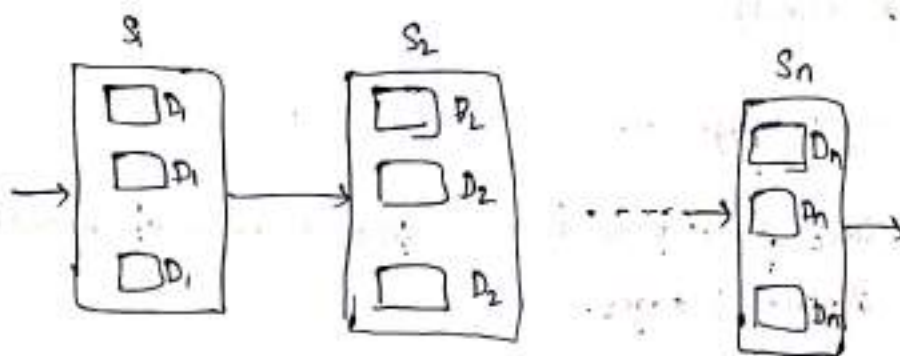
i.e. one design take o/p of another device as i/p



Every device D_i has a cost C_i and reliability r_i

→ So the problem is to maximize the reliability of the system such that cost of the design is less than given cost.

→ we increase reliability by adding more devices at each stage.



Let each stage S_i has m_i copies of device D_i such that

$$1 \leq m_i \leq u_i$$

where u_i is an upper bound.

→ Total cost of the system =
$$\sum_{i=1}^n m_i c_i$$

→ Reliability of the system is:

reliability of S_1

$$= 1 - (1 - r_1)^{m_1}$$

reliability of S_2

$$= 1 - (1 - r_2)^{m_2}$$

∴ Reliability of the design =
$$\prod_{i=1}^n [1 - (1 - r_i)^{m_i}]$$

Problem Defn:

Thus we need to maximize
$$\prod_{i=1}^n \phi_i(m_i)$$

subjected to
$$\sum_{i=1}^n m_i c_i \leq C \rightarrow \text{given cost}$$

where
$$\phi_i(m_i) = 1 - (1 - r_i)^{m_i} \text{ and } 1 \leq m_i \leq u_i$$

$$u_i = \left\lfloor \frac{C - \sum_{j=1}^i c_j + c_i}{c_i} \right\rfloor$$

Eg: Consider 3 stage system with

$$C=105; \langle c_1, c_2, c_3 \rangle = \langle 30, 15, 20 \rangle; \langle r_1, r_2, r_3 \rangle = \langle 0.9, 0.8, 0.5 \rangle$$

→ Let $f_n(c)$ represent reliability of an n -stage system with a total project cost of c ;

$$f_n(c) = \max_{1 \leq m_n \leq u_n} \left\{ f_n(m_n) * f_{n-1}(c - c_n m_n) \right\}$$

$$f_0(x) = 1$$

Eg: $n=3; C=105;$

$$\langle c_1, c_2, c_3 \rangle = \langle 30, 15, 20 \rangle$$

$$\langle r_1, r_2, r_3 \rangle = \langle 0.9, 0.8, 0.5 \rangle$$

$$u_1 = \left\lfloor \frac{105 - 15 - 20}{30} \right\rfloor = 2$$

$$u_2 = \left\lfloor \frac{105 - 30 - 20}{15} \right\rfloor = 3$$

$$u_3 = \left\lfloor \frac{105 - 30 - 15}{20} \right\rfloor = 3$$

Let $f_n \sim S^n = \{(\text{reliability}, \text{cost})\}$

∴ initially we have

$$S^0 = \{(1, 0)\}$$

For calculation S^1 we compute S^1_1 & S^1_2

where S^1_j represents i th stage & j th copies of stage i devices.

$$S_1' = \{(0.9, 30)\}$$

$$S_2' = \{(0.99, 60)\}$$

$$1 - (1 - 0.9)^2$$

$$1 - (0.1)^2$$

$$= 0.99$$

using S_1' & S_2' , we compute S'

$$S' = \{(0.9, 30), (0.99, 60)\}$$

Computing S^2 :

we compute S^2 by computing S_1^2, S_2^2, S_3^2

$$S_1^2 = \{(0.72, 45), (0.792, 75)\}$$

→ removed in purging rule.

$$\phi_2(m_2=2) = 1 - (0.8)^2 = 1 - 0.64 = 0.36$$

$$= 1 - (1 - 0.8)^2 = 1 - 0.04 = 0.96$$

$$S_2^2 = \{(0.864, 60), (0.9504, 90)\}$$

(X) $\hookrightarrow$ from here we cannot add even 1 copy of D_3 . so this path is not explored.

$$\phi_2(m_3) = 1 - (1 - 0.8)^3 = 1 - (0.2)^3$$

$$= 1 - 0.008 = 0.992$$

$$S_3^2 = \{(0.8928, 75), ($$

$\hookrightarrow$ infeasible

$$\therefore S^2 = \{(0.92, 45), (0.864, 60), (0.8928, 75)\}$$

Computing S^3 :

→ first we calculate S_1^3, S_2^3, S_3^3

$$S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$$

$$\phi_3(2) = 1 - (1 - 0.5)^2 = 1 - 0.25 = 0.75$$

$$S_2^3 = \{(0.54, 85), (0.648, 100), (\dots)\}$$

↳ infeasible

$$\phi_3(3) = 1 - (1 - 0.5)^3 = 1 - 0.125 = 0.875$$

$$S_3^3 = \{(0.63, 105)\}$$

↳ eliminated using pruning rule

$$\therefore S^3 = \{(0.36, 65), (0.432, 80), (0.54, 80), (0.648, 100)\}$$

∴ Ans: (0.648, 100)

Computing no of copies at each stage

(0.648, 100) is present in S_2^3

$$\Rightarrow m_3 = 2$$

now subtract ~~100~~ $2 * C_3 = 40$ from 100

$$100 - 40 = 60$$

now search for tuple $(-, 60)$ in S^2

$$\text{i.e., } (0.864, 60)$$

It is found in $S_2^2 \Rightarrow m_2 = 2$

Now search for $(-, 60 - (2)(15)) = (-, 30)$ in S'

i.e., found in S_1

$$\therefore m_1 = 1$$

$$\therefore (m_1, m_2, m_3) = (1, 2, 2)$$

$$\therefore \text{reliability} = (0.9) * (1 - (1 - 0.8)^2) * (1 - (1 - 0.5)^2) \\ = 0.648$$

Optimal Cost Binary Search Tree:

→ BST is used in compilers to maintain the keyword table. (contains all the keywords)

~~Cost of~~

Computing cost of BST:

Let T be a BST

$$\text{Cost}(T) = \text{Cost}(\text{successful searches}) + \text{Cost}(\text{unsuccessful searches})$$

$$\text{Cost}(\text{successful searches}) = \sum_{i=1}^n \text{Cost}(a_i)$$

where $\text{Cost}(a_i)$ is cost for searching a_i in T .

$$\therefore \text{Cost}(a_i) \propto \text{level}(a_i)$$

where $\text{level}(a_i)$ is the level of a_i in T

$$\text{Cost}(a_i) = P_i * \text{level}(a_i)$$

* where P_i is probability of using a_i in the application.

$$\therefore \text{Cost}(\text{succ. searches}) = \sum_{i=1}^n P_i * \underbrace{\text{level}(a_i)}_{\text{element comparisons}}$$

→ Now let us compute the cost of unsuccessful searches.

Unsuccessful searches end at null links.

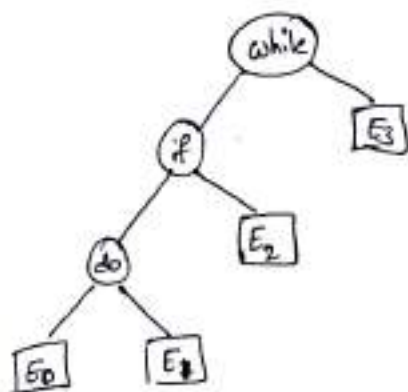
In a binary tree of 'n' nodes no of

$$\text{total links} = 2n$$

$$\text{no of used links} = n-1$$

$$\therefore \text{no of null links} = 2n - (n-1) = n+1$$

Eg: In BST with 3 nodes we have 4 null links.



whenever there is unsuccessful search it ends at ~~E0~~ E0/E1/E2/E3

Unsuccessful search ends at E0 if search word is < do

unsuccessful search ends at E1 if search word is (< if) and (> do)

~~cost of an~~

$$\therefore \text{cost}(\text{unsuccessful search}) = \sum_{i=0}^n \text{cost}(E_i)$$

$$\text{cost}(E_i) \propto [\text{level}(E_i) - 1]$$

$$\text{cost}(E_i) = q_i * (\text{level}(E_i) - 1)$$

↳ no of element comparisons.

where q_i is probability of searching a string from set E_i

$$\therefore \text{Cost}(\text{unsuccessful search}) = \sum_{i=1}^n q_i * (\text{level}(E_i) - 1)$$

$$\therefore \text{Cost}(T) = \sum_{i=1}^n P_i * \text{level}(a_i) + \sum_{i=0}^n q_i * (\text{level}(E_i) - 1)$$

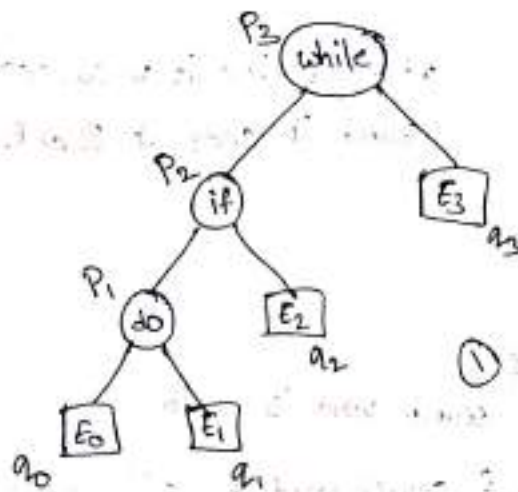
Ex: $n=3; \langle a_1, a_2, a_3 \rangle = \langle \text{do}, \text{if}, \text{while} \rangle$

$\langle P_1, P_2, P_3 \rangle = \langle 0.5, 0.1, 0.05 \rangle$

$\langle q_0, q_1, q_2, q_3 \rangle = \langle 0.15, 0.1, 0.05, 0.05 \rangle$

$\sum_{i=1}^n p(i) + \sum_{i=0}^n q(i) = 1$
Formally $q(i)$ is probability of searching key x such that $a_i < x < a_{i+1}$

Compute the cost below BST



Each E_i corresponds to class of string x such that $a_i < x < a_{i+1}$ also we assume $a_0 = -\infty; a_{n+1} = +\infty$

Sol:

The level of root is 1.

$$\text{Cost}(\text{suc. search}) = \sum_{i=1}^n \text{level}(a_i) * P_i$$

$$= \cancel{P_1 * 1 + P_2 * 2 + P_3 * 3} = P_1 * 3 + P_2 * 2 + P_3 * 1$$

$$= \cancel{0.5 + 0.2 + 0.15} = (0.5)(3) + (0.1)(2) + (0.05)(1)$$

$$= \cancel{0.85}$$

$$= 1.5 + 0.2 + 0.05$$

$$= \cancel{1.75} = 1.75$$

$$\text{Cost}(\text{unsuc. search}) = \sum_{i=0}^n (\text{level}(E_i) - 1) * q_i$$

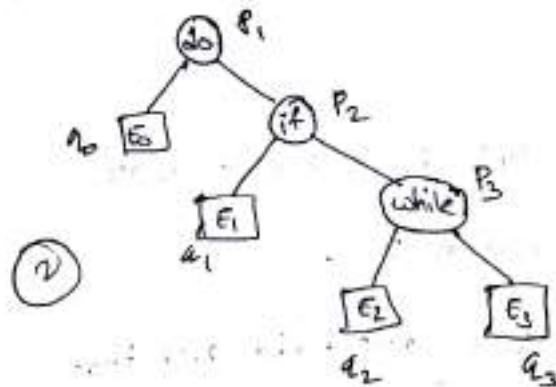
$$= (0.15)(3) + (0.1)(2) + (0.05)(1) + (0.05)(1)$$

$$= 0.45 + 0.2 + 0.1 + 0.05$$

$$= 0.8$$

$$\therefore \text{Cost}(BST) = \overset{1.75}{\cancel{0.85}} + 0.9 = \cancel{0.85} + 0.9 = 2.65$$

Eg: Compute cost of below tree



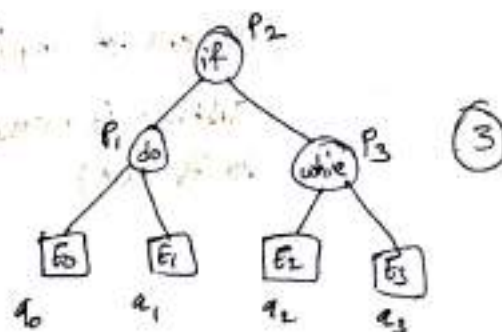
look at the array
If Tree in previous
example is considered,
then order of arrangement
of E_0, E_1, E_2, E_3 has
to be fixed in 2nd tree

$$\begin{aligned} \text{Cost}(\text{succ}) &= 1 \times 0.5 + 2 \times 0.1 + 3 \times 0.05 \\ &= 0.5 + 0.2 + 0.15 \\ &= 0.85 \end{aligned}$$

$$\begin{aligned} \text{Cost}(\text{unsucc}) &= (0.5)(1) + (0.1)(2) + (0.05)(3) + (0.05)(3) \\ &= 0.5 + 0.2 + 0.15 + 0.15 \\ &= 0.65 \end{aligned}$$

$$\therefore \text{Cost}(T) = 0.85 + 0.65 = 1.5$$

Eg: Compute cost of below tree



$$\text{Cost}(\text{succ}) = (0.5)(2) + (0.1)(1) + (0.05)(2) = 1 + 0.1 + 0.1 = 1.2$$

$$\begin{aligned} \text{Cost}(\text{unsucc}) &= (0.5)(2) + (0.1)(2) + (0.05)(2) + (0.05)(2) \\ &= 0.3 + 0.2 + 0.1 + 0.1 = 0.7 \end{aligned}$$

$$\text{Cost}(T) = 1.9$$

even though Tree 3 is Complete it has higher cost than Tree 2.

Construction of BST of optimal cost:

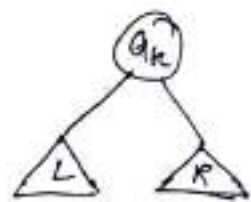
Let us consider n identifiers ~~with~~ $\langle a_1, \dots, a_n \rangle$ associated with probabilities $\langle p_1, \dots, p_n \rangle$. Let probabilities of external nodes be $\langle q_1, \dots, q_n \rangle$.

Assume $\langle a_1, a_2, a_3, \dots, a_n \rangle$ are in sorted order.

Let $\text{Cost}(T)$ denote Cost of BST.

Now we select a_k as root node such that

$$\text{Cost}(T) = \min_{1 \leq k \leq n} \{ p_k + \text{Cost}(L) + \text{Cost}(R) \}$$



Mathematical modelling:

$$C(0, n) = \min_{k=1 \dots n} \left\{ p_k + \underbrace{C(0, k-1)}_{+\Delta_1} + \underbrace{C(k, n)}_{+\Delta_2} \right\}$$

~~However~~ the

The value calculated are calculate for subtree considering it as a tree. So level number will be less. So we add $\Delta_1 + \Delta_2$

(The level of any identifier within subtree is measured with respect to its root)

Heap Algorithms:

Heap is used in the implementation of priority queue.

Defo: It is a complete binary tree with the property that the value of a node is greater (smaller) than the values of the nodes in their left & right subtrees.

→ In general it is implemented as binary heap. However we may use 3-ary or 4-ary or k-ary heap.

Operations:

- i) Create
- ii) Insert
- iii) Delete
- iv) Inc/Dec key

Insertion:

i) Inserting elements one after other:

Time complexity:

{ Best case : $O(n)$ (Dec order)
 { Worst case : $O(n \log n)$ (Inc order)

Framework:

- The max no of nodes at any level 'i' is 2^{i-1} (not at level 1)
- The no of level comparisons/movements of a node being inserted at level 'i' = $i-1$

→ The total no of level comparison for all nodes at level i

$$= (i-1) 2^{i-1}$$

→ Time for all nodes at all levels

$$= \sum_{i=1}^k (i-1) 2^{i-1}$$

where k is last level

$$= \frac{1}{2} \left[\sum_{i=1}^k i \cdot 2^i - \sum_{i=1}^k 2^i \right]$$

$$= \frac{1}{2} \left[(k-1) 2^{k+1} + 2 - (2^{k+1} - 2) \right]$$

$$\therefore \sum_{i=1}^n i \cdot 2^i = (n-1) 2^{n+1} + 2$$

$$= \frac{1}{2} \left[k \cdot 2^{k+1} - 2^{k+1} + 2 - 2^{k+1} + 2 \right]$$

$$= \frac{1}{2} \left[2^{k+1} (k-2) + 4 \right]$$

$$= 2^k (k-2) + 2$$

For a full binary tree / complete binary tree

if 'n' is no. of nodes

$$2^k \approx O(n) \quad \& \quad k = O(\log n)$$

$$\Rightarrow n(\log n - 2) + 2$$

$$= O(n \log n)$$

22/10/20

27

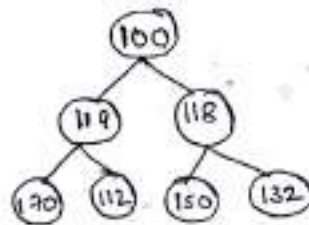
Build Heap (Heapify):

Heapify: The process of transforming a given complete binary tree (with n elements) to a heap using method of adjust.

Adjustment procedure is carried out level by level starting from level $k-1$ then $k-2$ upto level 1.

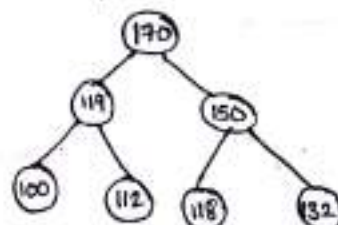
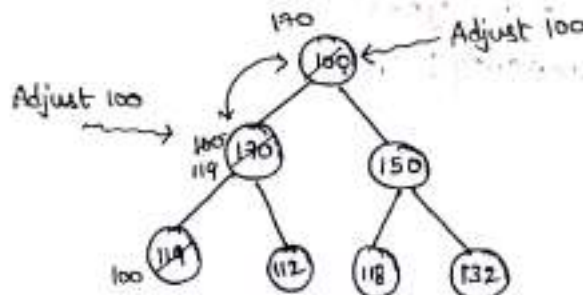
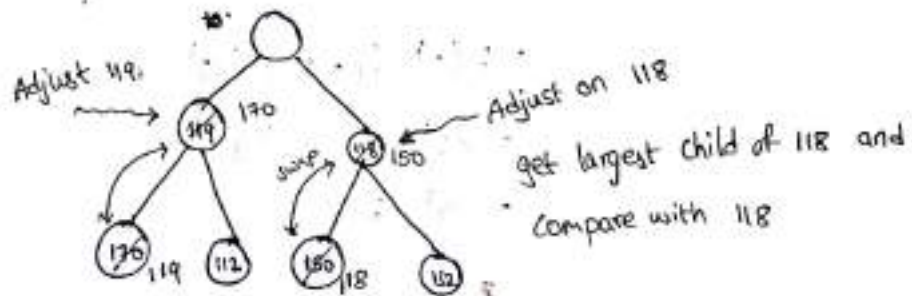
where k is last level

Ex: A: $\langle 100, 119, 118, 170, 112, 150, 132 \rangle$



100	119	118	170	112	150	132
-----	-----	-----	-----	-----	-----	-----

↓ Heapify



170	119	150	100	112	118	132
-----	-----	-----	-----	-----	-----	-----

Time Complexity:

→ Max no. of nodes at any level is 2^{i-1}

→ Max no. of level comparisons for a node that is getting adjusted at level i is $k-i$

where k is height (or, last level)

→ for all node at level i , total comparisons = $(k-i)2^{i-1}$

→ Time Complexity = total comparison for all nodes at all levels

$$= \sum_{i=1}^k (k-i) 2^{i-1}$$

$$= k \sum_{i=1}^k 2^{i-1} - \sum_{i=1}^k i \cdot 2^{i-1}$$

$$= k \cdot \frac{2^k - 1}{2 - 1} - \frac{1}{2} \sum_{i=1}^k i \cdot 2^i$$

$$= k(2^k - 1) - \frac{1}{2} [(k-1)2^{k+1} + 2]$$

$$= k(2^k - 1) - (k-1)2^k - 1$$

$$= k2^k - k - k2^k + 2^k - 1$$

$$= 2^k - k - 1$$

$$k = \text{height} = O(\log n)$$

$$= n - \log n - 1$$

$$\therefore \text{time complexity: } O(n)$$

Heap Sort:

Algo Heap_Sort(n)

{

1. Heapify(n) $\longrightarrow O(n)$

2. for $i \leftarrow n$ down to 2

{ swap($A[i]$, $A[1]$); $O(1)$

} Adjust($A[i]$); $\longrightarrow O(\log n)$

}

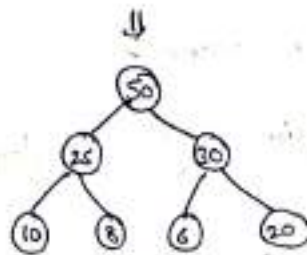
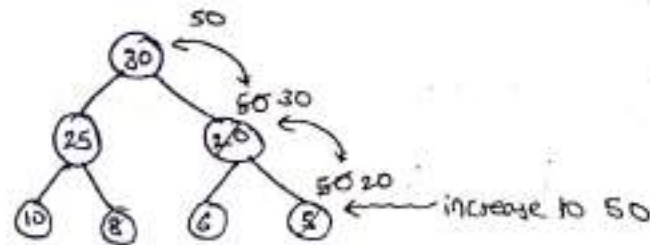
}

$\left. \begin{array}{l} \text{swap} \\ \text{Adjust} \end{array} \right\} O(n \log n)$

$\therefore$ Time complexity of heap sort : $O(n \log n)$

Increasing / Decreasing value of a Node :

Consider Max heap



Thus if it is increase operation, the node may go from ~~root~~ leaf to root in worst case and if it is delete operation, the node may go from root leaf.

$\therefore$ TC for inc/dec operation : $O(\log n)$

(H/61)

1st smallest element cannot be below 1st level

2nd smallest element cannot be below 2nd level

Similarly 7th smallest cannot be below 7th level.

$\therefore$ we need to search all 1st 7 levels.

$\therefore 2^7 - 1 = 128 - 1 = 127$ nodes have to be searched

$\therefore O(1)$

H/63

Q

options are

- a) $\log n$ b) $\sqrt{\log n}$ c) $O(1)$ d) $\frac{\log n}{\log \log n}$

option verification:

If k is no. of element can be sorted in $O(\log n)$ then

$$k \log k \geq \log n$$

- a) ~~$\log n$~~ $\log n$ elements

$\log n$ elements require $O(\log n (\log \log n))$ time.

$$O(\log n \cdot \log \log n) > O(\log n)$$

$\therefore$ false

- b) $\sqrt{\log n}$ elements

$\sqrt{\log n}$ elements require $O(\sqrt{\log n} (\log \log n))$ time

$$O\left(\frac{1}{2} \sqrt{\log n} \cdot \log \log n\right) \neq O(\log n)$$

$\therefore$ false.

- c) ~~constant time is~~

$O(1)$ --- Constant elements.

to sort constant no. of elements constant time is required

$$O(1) \neq O(\log n)$$

- d) to sort $\frac{\log n}{\log \log n}$ we need $O\left(\frac{\log n}{\log \log n} \cdot \log\left(\frac{\log n}{\log \log n}\right)\right)$

$$= O\left(\frac{\log n}{\log \log n} (\log \log n - \log(\log \log n))\right)$$

$$= O\left(\log n \left(1 - \frac{\log(\log \log n)}{\log \log n}\right)\right)$$

$$= O\left(\log n - \underbrace{\frac{(\log n)(\log \log \log n)}{\log \log n}}_{< \log n}\right)$$

$$= O(\log n)$$

$$\log \log \log n < \log \log n$$

$$\Rightarrow \frac{\log \log \log n}{\log \log n} < 1$$

$$\Rightarrow \frac{\log n \cdot (\log \log \log n)}{\log \log n} < \log n$$

(H/64) i) Take 1st element from each list and build a heap H .
this takes $\log \log n$ time.

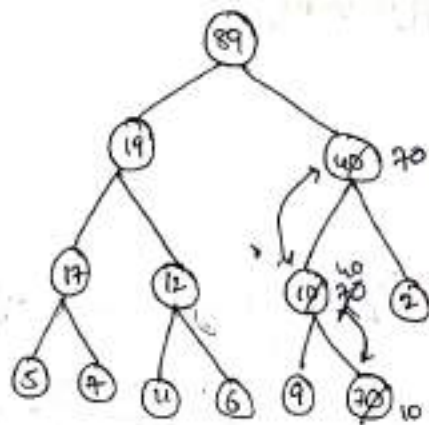
ii) Delete an element from this heap and put it in ~~the~~
the array (the array in which we want to store the sorted
result) --- $O(\log \log n)$.

iii) Now take an element from ~~array~~ list ~~for~~ whose
element got added in the array and insert into heap H
--- $O(\log \log n)$.

Repeat ~~step i~~ step (ii) & (iii) until all the elements ~~are~~ from
the list are finished i.e., n times

$$\text{i.e., } n \log \log n$$

$$\therefore \text{TC} : O(n \log \log n)$$



$\therefore$ 2 interchanges

Graph Techniques (Traversals):

Traversal: Visiting all the nodes of the tree/graph in an order & processing the information only once is traversal.

Tree traversal vs Graph traversal:

→ The traversal of tree (inorder or preorder or postorder) is unique whereas graph traversal need not to be unique.

Framework:

1. Status of a node:

During the traversal, any node ~~can~~ must be in one of the 3 states

a) E-Node:

The node that is currently under exploration.

There is only one E-node during traversal.

b) Live - Node :

The node which has not yet been explored fully or all of whose children has not yet been explored.

- stack in DFS & queue in BFS are used to store these node.

c) Dead - Node :

The node which has been explored fully.

2. Timing values during traversal :

→ every node is associated with 2 timing values

d = discovery time (Pre-value)

f = finishing time (Post-value)

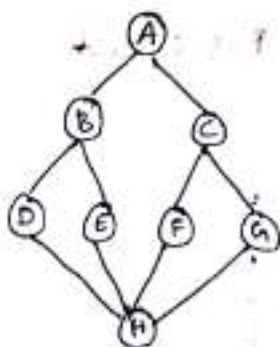
$$d \neq f > 0$$

d : the time at which node is first visited.

f : the time at which node becomes dead.

DFS :

1) DFS in undirected connected graph :

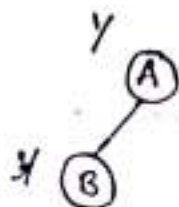


Let us start DFS from A

So initially we give discovery time of $A=1$

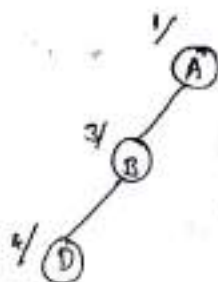
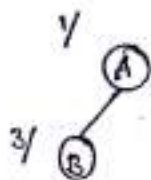
(However we can have any value greater than 0)

Now current A is E-Node.

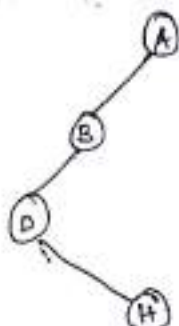
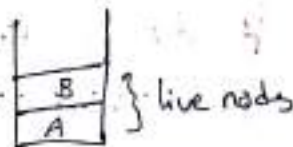


A; B

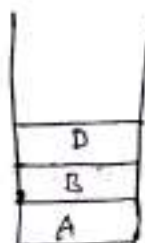
Now B becomes E-node, & A becomes live node and A is stored in stack



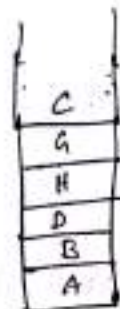
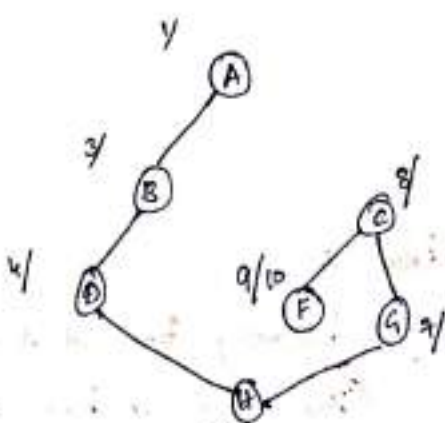
A; B; D



A; B; D; H



Here D is parent of H and E, F, G, and children of H



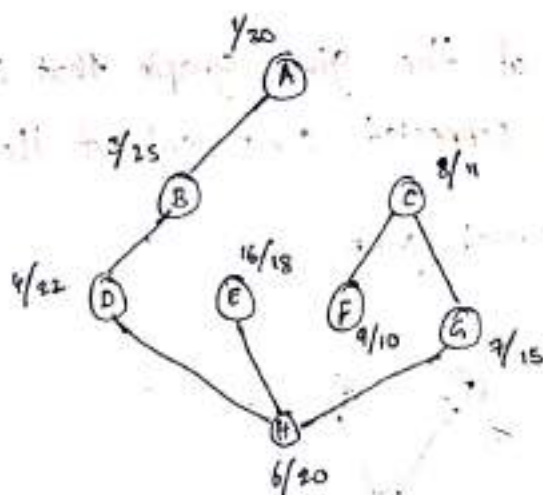
A; B; D; H; G; C; F

Now F has no adjacent unexplored nodes.

So we assign it finish time and 'F' becomes dead node.

Now pop off 'C' from the stack.

Now C becomes the E-node



In this traversal H becomes an E-node thrice

∴ traversal: A, B, D, H, G, C, F, E

Note:

→ ~~The graph is~~ ...

→ The undirected graph is connected

⇔ finish time of first node is the highest

⇔ traversal finishes when the stack becomes empty for first time.

⇔ traversal finishes when the first node becomes dead node

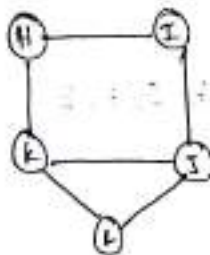
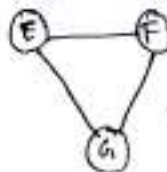
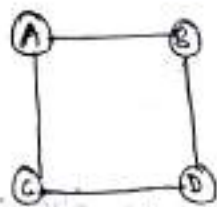
2) DFS in undirected disjoint (disconnected graph):

→ DFS when carried out on disconnected graph is called Depth First Tree (DFT).

Connected Component:

Maximal subgraph of the given graph that is connected is called ~~com~~ connected component of the graph.

Consider below disconnected graph



Start traversal from any of the unexplored node

1/8

2/7

(A)

(B)

(E)

(F)

(G)

(C)

(D)

4/5

3/6

(H)

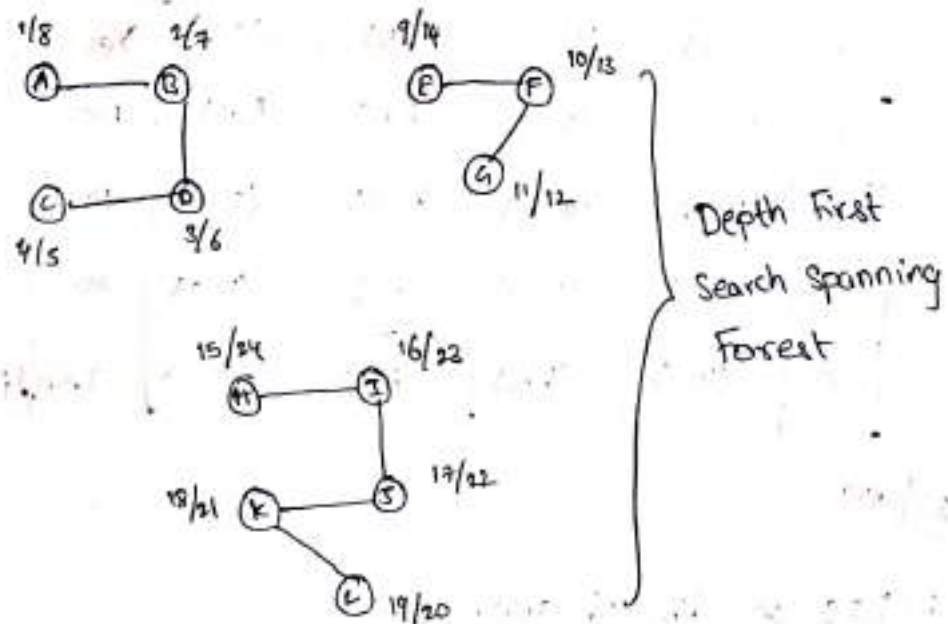
(I)

(K)

(J)

(L)

Now stack has become empty but still traversal is not finished since we have unexplored nodes.



Here finishing time of first node (A) is not the highest finishing time.

∴ disconnected

Also, graph is disconnected if there are more than one spanning trees.

no of disconnected components = no of spanning trees in depth first traversal.

- Q. Given an undirected graph with 4 vertices (P, Q, R, S) and their (d, f) values as follows. Indicate for each option whether the graph is connected or disconnected and no of connected components.

	Vertices				Connected	no of connected components
	P	Q	R	S		
	$\langle d, f \rangle$	$\langle d, f \rangle$	$\langle d, f \rangle$	$\langle d, f \rangle$		
a)	$\langle 5, 20 \rangle$	$\langle 6, 18 \rangle$	$\langle 8, 15 \rangle$	$\langle 10, 12 \rangle$	Yes	1
b)	$\langle 13, 14 \rangle$	$\langle 7, 11 \rangle$	$\langle 6, 12 \rangle$	$\langle 8, 10 \rangle$	No	2 (GRS) (P)
c)	$\langle 3, 10 \rangle$	$\langle 18, 20 \rangle$	$\langle 13, 15 \rangle$	$\langle 5, 8 \rangle$	No	3 PS R E
d)	$\langle 9, 10 \rangle$	$\langle 15, 20 \rangle$	$\langle 6, 8 \rangle$	$\langle 11, 12 \rangle$	No	4 R P S E
e)	$\langle 10, 11 \rangle$	$\langle 6, 12 \rangle$	$\langle 8, 13 \rangle$	$\langle 15, 14 \rangle$	Invalid	

23/10/20

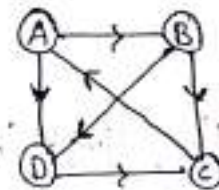
3) DFS on directed graph:

→ DFS when carried out over a directed graph leads to the following types of edges:

a) Tree edge:

The edges that are part of depth first search tree / forest

Eg: AB, BC, BD



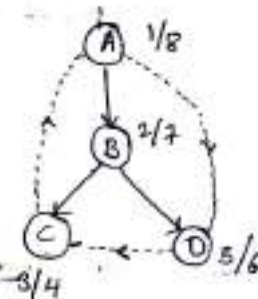
b) Forward Edge:

It is an edge that leads from a node to its non-child descendent.

(This edge is part of graph but not of spanning tree / forest)

Eg: AD is the only forward in the spanning tree

(∵ D is non-child descendent of A)



c) Back Edge:

It is an edge that leads from a node to its ancestor (This edge is present in graph but not in spanning tree).
 Eg: CA

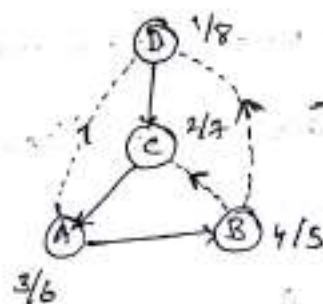
d)

d) Cross Edge:

It is an edge that leads from a node to another node which is neither ancestor nor descendent.

Eg: DC (This edge is also part of graph but not of tree)

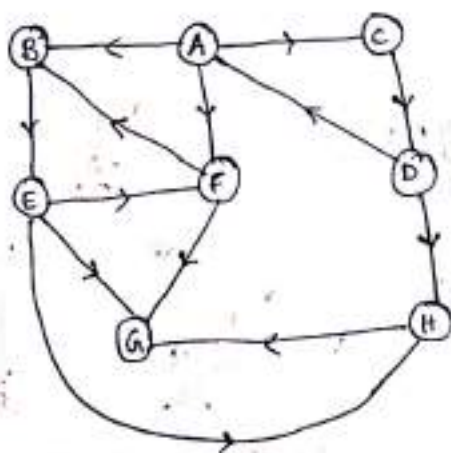
Eg: Consider below below DFS on node 'D'

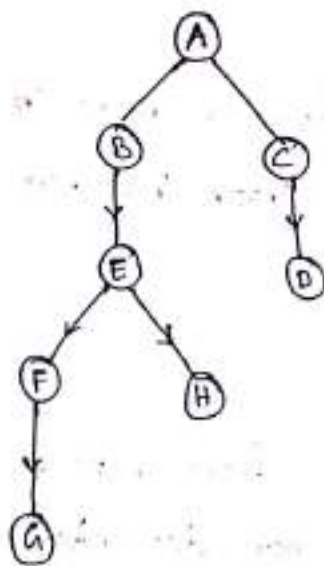


Fully decorated DFS spanning tree...
 (The DFS spanning tree in which all the types of edges are drawn)

DC, CA, AB are tree edges
 AD, BC, BD are back edges

Eg:





For above given graph and a spanning tree,

check whether the given ~~span~~ DFT tree is valid or not.

If it is valid find no of forward edges, backward edges and cross edges. and also mark discovery & finishing times.

Sol :

It is a valid depth first traversal spanning tree.

The order in which nodes are visited is

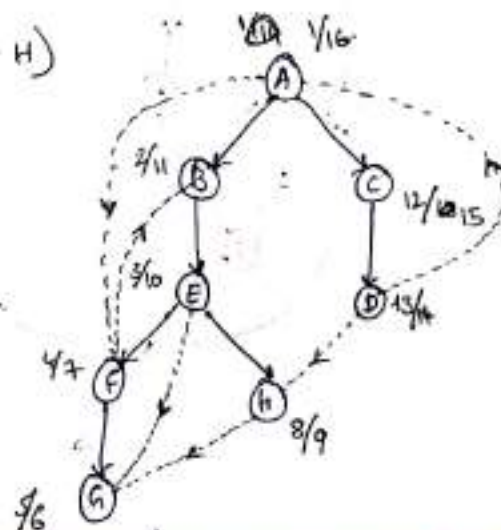
A; B; E; F; G; H; C; D

(Think why not C)
(Think why not H)

Forward Edges: AF; EG;

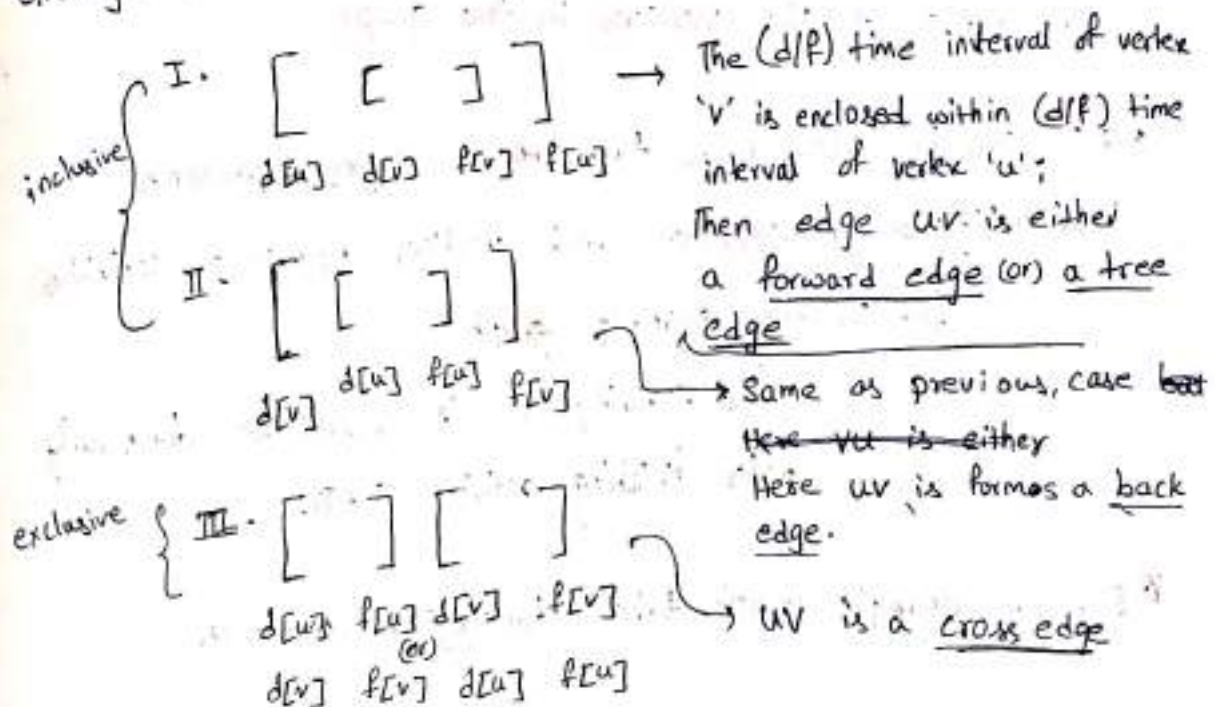
Back Edges: DA; FB;

Cross edges: DH; HG;



Parenthesis Theorem:

In any DFS of a directed graph $G=(V,E)$,
for any two vertices 'u' & 'v' having ^{directed from u to v.} an edge ~~between them~~;
exactly one of the condition holds



4) DFS on DAG (Directed Acyclic Graph) (Precedence Graph)

Every DAG is characterized by two kinds of vertices

(i) Source Vertices:

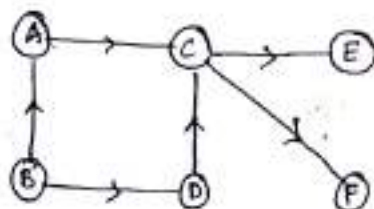
Vertices that doesn't have any precedences.

i.e., vertices that don't have any incoming edges

(ii) Sink Vertices:

Vertices that don't have any out bounded (outgoing) edges.

Eg:



B $\rightarrow$ source vertex

E, F $\rightarrow$ Sink vertices.

→ One of application of DAG is to perform topological sort (or) topological ordering.

→ Topological sort :-

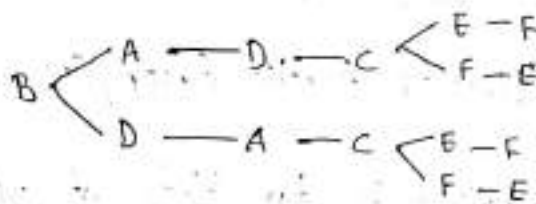
It is a linear ordering that exhibits the precedences of the vertices in the graph.

DAG can be viewed as task where edges represents dependencies or precedences and vertices represent activities.

Eg: In the example graph

Activity C should be done only after finishing activity A & D.

Eg: For example graph taken topological order is



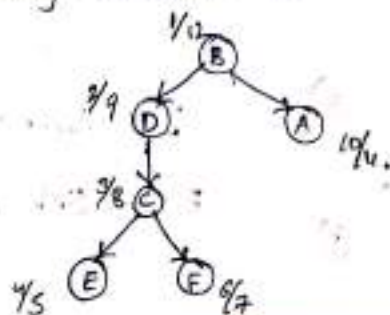
∴ topological sorts possible are

BADCEF
BADCFE
BDACEF
BDACFE

∴ no of topological orders possible = 4

Finding topological sort from DFS

Consider starting DFS from 'B'

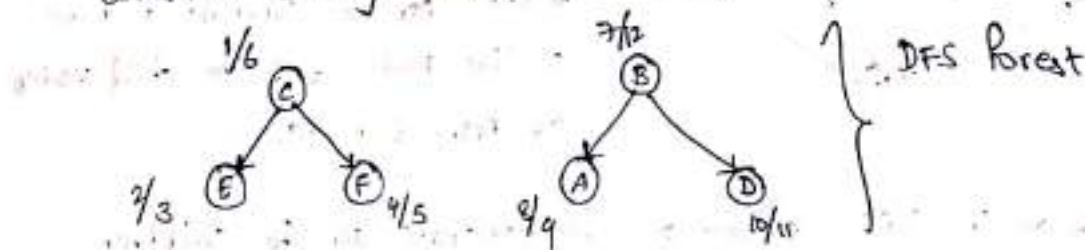


The decreasing order of finishing times gives topological sort.

i.e., BADCFE for this traversal

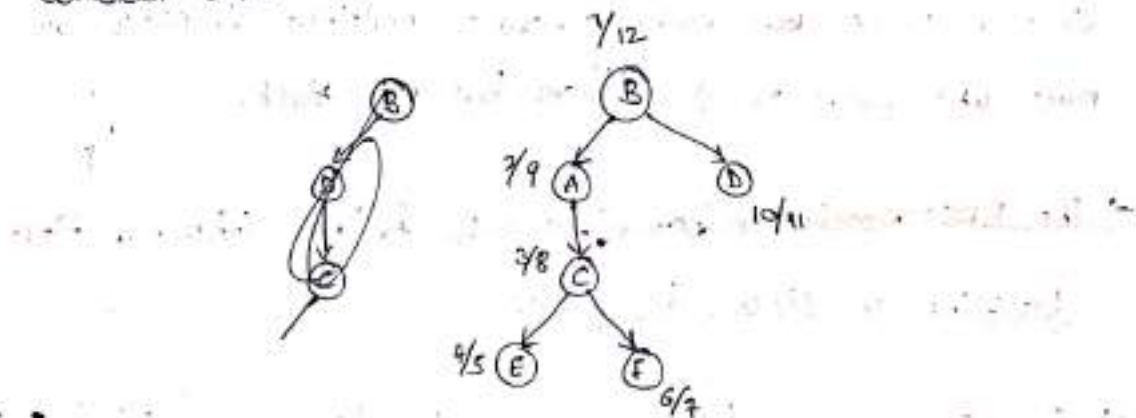
Note that to obtain topological sort we can start from any vertex.

Consider starting from vertex 'C'



∴ Topological sort: BDACBFE

Consider another traversal



∴ topological sort: BDACFE

Consider another traversal



topological sort: BADCFE

Algo topological_sort()

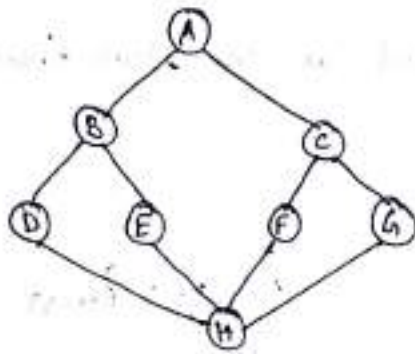
{

DFS(v); // v can be any vertex.

Print nodes in descending order of their finishing times

}

Breadth First Search (BFS)



In DFS, when we explore a node, we generate a child node of current E-node and make ~~the~~ the current E-node as live node and the child nodes as new E-node.

whereas in BFS, ~~after~~ we generate all the children of current E-node and the unexplored nodes of these nodes are made live nodes (stored in the queue)

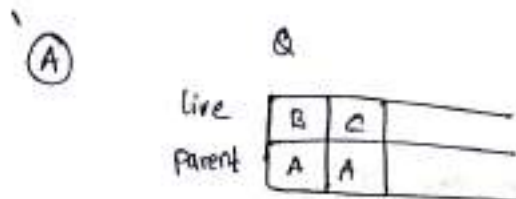
Since a single E-nodes generates multiple live nodes, we must use queue to store them but not stack.

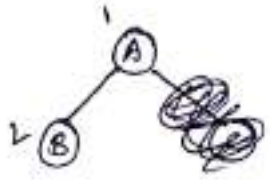
~~→ The BFS ~~data~~ can be either B follows either a FIFO discipline or LIFO discipline.~~

→ The queue may follow any of the below 3 disciplines.

- (i) FIFO (FIFO BFS) → taken as default
 - (ii) LIFO (LIFO BFS or Dsearch)
 - (iii) Priority Queue (Least Cost BFS)
- } Corresponding BFS

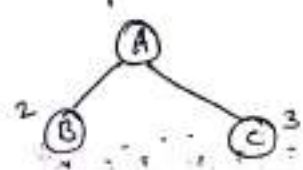
FIFO BFS





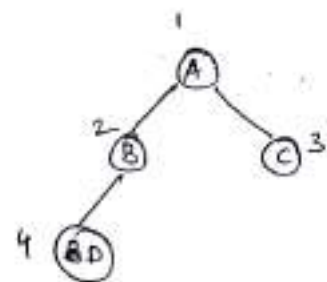
live	B	C	D	E	
parent	A	A	B	B	

Now obtain C from queue and add to tree
↳ as E-node



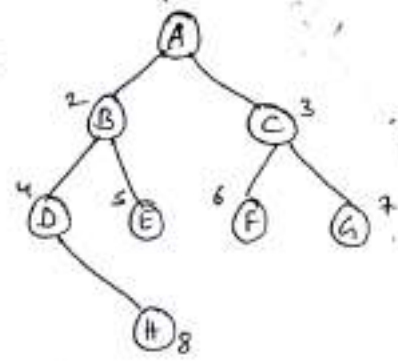
B	C	D	E	F	G
A	A	B	B	C	C

Now choose obtain D as next E-Node



B	C	D	E	F	G	H
A	A	B	B	C	C	D

Choose E as next E-Node

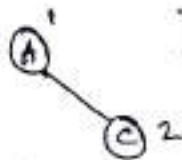


B	C	D	E	F	G	H
A	A	B	B	C	C	D

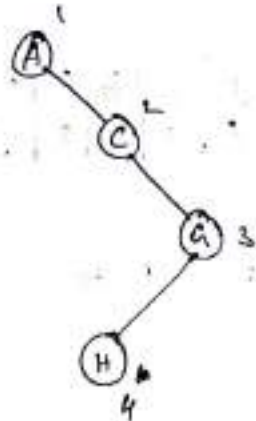
Breadth First Search - Spanning tree

This is FIFO BFS and the order of visiting will be the order in which they are pushed in queue

LIFO BFS:

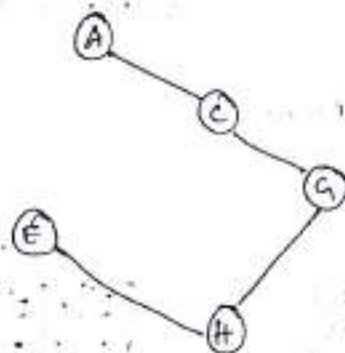


B	A	F	G
A	A	C	C



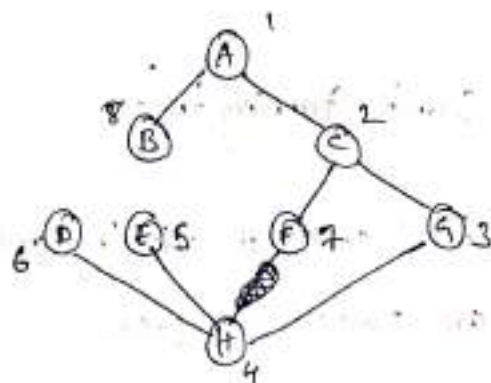
B	A	F	A	H	D	E
A	A	C	C	G	H	H

Choose E



B	A	F	A	H	D	E
A	A	C	C	G	H	H

choose D



B	A	F	A	H	D	E
A	A	C	C	G	H	H

At every node we move to depth.

Hence we call it D-search (depth-search)

D-search sequence is A C G H E D F B

↳ (This is invalid DFS)

Note:

~~Even~~ D-search need not to be a depth first search.

Priority Queue (LC-BFS)

→ This is used in - gaming algorithms

Eg: Consider 15-puzzle problem

1	3	4	15
2		5	12
7	6	8	14
8	9	10	13

Initial arrangement

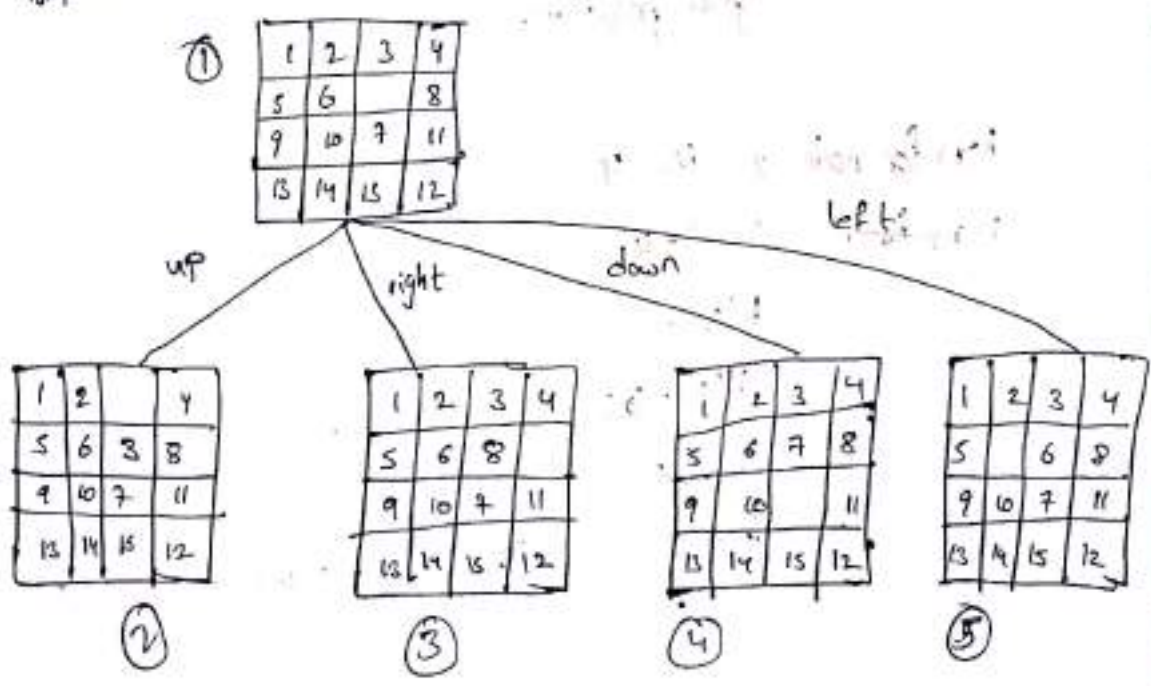
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Goal Arrangement

The go objective is to reach goal arrangement with min possible moves.

we LC-BFS for this

Based empty slot we may have 4 (or) 3 (or) 2 moves at each step



→ Using FIFO or LIFO here is a blind search.

→ So we need to prioritize one of the live nodes and make it as E-node.

Thus to perform every A* BFS, every live node is associated with a cost.

We select a node with minimum cost and hence the priority queue is min-heap.

→ Cost function of a live node is indicated as $\hat{C}(x)$

$$\hat{C}(x) = f(x) + h(x)$$

$f(x)$ → cost of reaching node x from root.

$h(x)$ → estimated cost of reaching goal state from node x .

Formulating $h(x)$ may vary from node to node.

→ One way to estimate $h(x)$ in this problem is

$h(x) = \text{no of non-blank tiles that are not in goal position.}$

~~$h(x)$ for node 2 is 4~~

~~$h(x)$ for~~ $h(2) = 4;$

$$h(3) = 4$$

$$h(4) = 2$$

$$h(5) = 4$$

$$\Rightarrow \hat{C}(2) = 5; \hat{C}(3) = 5; \hat{C}(4) = 3; \hat{C}(5) = 5;$$

Thus we explore ~~any~~ node '4'.

i.e., ~~that~~ make 4 as E-node.

→ After exploring node '4' we get 3 more live nodes and total live nodes at this point = 6.

Now we choose least cost node and continue the process.

Time Complexities:

→ The time complexity of both DFS & BFS with

(i) adjacency matrix $O(n^2)$

(ii) Adjacency list $O(n+e)$

Applications:

→ Both BFS & DFS can be used to determine the presence of cycle in graph.

→ either using DFS or BFS
The presence of backedge in directed (back/forward in undirected) confirms the presence of cycle.

→ DFS & BFS can be used to know existence of a path b/w a given pair of vertices.

→ DFS is used as a searching strategy in backtracking design strategy.

→ BFS is used as a searching strategy in branch & bound strategy.

→ For undirected graph, the BFS spanning tree will act as the solution for the single source shortest path problem by considering unit wt. cost graph.
Optimal ~~graph~~ algorithm

→ DFS is used to find ~~conn~~ Components & Articulation points



Components:

(i) Connected Components:

Maximal subgraph that is connected is called connected component of given graph.

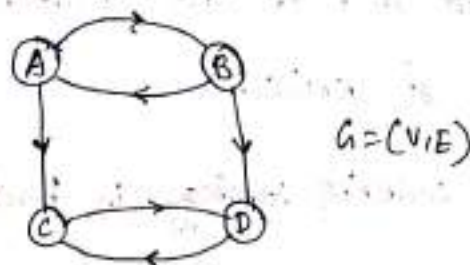
This concept applies only of undirected graphs.

(ii) Strongly Connected Components (This concept applied to directed graphs)

Two vertices 'u' & 'v' in a directed are said to be connected, iff there is a path from 'u' to 'v' and 'v' to 'u'.

→ This relation of being connected, partitions the vertex set 'V' of the graph into maximal disjoint sets known as strongly connected components.

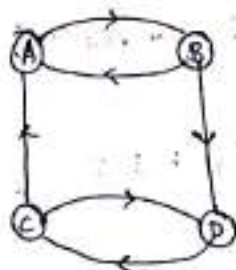
Eg:



The above graph has 2 strongly connected components
i.e., $\{A, B\}$ $\{C, D\}$



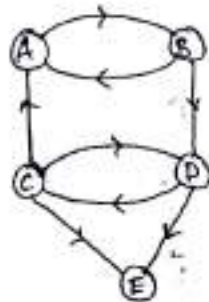
Ex:



no of strongly connected components = 1

i.e., $\{A, B, C, D\}$

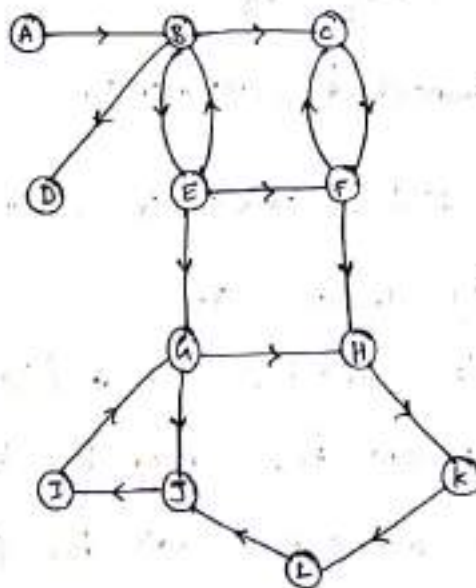
Ex:



no of strongly connected components = 2

i.e., $\{A, B, C, D\}$ & $\{E\}$

Ex:



Sol:

A has 1 out going edge & D has only 1 incoming edge

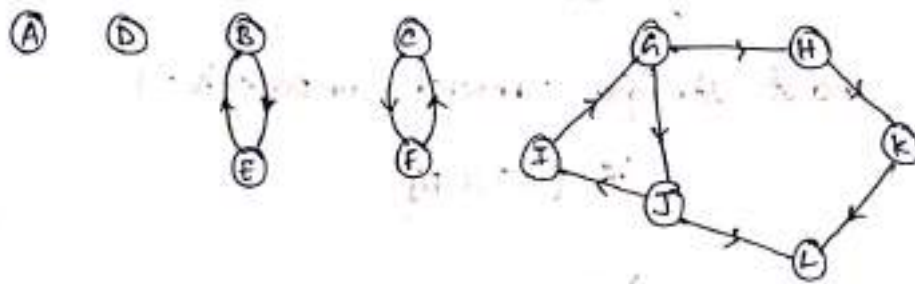
$\therefore \{A\}$ $\{D\}$ are 2 strongly connected components

G H K L J I is a cycle •

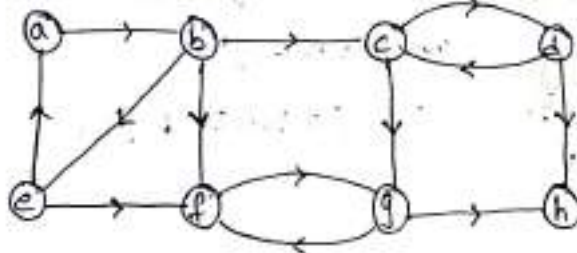
$\therefore \{G, H, K, L, J, I\}$ is a strongly connected component

∴ we have 5 strongly connected components

$\{A\}$ $\{D\}$ $\{B, E\}$ $\{C, F\}$ $\{G, H, K, L, J, I\}$



Eg:



$\{a, b, e\}$ $\{f, g\}$ $\{c, d\}$ $\{h\}$

∴ 4 strongly connected components.

Approach for this kind of problem is create a set initially containing vertex a.

Now check if 'b' is connected with 'a' (i.e., path present ~~from~~ from present from a to b & b to a.)

Since b & a are connected add b. $\{a, b\}$.

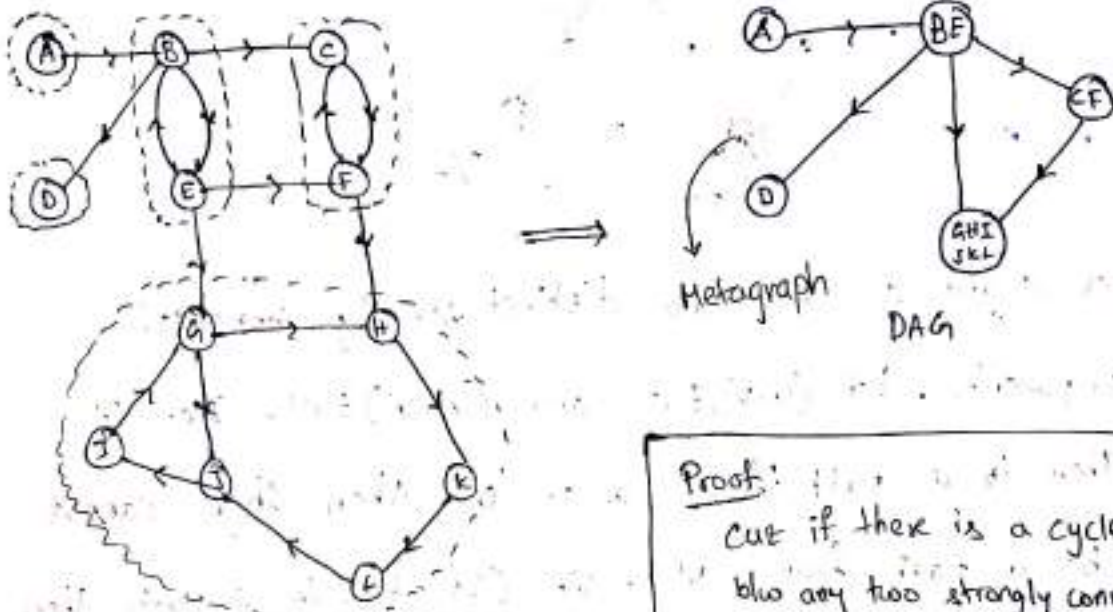
Now check with c.

C is not connected with 'a'. So create a new set containing c.

Continue this process for all the vertices.

Properties:

- 1) Every directed graph is a DAG of its strongly connected components.

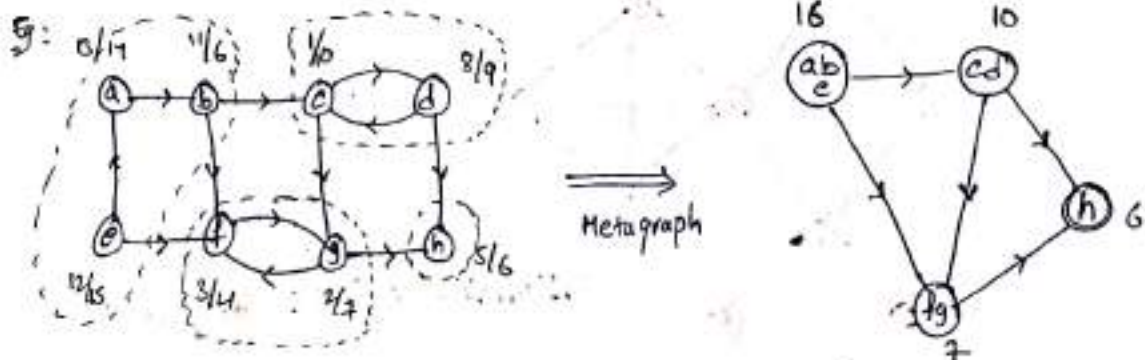


Metagraph: If $G=(V, E)$ is graph then $G'=(V, E')$ such that $E' \subseteq E$ is called metagraph. i.e. metagraph is a subgraph with all the vertices in graph.

Proof:

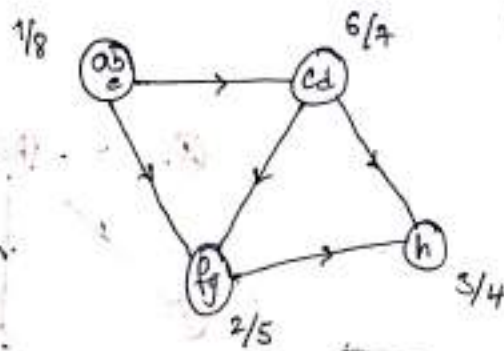
cut if there is a cycle b/w any two strongly connected components then the two components would become one.

- 2) Let C & C' be two strongly connected components. If there is a path from a vertex in C to a vertex in C' , then the highest finishing time of C will always be greater than the highest finishing time (of any vertex) of C' . (Think why)



The property can be verified from above graph

Consider carrying DFS on DAG of connected components, of a directed graph. The same property holds again



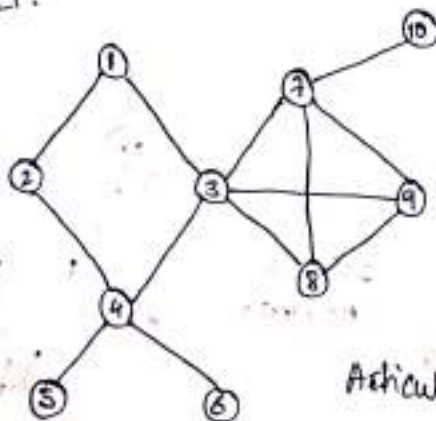
- 3) Let C and C' be two distinct strongly connected components. Let $(u, v) \in C$ and $(u', v') \in C'$. Then if there is a path from u to u' , then there cannot be a path from v' to v . (If there is a path then the whole thing forms a single connected component)

Articulation Point (cut vertex): & Biconnected Components

The vertex whose removal makes ^{connected} an undirected graph ~~is~~ disconnected is called articulation point.

→ The graph is said to be biconnected if it does not have articulation point.

Eg:



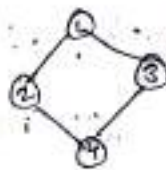
Articulation points: 3, 4, 7

Biconnected Components:

Maximal subgraph that is biconnected is called biconnected component of graph G .

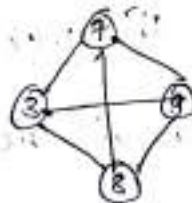
→ To find no of biconnected ~~graph~~ components in a given graph note that biconnected points form at the point where articulation points are present.

i.e., for 3 & 4 draw all the vertices adjacent to 3 & 4

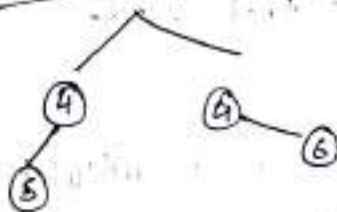


now choose 3 again

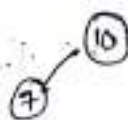
7 also gets added since it is adjacent to 3.



choose 4

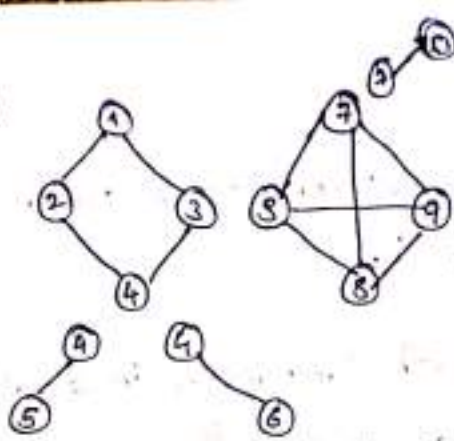


choose 7



∴ no of biconnected components for this graph = 5

i.e.,



→ Minimum no of edges that required to be added to make a graph biconnected $\leq$ no of articulation points.

Ex: In the previous graph, adding edges

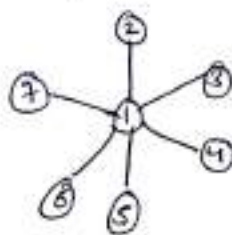
(1,10) (2,5) (8,6) will make the graph biconnected.

∴ However adding only two edges (5,10) & (6,8) will also will also make the graph biconnected.

(or) adding (1,5) & (6,10) will also make the graph biconnected.

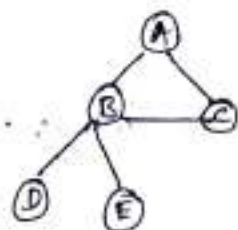
→ This formula generally applies to all graph except to star graph or graph of that kind.

Ex:



Here no of articulation points = 1.
However adding (≤ 1) edges won't make the graph biconnected.

Ex,

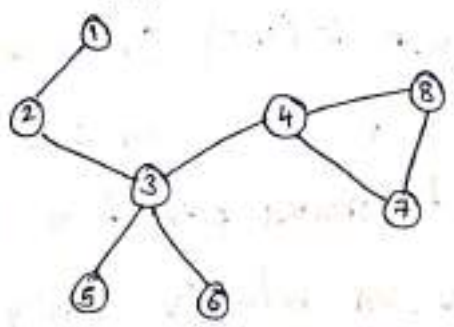


B is the articulation point:

no of articulation points = 1.

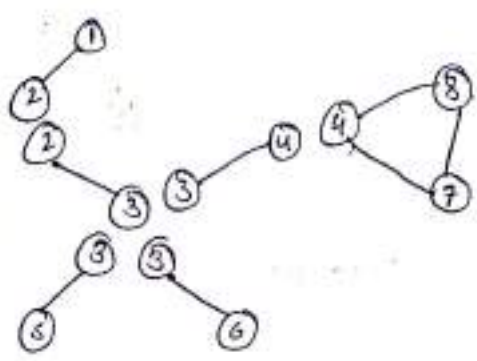
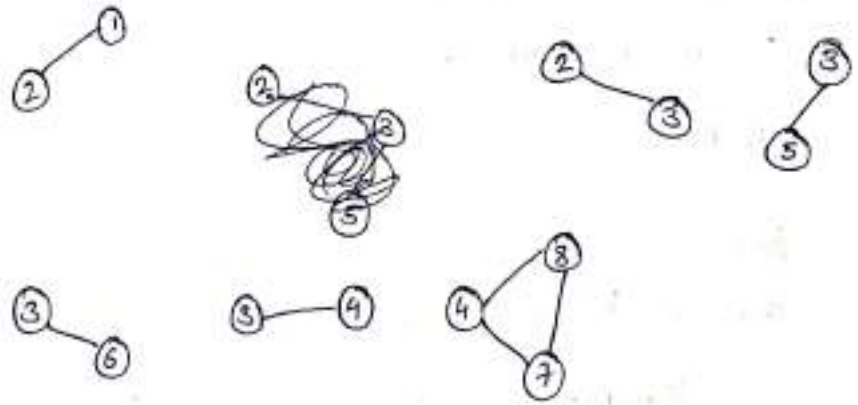
However min no of edges req to make the graph biconnected = 2

eg: Identify the articulation points and draw the biconnected components of graph and find min of edges to be added to make the graph biconnected.



Articulation points $\rightarrow 2, 3, 4$

Biconnected Components



$\therefore 6$ biconnected components

Min of no of edges to be added to make the graph biconnected = 3

i.e., Add ~~(1,5)~~ (2,5) (6,7) (1,4)

Bridge:

The edge whose removal makes the graph disconnected.

Q) Consider an undirected graph $G=(V,E)$ (unweighted). If BFS of G is done from a node 'x', let $d(x,u)$ and $d(x,v)$ be the lengths of shortest paths from x to u & v respectively. If ' u ' is visited before ' v ' during the traversal then which is true.

a) $d(x,u) \leq d(x,v)$

b) $d(x,u) > d(x,v)$

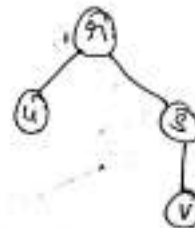
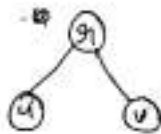
c) $d(x,u) \leq d(x,v)$

d) none.

Sol:

This can happen when

$$d(x,u) = d(x,v) \quad \text{or} \quad d(x,u) < d(x,v)$$



$\therefore$ opt c

Q) In a DF-traversal of a graph G with n -vertices, ' k ' edges are marked as 'tree edges'; the no. of connected components of ' G ' is _____

- a) k b) $k+1$ c) $n-k+1$ d) $n-k$

If a forest of n vertices have k edges, then
no. of trees (components) in forest = $n - k$.

Q. A DFS is performed on a DAG. $d(x)$ is discovery time and $f(x)$ is finishing time of a vertex x . which is always true for all edges (u, v) in the graph?

- a) $d[u] < d[v]$ b) $d[u] < f[v]$
c) $f[u] < f[v]$ d) $f[u] > f[v]$

Sol:



opt (a) & (b) fails if we start traversal from v .
A (c)

opt (d) will be successful in any case.

Sorting Techniques :

Sorting techniques are classified into ~~two~~ types: as

- (i) Stable vs unstable
- (ii) Internal vs external
- (iii) Inplace vs not inplace
- (iv) Comparison based vs non-comparison based.
- (v) Recursive vs non-recursive.

Comparison based:

The sorting algorithm that use comparison b/w elements is called comparison based sort.

Eg: Quick sort, heap sort etc.

Non-Comparison based:

The sorting technique that doesn't use comparison.

Eg: radix sort, counting sort, address calculation sort.

→ No comparison based sorting takes time less than ~~of any~~ $O(n \log n)$ for any best case.

Inplace vs Not-inplace:

Sorting technique is said to be inplace if its space complexity is $O(1)$. However for recursive sorting technique it can be $O(\log n)$ for recursion stack.

Otherwise it is said to be not inplace.

Eg: Inplace → Quick sort

not inplace → Merge sort

Internal vs External:

The sorting technique which puts a constraint that all elements must be present in memory before the sorting begins.

Where as external sorting technique allows partial no of elements to be present in memory or allows

new elements to be added to the list of the numbers to be sorted.

Eg: Insertion sort behaves like external sorting. But it is not completely external sorting.

Stable vs Unstable

A sorting method is said to be stable iff the relative order of non-distinct elements is maintained in the sorted list.

otherwise it is unstable.

Eg: Merge Sort $\rightarrow$ stable

Quick sort, heap sort $\rightarrow$ unstable.

Generally stable sorting techniques are more desirable.

Note:

$\rightarrow$ Time complexity of a comparison based sort depends on no of comparisons and no of swaps.

$\therefore$ time complexity = $O(\max\{\text{no of comparison, no of swaps}\})$

$\rightarrow$ No of swaps depends on inversions in the list to be sorted.

if $(i < j)$ and if $A[i] > A[j]$ then the pair (i, j) is known as inversion of the array.

~~Thus no of inversions forms a lower bound for no of swaps.~~

Bubble Sort

Algo BS (A,n)

// A[1...n]: array

{

 for pass $\leftarrow$ n down to 1

 {

 for i $\leftarrow$ 1 to (pass-1)

 {

 if (A[i] > A[i+1])

 swap(A[i], A[i+1]);

 }

 }

}

Eg: A: 85 65 25 15 40 5

1st pass:

i=1: 65 85 25 15 40 5

i=2: 65 25 85 15 40 5

i=3: 65 25 15 85 40 5

i=4: 65 25 15 40 85 5

i=5: 65 25 15 40 5 85

2nd pass:

i=1: 25 65 15 40 5 85

i=2: 25 15 65 40 5 85

i=3: 25 15 40 65 5 85

i=4: 25 15 40 5 65 85

3rd pass:

i=1: 15 25 40 5 65 85

i=2: 15 25 40 5 65 85

i=3: 15 25 5 40 65 85

4th pass:

i=1: 15 25 5 40 65 85

i=2: 15 5 25 40 65 85

5th pass:

i=1: 5 15 25 40 65 85

Note:

→ No of comparison in bubble sort = $\frac{n(n-1)}{2}$ (irrespective of data set)

→ No of swaps = no of inversions

∴ Time complexity: $O(n^2)$

→ Even if the elements are in increasing order, we still make $\frac{n(n-1)}{2}$ comparison.

To overcome this we keep track of whether swap has occurred in a pass or not. If no swap has occurred then the algorithm terminates. The below is the modified algorithm:

for pass ← n down to 1

{ flag = 0;

for i ← 1 to (pass - 1)

{ if $A[i] > A[i+1]$

{ swap $A[i], A[i+1]$;

flag = 1;

} if (flag == 0) break;

→ Now for this modified B.S

best case $TC = O(n)$

worst case $TC = O(n^2)$

The disadvantage is that it requires flag and this is overcome by insertion sort.

→ The recurrence relation for ~~the~~ ~~comparisons~~ TC of BS is given by

$$T(n) = T(n-1) + O(n)$$

$$\Rightarrow T(n) = O(n^2)$$

Selection Sort:

Algo SS (A, n)

{

for $i \leftarrow 1$ to $n-1$

{ min $\leftarrow i$

for $j \leftarrow (i+1)$ to n

{ if $(A[j] < A[\text{min}])$

min $\leftarrow j$

}

swap $(A[\text{min}], i)$

}

This algorithm selects i th smallest element in i th pass and places it in i th place. So we repeat this for $n-1$ times.

Ex A: 80 60 20 15 40 10

i=1: 10 60 20 15 40 80

i=2: 10 15 20 60 40 80

i=3: 10 15 20 40 60 80

i=4: 10 15 20 40 60 80

i=5: 10 15 20 40 60 80

Time Complexity:

Time = no of comp + no of swaps

$$= \frac{n(n-1)}{2} + (n-1)$$

$$T.C = O(n^2)$$

→ Selection ~~is the~~ sort requires least no of swaps compared to any other ~~sorting tech~~ comparison based sorting technique.

→ Rec. relation:

$$T(n) = T(n-1) + O(n)$$

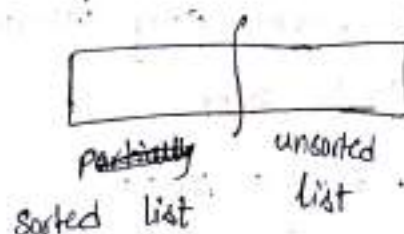
$$\Rightarrow T(n) = O(n^2)$$

Insertion Sort:

working; Insert one element at a time from an unsorted list to a partially sorted list.

→ It works in $(n-1)$ passes

→ At every pass, we have the below scenario



Take an element from unsorted list and put it in the sorted list in its position

Ex: $A: 4 \ 3 \ 8 \ 6 \ 9 \ 2$

initially size of sorted list = 1

$A: \langle 4 \mid 3 \ 8 \ 6 \ 9 \ 2 \rangle$

Now insert 3 into partially sorted list

pass 1 $\rightarrow A: \langle 3 \ 4 \mid 8 \ 6 \ 9 \ 2 \rangle$

pass 2 $\rightarrow A: \langle 3 \ 4 \ 8 \mid 6 \ 9 \ 2 \rangle$

pass 3 $\rightarrow A: \langle 3 \ 4 \ 6 \ 8 \mid 9 \ 2 \rangle$

pass 4 $\rightarrow A: \langle 3 \ 4 \ 6 \ 8 \ 9 \mid 2 \rangle$

pass 5 $\rightarrow \langle 2 \ 3 \ 4 \ 6 \ 8 \ 9 \rangle$

$A: \langle 2 \ 3 \ 4 \ 6 \ 8 \ 9 \rangle$

From this we say insertion sort works as external sorting.

Since at any point of time insertion sort doesn't both about where the unsorted list is stored.

Time Complexity.

(1) Best Case:

Best case happens when: every insertion finding with one comparison. i.e., Inc order

$$\begin{aligned} \Rightarrow \text{Time} &= \text{no of comp} + \text{no of swaps} \\ &= O(n-1) + 0 \end{aligned}$$

$$\Rightarrow T.C = O(n)$$

ii) Worst Case:

worst case happens when elements are in descending order.

$$\text{time} = \text{no of comp} + \text{no of swaps} = O(n^2)$$

→ In general, it is found that TC of insertion sort depends on no of inversions.

$$\Rightarrow TC = O(n+d)$$

where d is no of inversions.

Thus if $d=0$ (i.e., no inversions)

$$TC = O(n)$$

Ex: Consider a list with n inversions. Then, TC for insertion sort is _____?

Sol:

$$TC = O(n+d)$$

$$= O(n+n) = O(n)$$

25/10/20

Non-Comparison based sorting techniques

i) Radix Sort:

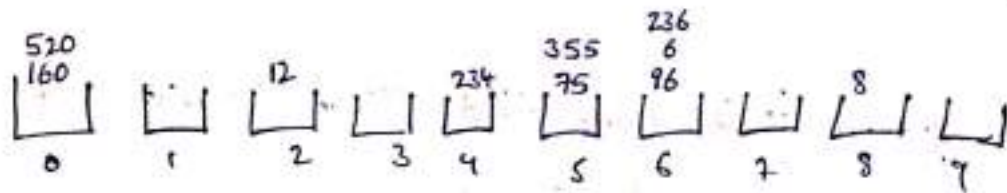
→ Here the term radix indicates base of the system.

→ If base is 'b' we take 'b' no of buckets.

Ex: A: 234 60 96 12 8 75 6 355 520 236

Pass 1:

Distribute the numbers into buckets based on unit digit.

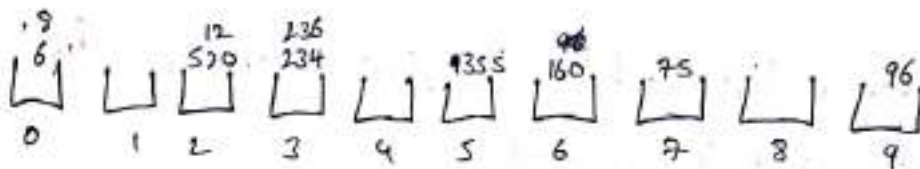


∴ Elements after pass 1

160 520 12 234 75 355 96 6 236 8

Pass 2:

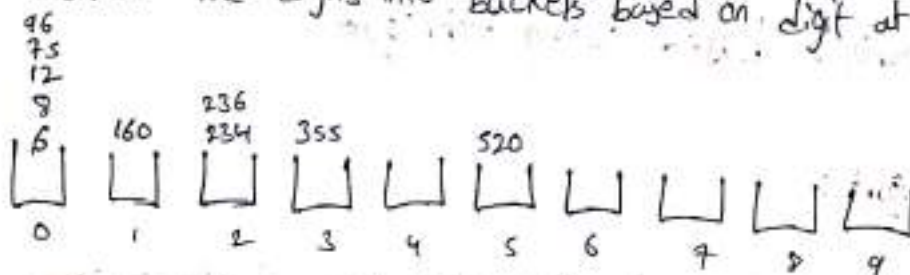
Distribute the digits into buckets based on digit at 10's place



∴ 0 6 8 520 12 234 236 355 160 75 96

Pass 3:

Distribute the digits into buckets based on digit at 100's place.



∴ 6 8 12 75 96 160 234 236 355 520

Drawback:

- Special implementation is needed if we need to sort negative numbers.
- Sorting fractional numbers is not possible.

Time complexity:

$$T.C = O(d(n+b))$$

→ since 'b' is const we can also write $T.C = O(dn)$

$d \rightarrow$ max digits in a number

$n \rightarrow$ size of list

$b \rightarrow$ base

If x is max number possible in the base then

$$d = \log_b x$$

$$T.C = O((n+b) \log_b x)$$

$$= O(n \log_b x)$$

$$\text{let } x \leq n^c$$

c is a constant

$$T.C = O(n \log_b n)$$

Thus radix sort performs good when $b \geq n$

$$\text{If } b \geq n$$

$$O(n \log_b n) = O(n)$$

H/82 Method 1:

we ~~heap~~ use heap for this (similar to problem H/64)

we have mn elements.

$$\text{size of heap} = \text{no of lists} = n$$

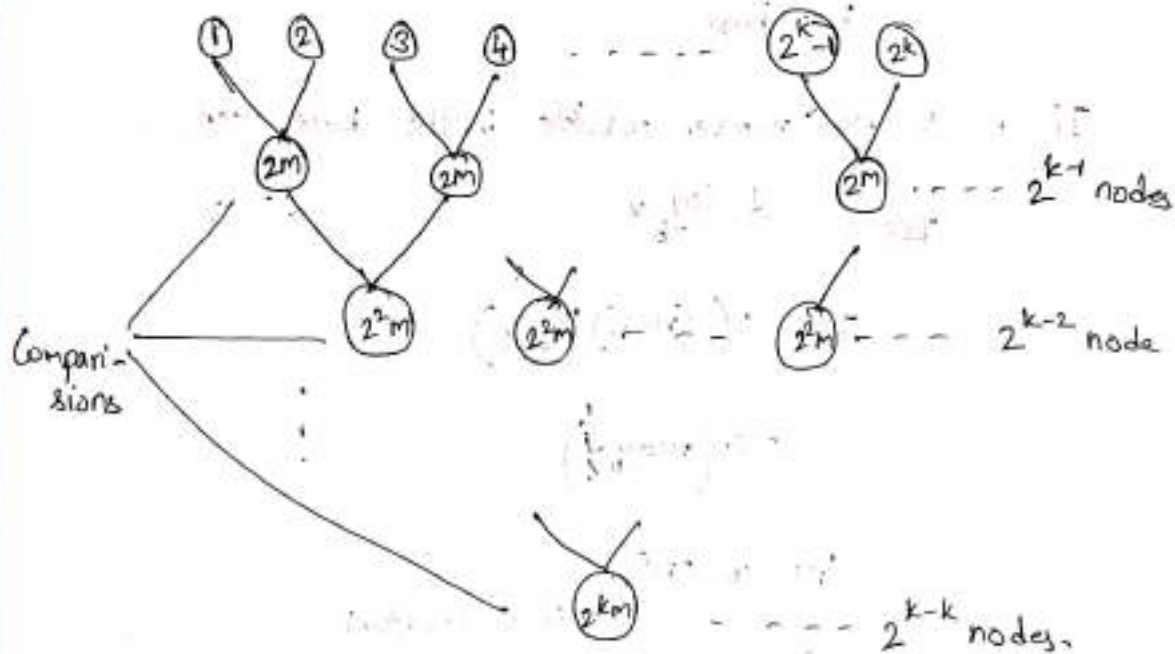
$$\therefore \text{del \& insert on heap} \rightarrow \log n$$

$$\therefore O(mn \log n)$$

Method 2:

This can also be solved by considering optimal merge pattern.

$$\text{let } n = 2^k$$



$\therefore$ total comparisons required

$$\begin{aligned} & (2^{k-1})(2^{k-1}) + (2^{k-2})(2^{k-2}) + \dots + (2^0)(2^0) \\ &= (2^{k-1} + 2^{k-2} + \dots + 2^0) \cdot k \\ &= n \log n \end{aligned}$$

(H/83)

let b be base.

$$\text{no of digits } d = \log_b n^k = k \log n$$

$$\text{TC of radix sort} = O(d(n+b))$$

$$= O(dn)$$

$$= O(n \cdot k \log n)$$

$$= O(n \log n)$$

H/88

131

If assume base is n

then TC of radix sort = $O(n)$

Disjoint Set Data structure

Operations:

(i) Create(): Create the set with required element

(ii) Union(): Combining the two sets into one.

(iii) Find(): returns the representative element of the set.

Every set has a representative element

Representation:

(i) forest:

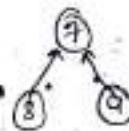
Eg: $S_1 = \{1, 2, 3\}$



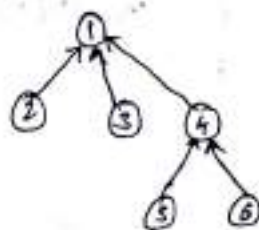
$S_2 = \{4, 5, 6\}$



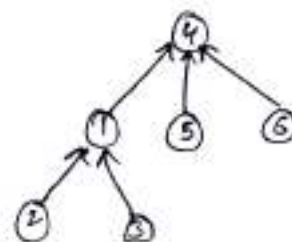
$S_3 = \{7, 8, 9\}$



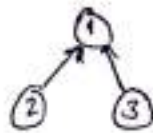
$S_1 \cup S_2$: This can be done by choosing one of ~~two~~ the representative element as new representative element for $S_1 \cup S_2$



(a)



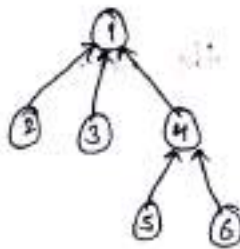
Find operation:



Here $\text{find}(1) = 1$
 $\text{find}(2) = 1$
 $\text{find}(3) = 1$



Here $\text{find}(4) = 4$
 $\text{find}(5) = 4$
 $\text{find}(6) = 4$



Here $\text{find}(1) = 1$
 $\text{find}(2) = 1$
 $\text{find}(3) = 1$
 $\text{find}(4) = 1$
 $\text{find}(5) = 1$
 $\text{find}(6) = 1$

TC of find depends on ~~height~~ height of the tree.

(ii) Array based Representation:

$S_1 = \{1, 2, 3\}$ $S_2 = \{4, 5, 6, 7\}$ $S_3 = \{8, 9\}$

Let 1, 4, 8 be corresponding representative elements.

Array:

	1	2	3	4	5	6	7	8	9
parent	-1	1	1	-1	4	4	4	-1	8
element	1	2	3	4	5	6	7	8	9

with amortized analysis
 it is found that TC
 of both union & find
 is $O(1)$

Kruskal's algorithm for MST

153

Algo Kruskal($E, \text{cost}, n, T, \text{mincost}$)

```
{
    // E is edge set in G. G has n vertices
    // COST[1..n, 1..n] is cost matrix
    // T is set of edges in MST

    1. real mincost COST(1..n, 1..n);
    2. integer Parent(1..n), T(1..n, 1..2), n;
    3. Construct a heap out of edge cost using heapify —  $O(e)$ 
    4. Parent  $\leftarrow -1$ 
    5.  $i \leftarrow \text{mincost} \leftarrow 0$ 
    6. while  $i < n-1$  and heap not empty —————  $O(e)$ 
        {
            a) delete a min cost edge  $(u, v)$  from the heap }  $\rightarrow O(\log e)$ 
               and reheapify using ADJUST
            b)  $j \leftarrow \text{FIND}(u); k \leftarrow \text{FIND}(v);$ 
            c) if  $j \neq k$  then  $i \leftarrow i+1$ 
                {
                     $T(i, 1) \leftarrow u; T(i, 2) \leftarrow v$ 
                     $\text{mincost} \leftarrow \text{mincost} + \text{cost}(u, v)$ 
                    call UNION( $j, k$ )
                }  $\rightarrow O(1)$ 
            endif
        }
    repeat
    7. if  $i \neq n-1$  then print ("no spanning tree") end if
    8. return;
}
```

$\therefore T.C = O(e \log e)$

S.C = $O(n)$ [i.e., space for parent array]

Algorithm to find connected components:

CONNECTED COMPONENTS(G)

{

1. For each vertex $v \in V(G)$

do MAKE-SET(v)

2. For each edge $(u,v) \in E(G)$

if find(u) $\neq$ find(v) then

union(u,v)

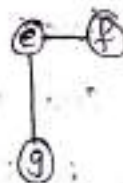
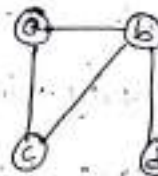
}

~~Eg:~~

Eg: Consider below graph & its tracing

Step 1:

~~{a} {b}~~



Step 1.

{a} {b} {c} {d} {e} {f} {g} {h} {i} {j}

Step 2.

edge (a,b)

{a,b} {c} {d} {e} {f} {g} {h} {i} {j}

edge (b,c)

{a,b,c} {d} {e} {f} {g} {h} {i} {j}

edge {b,d}

{a,b,c,d} {e} {f} {g} {h} {i} {j}

edge $e(a,c)$:

$$\text{find}(a) = \text{find}(c)$$

$\therefore$ no union will be performed

edge (e,f) & edge (e,g)

$$\{a,b,c,d\} \quad \{e,f,g\} \quad \{h\} \quad \{i\} \quad \{j\}$$

edge (h,i)

$$\{a,b,c,d\} \quad \{e,f,g\} \quad \{h,i\} \quad \{j\}$$

$\therefore$ 4 connected components.

TC = $O(E)$ [assuming that find & union can be done in $O(1)$]

Sum of Subsets (SOS):

→ Given a set of n -elements and another positive integer M .
It is required to determine whether there exists a subset of given numbers whose sum equals M .

→ TC of brute force — $O(2^n)$

$$\text{SOS}(n, M) = \text{True} \quad \text{if } M=0, n \geq 0$$

$$= \text{False} \quad \text{if } n \geq 0, M > 0$$

$$= [\text{SOS}(n-1, M) \parallel \text{SOS}(n-1, M - a[n])]$$

$\rightarrow$ only if $a[n] \leq M$

Tabulation Method for sos

$n=5$; $A = \langle 2, 8, 4, 11, 9 \rangle$; $M=6$

$S[0 \dots n, 0 \dots M]$

	0	1	2	3	4	5	6
0	T	F	F	F	F	F	F
1	T	F	T	F	F	F	F
2	T	F	T	F	F	F	F
3	T	F	T	F	T	F	T
4	T	F	T	F	T	F	T
5	T	F	T	F	T	F	T

→ soln
ie, $sos(5,6)$

$$\begin{aligned} sos[1,1] &= sos(0,1) \parallel sos(0,1) \\ &= F \end{aligned}$$

$$\begin{aligned} sos[1,2] &= sos(0,1) \parallel sos(0,0) \\ &= F \parallel T \\ &= T \\ \therefore sos[5,6] &= T \end{aligned}$$

Think how to backtrack
and obtain the
subset required

Note:

If $s(i,j)$ is true then $s(k,j)$ is true $\forall k \geq i$

Algo $sos(A, n, M)$

{

for $i \leftarrow 0$ to n

{ for $j \leftarrow 0$ to M

{

$sos(i,j) \leftarrow$ implement code for recursive eqn

}

}

}

TC: $O(nM)$

However if value of n is too large like 2^n (or) n^n then

TC will be $O(n \cdot 2^n)$ (or) $O(n \cdot n^n)$

and this case brute time complexity is better.

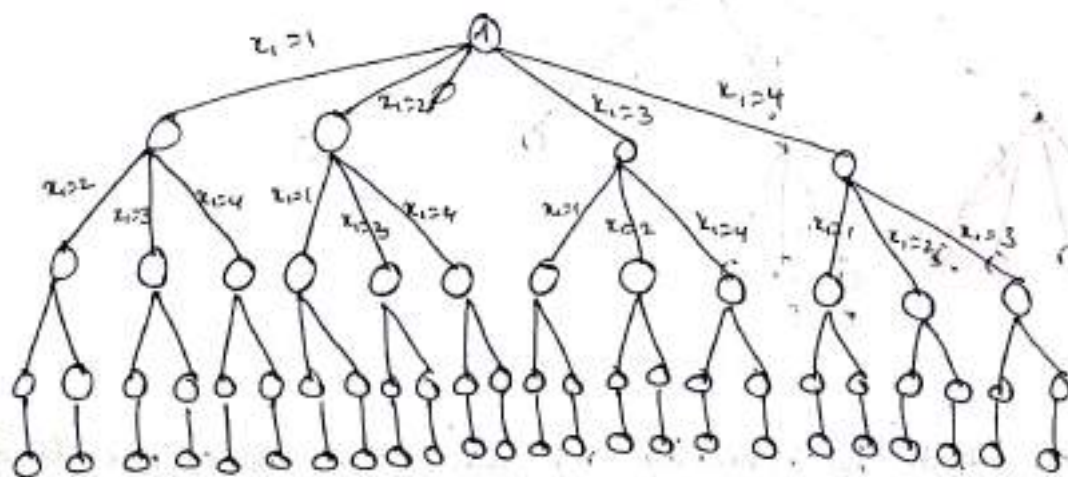
→ ~~See~~ The time complexities like $O(n \cdot M)$ are called pseudo-polynomial Tcs.

Introduction to Backtracking and Branch & Bound

Backtracking ^(or) ~~is an~~ algorithm design strategy that uses depth first searching ^{BFS} to find feasible/optimal soln in solution space (state space tree)

Eg: n-queens

Consider 4 queens problem

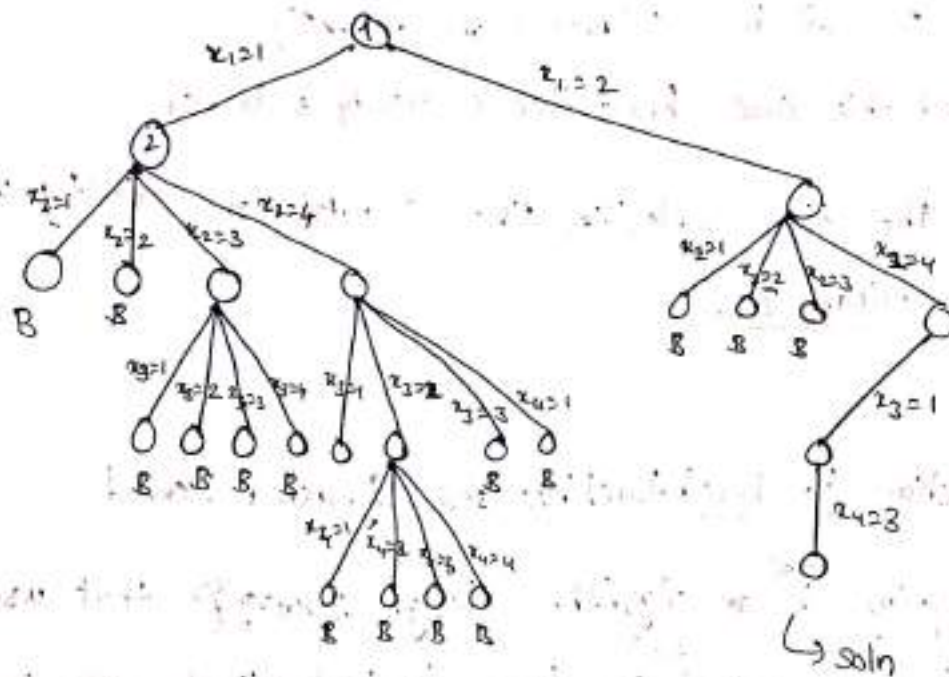


State Space tree

If DFS is used to search for soln in state space tree then it is called Backtracking.

If BFS is used then it is called Branch & Bound.

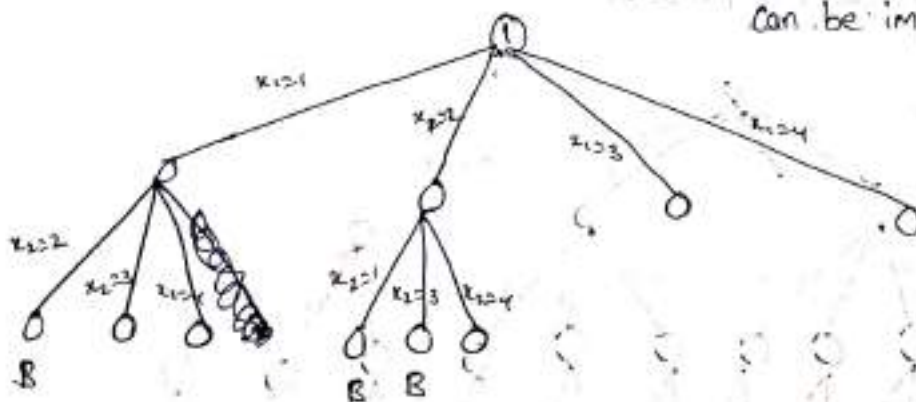
DFS: Backtracking



B → Bound

(soln is not feasible)

BFS: Branch & Bound



can be implemented with queue

→ It clear that 'backtracking' req less no of nodes explored compared to branch & bound.

Thus n-queens can be efficiently implemented using backtracking.

→ Similarly we can also solve backtracking in this approach.

→ Also TSP can be more efficiently solved using branch & bound.

Kadane's algorithm for max sum sub array

→ Bruteforce Tc : $O(n^2 \cdot n) = O(n^3)$

all subarray computing sum

→ DP approach (Kadane's algo)

~~we maintain 3 variable~~

we perform n iterations (n is no of elements)

we maintain 3 variables max-sum-so-far, current-sum, index.

~~In index says from which iteration we start summing~~

~~Thus n iterations correspond to index value from 1 to n .~~

Index says till which index max sum is obtained.

Algo kadane ()

{

// start_index, end_index // solun.

// MaxSum

for $i=1$ to n

{

temp_start = i ; max-sum-so-far = 0; curr-sum = 0; index = i ;

for $j=i$ to n

{

curr-sum += $a[j]$;

If (curr-sum > max-sum-so-far)

{

max-sum-so-far = curr-sum;

index = j ;

}

}

if (max-sum-so-far > MaxSum)

{

MaxSum = max-sum-so-far;

```

        start_index = i;
        end_index = index;
    }
}
return [start_index, end_index];
}

```

Te: $O(n^2)$

LIS:

- One approach is solving this using LCS.
- Let A be given array and store the sorted array of A in another array B.
- Now $LCS(A, B)$ will be the $LIS(A)$.
- Also we can use pure DP approach and solve this.